



INVESTIGATION REPORT

Creating a Highly Replayable Roguelike Game

By

GOH SHAO HANG

TP063859

APD3F2305CGD

A report submitted in partial fulfillment of the requirements for the degree of

B.Sc. (Hons) Computer Games Development

at Asia Pacific University of Technology and Innovation.

Supervised by Dr. LAI NGAN KUEN

2nd Marker: Mr. JACOB SOW TIAN YOU

9-August-2023

DECLARATION OF THESIS CONFIDENTIALITY

Author's full name: **GOH SHAO HANG**

IC No./Passport No.: **TP063859**

Thesis/Project title: **CREATING A HIGHLY REPLAYABLE ROGUELIKE GAME**

I declare that this thesis is classified as:

☑ CONFIDENTIAL

¨ RESTRICTED

¨ OPEN ACCESS

I acknowledged that Asia Pacific University of Technology & Innovation (APU) reserves the right as follows:

1. The thesis is the property of Asia Pacific University of Technology & Innovation (APU).
2. The Library of Asia Pacific University of Technology & Innovation (APU) has the right to make copies for the purpose of research only.
3. The Library has the right to make copies of the thesis for academic exchange.

Author's Signature: 

Date: 27 September 2015

Supervisor's Name: **DR. LAI NGAN KUEN**

Date: 27 September 2015

Signature: 

Library Form

First Name: Goh
Middle Name (only if applicable) : Shao
Last Name: Hang
Title of the Final Year Project / Dissertation / Thesis : Creating a Highly Replayable Roguelike Game
Abstract : Nowadays, developing captivating game content is becoming increasingly harder as the attention span of young video game players are getting shorter. This quickly becomes a problem as it takes a lot of effort and resources to develop video games, producing game assets and publish them, but developers might not be able to get what they expected in return due to this issue. To overcome this problem, procedural content generation is often used to enable the reusing of various assets. It allows for the creation of endless variation in game content to further increase replayability, which can help in increasing long term income and save up on expenses.
A few keywords associated with the work :

Procedural Content Generation (PCG), Algorithms, Replayabiltiy, Game Development, Game Design Techniques
General Subject: (e.g Management information systems, Organizational behaviour, Risk management, Computer Software) Computer Games
Date of Submission : 10/1/2023

Acknowledgement

First and foremost, the developer would like to express his heartfelt gratitude towards Asia Pacific University for providing the developer with the chance to work on the final year project titled “Creating a Highly Replayable Roguelike Game” and the profound assistance and guidance provided by lecturers and staffs in order to complete this project.

This gratitude is further extended towards the dedicated supervisor and second marker of this project, Dr. Lai Ngan Kuen and Mr. Jacob Sow Tian You. It is thanks to their kind and undivided support throughout the duration of this project that the project has been able to take shape and reach completion. Their invaluable feedback has been of immense help during the developing and eventual completion of the project.

Last but not least, the developer is extremely thankful towards every individual who willingly participated in this project, either by providing advices via the questionnaire or aiding in the research process in anyways, include performing user acceptance tests and providing verbal feedback. It is undoubted that all the help received are indispensable towards the current state of the project.

Abstract

Nowadays, developing captivating game content is becoming increasingly harder as the attention span of young video game players are getting shorter. This quickly becomes a problem as it takes a lot of effort and resources to develop video games, producing game assets and publish them, but developers might not be able to get what they expected in return due to this issue.

To overcome this problem, procedural content generation is often used to enable the reusing of various assets. It allows for the creation of endless variation in game content to further increase replayability, which can help in increasing long term income and save up on expenses.

Keywords – Procedural Content Generation (PCG), Algorithms, Replayabiltiy, Game Development, Game Design Techniques

Table of Contents

Library Form 3
Acknowledgement 5
Abstract 6
Table of Contents 7
Table of Figures 10
Chapter 1: Introduction to the Study 12
1.1 Introduction 12
1.2 Problem Background 13
1.3 Project Aim 17
1.4 Objectives 17
1.6 Potential Benefits 18
1.7 Overview of the IR 20
1.8 Project Plan 21
Chapter 2: Literature Review 24
2.1 Domain Research 24
2.2 Similar Systems 26
2.3 Technical Research 27
2.4 Summary 31
Chapter 3: Methodology 32
3.1 System Development Methodology 32
3.1.1 Agile Methodology 32
3.1.2 Waterfall Methodology 33
3.1.3 Justification 34
3.1.4 Description of System Development Methodology 35
3.2 Data Gathering Design 37
3.2.1 Identifying Research Method 37
3.2.2 Design 39
3.3 Analysis 47
3.3.1 Section A: Demographics 47
3.3.2 Section B: Gaming Habits and Preferences 50

3.3.3 Section C: About Game Replayability 56
3.3.4 Summary 69
Chapter 4: Design and Implementation 70
4.1 Introduction 70
4.2 Design 70
4.2.1 Level Generation Algorithm 70
4.2.2 Player Controller 77
4.2.3 Combat System 79
4.3 Interface Design 97
4.4 Execution 106
4.5 Screenshots 114
4.6 Summary 119
Chapter 5: Result and Discussion 120
5.1 Introduction 120
5.2 Testing Design / Plan 120
5.3 System Testing and Discussion 125
5.3.1 Unit Testing 125
5.3.2 User Acceptance Testing 130
5.4 Summary 140
Chapter 6: Conclusion 141
6.1 Critical Evaluation 141
6.2 Limitations 144
6.3 Recommendation 145
References 147
Appendices 149
Appendix A: PPF – Title Registration Proposal 149
Appendix B: Ethic Forms (Fast Track) 156
Appendix C: Log Sheets 166
Appendix D: Poster 172
Appendix E: Gantt Chart 173
Appendix F: Sample Code Implementation 174

Level Generation 174
Player Controller..... 184
Appendix G: Respondence Demographics Profile..... 196

Table of Figures

Figure 1 Enter the Gungeon, 2016 26
Figure 2 Slay The Spire, 2017 27
Figure 3 Agile methodology 33
Figure 4 Waterfall Methodology (LucidChart, 2022) 34
Figure 5 Example Prefabs 72
Figure 6 Example Visualization 73
Figure 7 Custom Rules for Room Spawner 73
Figure 8 Example rules set used in the first level of the game 74
Figure 9 Exit Shader 75
Figure 10 Level Generator Speed Test 77
Figure 11 PlayerCombatManager Inspector View 79
Figure 12 Weapon Data Scriptable Object 80
Figure 13 Weapon.cs 91
Figure 14 Timer Condition 93
Figure 15 Health Depleted Condition 94
Figure 16 Skeleton State Machine Flow 95
Figure 17 Skeleton enemy state machine 96
Figure 18 Startup Screen 97
Figure 19 Main Menu 98
Figure 20 ButtonScrollList.cs 99
Figure 21 CustomButton.cs 100
Figure 22 In Game HUD 101
Figure 23 Map (Partially Revealed) 102
Figure 24 Map (Fully Revealed) 103
Figure 25 Pause Screen 104
Figure 26 Death Screen 105
Figure 27 Entrance / Boss room 106
Figure 28 Single Exit Rooms 108
Figure 29 Two Exits (I-Shaped) Rooms 109
Figure 30 Two Exits (L-Shaped) Rooms 110
Figure 31 3 Exits (T-Shaped) Rooms 111
Figure 32 Four Exits Rooms 112
Figure 33 Treasure Rooms 113
Figure 34 Screenshots of the Games 118
Figure 35 Poster 172
Figure 36 Gantt Chart 173
Figure 37 LevelGenerator.cs 176
Figure 38 RoomSpawner.cs 180
Figure 39 LevelManager.cs 183

Figure 40 PlayerController.cs 190
Figure 41 PlayerMovementBaseState.cs 191
Figure 42 PlayerWalkState.cs 191
Figure 43 PlayerSprintState.cs 192
Figure 44 PlayerWallRunState.cs 193
Figure 45 PlayerCrouchState.cs 194
Figure 46 PlayerChargedAttackState.cs 195

Chapter 1: Introduction to the Study

1.1 Introduction

This project revolves around the development of a game project titled “Curse Breaker: Awakening”. It is a dark fantasy themed hack-and-slash roguelike game that will be built upon these design pillars:

- High replayability created via random design techniques, such as procedural level generation and randomized loot
- Fast-paced hack-and-slash combat system
- Easy to learn but hard to master movement system
- Hardcore gameplay that requires multiple attempts to sharpen reflexes and strategic resource management to achieve victory

Throughout this report, the tools and methods used to develop the game as well as considerations and decisions made during development will be outlined and explained, providing a clear vision of how a game of satisfactory quality can be developed by one person with enough planning and tactful decision making while maintaining the scope and budget in control, which a lot of indie game developers struggle with. In the end of the project, the developer hopes that it can provide a basic guideline for new developers who are not familiar with defining a viable scope for their project, and understand that anyone can make a game with enough dedication and careful planning.

1.2 Problem Background

In the ever-evolving landscape of video game development, it is becoming increasingly challenging for game developers to create a captivating gameplay experience all while maintaining various factors such as the scope, development lifetime and budget of the project. To develop a successful video game, a lot of considerations have to be made with utmost cautiousness.

- **Project scope and budget**

  The development of video games, especially for indie game developers, require careful control over the scope and budget of the project, as these factors can directly affect the final quality of the game and even bring impact on the game's sales.

  A lot of indie game developers suffers from keeping their scope in check, as they want to get their creative ideas into the game as much as possible to make the game more attractive. However, this can pose a lot of issues, such as endless extension of the scope and development lifetime of the project. In the end, the game will end up far from complete even though the developer(s) has dedicated years into the project or run out of budget.

  Developing a video game also requires the development and creation of various game assets and features. If the team is does not carefully monitor their expenses, it is totally possible for them to run out of budget before they manage to finish the project even if they did not expand the scope during development. It is also possible that as the development time increases, the assets developed diminishes in quality due to fluctuations in the teams' financial status, breaking cohesiveness of the art style and affecting player immersion.

  Aside from carefully monitoring the scope and budget, the roguelike genre offers a new perspective for the problem: the reuse of game assets and features. It's nature of requiring players to play through the same content repeatedly justifies the reusing of the same assets that the developers spend a lot of resources developing, making them no longer just disposable assets that will only be seen by the players once. If incorporated carefully, this aspect can save on a huge amount of budget and development time while producing maintaining the quality of the game.

- **Replayability and Player retention**

Most games aside from roguelike are one-time experiences, where developers spend a lot of effort enhancing the experience to make this experience as enjoyable and unforgettable as possible. While this ensures the quality of these games, it also come with several downsides. For example, some player might be looking for games that have longer playtime to kill time with, and will eliminate short but good games from their list. It is also difficult for these games to generate long term income, unless the developers actively push out additional content regularly such as free updates and downloadable content (DLC), but that requires further constant effort from the developers.

To overcome this problem, some developers go out of their way to introduce replayability into their games. It is important to note that while replayability is often seen in roguelike games, it is not limited to just roguelike games. For instance, story driven games often introduce branching paths, where the story will progress and eventually reach very different endings based on the players' choices. Players are encouraged to play the game multiple times and explore different choices in order to collect all endings to see the full story. In the case of roguelike games, the levels are randomly generated to maximize replayability, making each playthrough unique, therefore making the game highly replayable. Aside from making the game take longer to complete, having created a base system to support replayable gameplay can also be extremely beneficial for development post-game update. A good example of this would be Dead Cells (2017), where 4 paid DLCs are released after its initial release in just 6 years.

- **Game Predictability**

The premises of a game's attractiveness are usually built on intricate gameplay mechanics, which can directly affect player retention and interest. However, if the gameplay does not introduce incremental mechanics regularly over the course, players will become able to predict the upcoming content that they have not yet experienced and get demotivated to continue. It is hard to keep the experience fresh throughout the whole game, not to mention having to maintain the quality of the addition of each feature.

The roguelike genre solves this problem with a rather unique approach: randomization that removes almost all traces of predictability from the game. By procedurally generating levels, loot and challenges that the player will face, it is impossible for the players or even the developer to know what exactly will come up beforehand, forcing them to make the most out of whatever random resources they managed to secure in the current playthrough. The introduction of each random element can exponentially increase the unpredictability, creating a huge number of combinations of resources and challenges which becomes even more unpredictable due to different playstyles.

Recently, one particularly new genre has been garnering attention, which is the **roguelike** genre. Not only does it's hardcore and strategic gameplay appear captivating to many players, its nature of encouraging the reuse of assets and gameplay features is also why it is so popular among developers. Over time, it has developed into one of the most popular genres in the current game industry.

The term "roguelike" originates from the 1980 game "**Rogue**", which is a dungeon crawling video game developed by Micheal Toy and Glenn Wichman. (Bill Loguidice, 2009) It is known for its procedurally generated levels, permanent death mechanics, and hardcore gameplay. Back in 1980, this game has been a great hit among video game players, and its impact had been long-lasting enough to shape the game industry of modern times. As the game industry involves over time, roguelike genre has also been evolving a lot. Modern roguelike games usually blend together elements of strategic resource management, exploration mechanisms, and role-playing elements to further enhance the richness of the gameplay. The genre emphasizes also has a huge focus on replayability, where it is almost impossible to finish the game in one go. Player usually has to replay the game multiple times to complete it or even achieve their first win. However, each playthrough of the game offers unique challenges and encounters, creating a sense of unpredictability and excitement for the players every time they play the game and thus prevents the game from getting boring as the player go through the same levels all the time.

This project is focused on creating a similar game experience that has an emphasis on game replayability. Research will be conducted to find out what makes a game enjoyable even though they need to / are played through multiple times, how to make them more replayable, and last but

not least, what improvements and innovations can be made to further enhance the advantages of the genre and overcome the weaknesses.

1.3 Project Aim

To develop a fully playable demo for a 3D roguelike game using procedural level generation and game design techniques to without having to create a substantial amount of high-quality assets within a manageable scope and minimum budget.

1.4 Objectives

The objectives of the project are as follow:

- To create an immersive and enjoyable gameplay experience.
- To develop a content generation system that produces high gameplay variation while remaining highly customizable with high efficiency.
- To evaluate the importance of variety in assets and gameplay systems in terms of their contribution towards gameplay experience.

1.5 Scope

The scope of the project is as follow:

1. Development of the core gameplay systems, including player movement and player-environment interaction
2. Creation of a playable combat system that emphasizes in fast-paced movement-based combat, encouraging the player to utilize their movement abilities in battles
3. Development of a highly efficient procedural level generation system that can produce different results every time as well as introducing parameters for customization of the generated result, such as number of rooms and max connections allowed between rooms,
4. Building of individual rooms (levels) to be generated within the procedural level generation algorithm while taking considerations of utilizing the player movement system
5. Creation of other randomization systems such as loot randomization to further increase replayability of the game
6. Implementation of a navigation system that helps the player navigate through the generated levels that is flexible and applicable to any generation result

7. Development of highly customizable AI-driven enemies that utilizes the state pattern to implement varying behaviors to impose a challenge for the players
8. Creation of a UI system that allows creating various interfaces, as well as implementation of actual user interfaces such as main menu, pause menu and death screen
9. Completing the game loop by implementing a start and win condition and with a total gameplay length of 15 – 30 minutes, ensuring that the project qualifies as a full-fledged game demo
10. Development of a flexible audio system and integration of audio on important game aspects, including background music, in-game sounds such as player attack, and also menu sounds when navigating through various menus in the game
11. Partial polishing of the game to make it more attractive as a demo, which includes adding visual effects, using a tweening library (DOTween) to animate UI and adding extra sound effects

While the developer will be responsible for fulfilling all the tasks above, some assets and resources used to complete the tasks above will be created by the developer, while others shall be obtained from legitimate sources on the internet and given proper credit by the developer.

1.6 Potential Benefits

This project aims to benefit both video game players and game developers, especially indie game developers in a number of ways.

1.6.1 Tangible Benefits

- **Enhancement of technical skills:**
  Developing this project involves studying and implementing difficult programming challenges such as procedural content generation, AI algorithms, and other advanced game development techniques, enhancing your technical skill set.
- **Industry Recognition:**
  By developing this project, the developer hopes to prove that it is totally possible for someone to complete a high-quality game project with limited resources just by having enough dedication and being passionate towards video games.

- **Monetization and Collaboration Potential:**
  Assuming that the project is successful, it is possible for future collaboration with other passionate individuals to continue working on the game even after the submission of the project and potentially release the game as a full-fledged product in the market.
- **Sharing of technical knowledge:**
  By sharing this project to other game developers, this project will prove that it is entirely possible for indie game developers with a limited budget is able to create quality game content by applying techniques such as procedural generation systems and algorithms.

1.6.2 Intangible Benefits

- **Development of Knowledge:**
  Throughout the development of the project, the developer will be able to acquire more knowledge about the various tools used in game development, including game engines, programming languages, rendering systems and level design techniques and good practices
- **Creative Expression:**
  This project also serves as a platform for creative expression of the developer, allowing the showcase of not only technical skills but also creative decisions and thought process behind the making of the game, including story writing, visual and audio effects design and level design.
- **Producing an Enjoyable Experience:**
  This projects also aims to provide a fun and memorable experience for players, creating emotional value as well as help relieving stress from players' daily lives by offering entertainment.

1.6.3 Target Users

The target users of this project are:

1. Video game players who are seeking a challenging and enjoyable game experience
2. Fans of the roguelike game genre
3. Game developers looking to improve their design prowess and technical knowledge, as well as ways to utilize generation algorithms to ease their workload.

1.7 Overview of the IR

The investigation report is responsible for highlighting the research and investigations conducted in order to develop the game project. This includes studying other similar game products in the market that are successful, researching the best tools and development techniques to be used for development and improving technical knowledge required to complete the project.

The first section of the investigation report is a simple introduction to the study, explaining the background, motives and objectives behind the studies as well as outlining the various aspects that will be studied upon and researched such as technical / non-technical knowledge as well as the actions that will be taken to achieve fruitful research.

The second section of the investigation report features a literature review section, where the existing problems and their respective solutions will be studied. Various aspects regarding game replayability or game development in general will be discussed, including challenges in creating similar systems as well as preexisting techniques that can be used to develop the project and the advantages of applying them.

On the other hand, the third section of the report dives into technical research conducted for the project, including the though process behind selecting the proper tools for developing the project, including the selection of game engine. This section will also explain how various technical aspects will be implemented and the reasons behind those decisions, such as writing the story and characters, the design process of the levels and UI and implementation of game systems such as artificial intelligence (AI) for enemies, target platforms, data persistency and most importantly, how procedural level generation will be implemented.

The fourth section further explains the system development methodology used throughout the project to maximize productivity and prevent hiccups in development. The reason behind the choice of the respective methodology will be explained and how they will be integrated into the development process. The section also includes diagrams to further clarify the usage of the methodology applied.

Section 5 highlights the research conducted by the developer in order to shape the development process of the project according to external input. Various research method will be identified and

lastly selected, with justification provided on the choice. The designing of implementation of said research method will also be included.

Last but not least, section 6 records the validation of requirements, where the data gathered via previously mentioned research method will be analyzed. A summary will be provided to outline the results of performing said analyzation and how the analyzed data will ultimately affect the final outcome of the project.

1.8 Project Plan

Task Name	Duration (days)	Start date	End date	Status
Project Proposal Form (PPF)	7	28/5/2023	4/6/2023	Completed
Chapter 1: Introduction				
Problem Background	2	5/11/2023	6/11/2023	Completed
Project Aim	1	6/11/2023	6/11/2023	Completed
Objectives	1	6/11/2023	6/11/2023	Completed
Scope	1	6/11/2023	6/11/2023	Completed
Potential Benefits	1	8/11/2023	9/11/2023	Completed
Overview of the IR	3	10/11/2023	12/11/2023	Completed
Project Plan	1	7/1/2024	5/1/2024	Completed
Chapter 2: Literature Review				
Domain Research	7	14/11/2023	20/11/2023	Completed

Similar Systems / Works	3	21/11/2023	23/11/2023	Completed
Technical Research	5	24/11/2023	28/11/2023	Completed
Chapter 3: Methodology				
System Development Methodology	3	1/12/2023	3/12/2023	Completed
Data Gathering Design	7	4/12/2023	10/12/2023	Completed
Analysis	2	11/12/2023	12/12/2023	Completed
Chapter 4: Design and Implementation				
Introduction	1	13/12/2023	13/12/2023	Completed
System Design	7	14/12/2023	20/12/2023	Completed
Interface Design	2	21/12/2023	22/12/2023	Completed
Execution	2	22/12/2023	23/12/2023	Completed
Screenshot	2	23/12/2023	24/12/2023	Completed
Summary	1	25/12/2023	25/12/2023	Completed
Chapter 5: Result and Discussion				
Introduction	1	25/12/2023	25/12/2023	Completed
Testing Design / Plan	3	25/12/2023	27/12/2023	Completed

System Testing and Discussion	2	27/12/2023	28/12/2023	Completed
Summary	1	29/12/2023	29/12/2023	Completed
Chapter 6: Conclusion				
Critical Evaluation	1	2/1/2024	2/1/2024	Completed
Limitations	1	3/1/2024	3/1/2024	Completed
Recommendations	1	4/1/2024	4/1/2024	Completed

Chapter 2: Literature Review

This chapter focuses on performing literature review on existing works, studying any preexisting knowledge, problems and their respective solutions regarding procedural level generation and game development in general, as well as diving into both the technical and non-technical knowledge involved in the creation of these games. A deep analysis will also be performed on the domain of roguelike games, studying how similar games in the market do their job well and what it takes to develop such a successful video game.

2.1 Domain Research

2.1.1 Game Replayability

Nowadays, game developers are required to constantly producing high-quality content due to the short attention span of young video game players. The introduction of game replayability to video games is one among many other ways to solve this problem, which propose an elegant approach to overcome this issue by providing the player with new content each time they play through the game, keeping them engaged and excited.

There are a few ways to achieve replayability, such as procedural content generation which includes the generation of levels, rewards and various other game content, as well as other methods such as branching storyline that requires the player to replay the game in order to collect all endings and see the full story for themselves.

Procedural generation is a technique used that can automatically generate data by computer, often achieved in the form of computer algorithms. (Zhenyuan Shen, 2022). Popular procedural generation methods include wave function collapse (WFC), depth first search, binary-space partitioning and many more. Procedural generation of game content can be applied to aid in the development video games, more specifically by allowing the game developer to generate game content during runtime. The generation of content can also come in different forms to serve widely different purpose, such as terrain generation which generates a 3D mesh during runtime, dungeon generation which focuses on generating rooms and their respective connections to create a level, or something as simple as random loot drops. While there are also many variations of dungeon

generation algorithms, game developers usually prefer variations that they have more control over, whether to implement design choices or balance the gameplay difficulty. One of the great examples is Enter The Gungeon (2016), which utilizes a graph-based dungeon generation algorithm to create unique levels every time players enter the game.

2.1.2 Roguelike Games

While many genres utilize the power of procedural content generation, roguelike genre stands as the predominant genre that uses procedural content generation. Greg Kasavin, the creative director of the renowned indie game studio Supergiant Games, defines a roguelike game as a game structured around failing and restarting, and no two playthroughs of the game are exactly alike. (Wirecutter, 2023) In order to complete the game, the player will have to replay the game multiple times, and in this process, they will experience varying experiences and challenges and therefore have to come up with different strategies in order to progress. Even though the players are in truth playing the same level over and over again, this fact is hidden by the introduction of randomization, creating freshness and new challenges, making each playthrough unique in its own way.

One of the many key components that make roguelike games so captivating is the concept of risk and reward. Roguelike games are known for having high difficulty, forcing the player to practice caution and strategically manage whatever resource they secure in the randomized run. Any mistake can easily lead to the failure of the entire run, which they might have spent more than just a few hours on. This makes the progression all the more rewarding, incentivizing the player when they perform well.

However, there have also been more casual-oriented roguelike games lately, such as Cult of the Lamb (2022). While the game still requires multiple playthroughs to complete, it appears to be more forgiving on the players' actions compared to its competitors.

2.2 Similar Systems



Figure 1 Enter the Gungeon, 2016

Enter The Gungeon, a top-down roguelike shooter is one of the most popular works among the genre. It features a diverse cast of different characters as well as a huge arsenal of weapons and items that the player can utilize in each playthrough. The game features a complex procedural content generation system, which involves randomizing not only the layout of game levels, but also the loot that the player can obtain under various circumstances. Defeating the same boss enemy can drop anything from a useless item that does nothing to a powerful weapon that can easily destroy most enemies, making the game a lot more unpredictable and fun.

For the levels, the game uses a graph-based level generation algorithm, where the game will choose between a few graph templates, and then generate random pre-designed rooms according to the requirements specified by the graph. Special rooms can also be randomly injected into the graph to create even more variation, ensuring that the player can never expect encountering the same level layout twice while, while also giving them a lot of control to make custom designs to each individual room.



Figure 2 Slay The Spire, 2017

Slay the Spire is a roguelike deck-building card game released in 2017. It is worth mentioning that although it utilizes procedural level generation, it does not generate levels in the traditional way. The "levels" in Slay the Spire are, in essence, just a graph of nodes connected with lines. The game draws a graph for each level and the player is given the choice to advance on a linear path on the graph, gradually advancing towards the final node of the level which features a boss fight. The graph contains randomly generated nodes that trigger various interesting events, such as combat encounters, special events or treasures. The loot gained by completing events are also randomized, as well as random events that can affect the player both positively and negatively, forcing players to carefully evaluate the risk of each and every action.

2.3 Technical Research

This section explains the tools, software and hardware that has been considered by the developer to use during development of the project, the final selection as well as the justification behind the choices made.

2.3.1 Game Engine

Game engine is the core of creating and running a game efficiently. While a game engine is not absolutely required to create a game, it can provide a great boost in workflow and production speed by eliminating the need of manually handling graphics rendering, light calculations and many more intricacies within a video game.

While the developer has considered two viable options, which are **Unity** and **Godot**, Unity emerged as the ultimate choice for the creation of Curse Breaker: Awakening.

Unity is a cross-platform game engine developed by Unity Technologies and has been released since 2005 June. (H. Afzal et al., 2020). It is a powerful and versatile game engine widely used by various companies, including both independent game developers and large companies to create video games or even other applications, such as educational programs and VR applications. It is also known for being one of the most accessible and beginner friendly game engines.

Features

1. **Visual Editor:** Unity utilizes a visual editor to convert a large portion of the game development workflow from entirely within a code editor / integrated development environment (IDE) into a clean, easy-to-read graphical interface. Using the Unity Editor, developers are able to directly place assets and implement some logic and see immediate change in the scene view, which is a viewport that displays the game environment with a camera that the developer can move and rotate around to easily access the game world.

2. **Graphics Rendering / Lighting:** Unity handles 3D graphics rendering for the players out of the box, and offers a lot of lighting options. It supports both realistic, physically based rendering techniques, as well as providing the option to utilize shaders for custom rendering styles. For example, developers can use Unity's shader graph to create stylized shaders that make their game appear cartoony instead of realistic to fit their game.

3. **Physics Support:** Unity also takes care of various difficult game logic for developers with its outstanding physics support. Unity has built in physics support for both 2D (Box2D physics engine) and 3D (Nvidia PhysX engine). This includes handling collision, gravity, real-life object simulation and more.

4. **Cross Platform Support:** Unity allows developers to create games that can run on a wide range of platforms, including Windows, macOS, Linux, IOS, Android and many more. Such versatility enables the developers to target audiences across multiple platforms without having to make huge changes to their codebase.

5. **Asset Store:** Unity has their official asset store, allowing developers to source high quality assets with ease. Unity also makes use of the package manager pipeline to make importing assets from the asset store into the game engine very easy.

Besides the features listed above, Unity also provides a lot of other interesting features that helps smoothing out the game development process, such as a built-in animation system and online gaming services (in-game advertisements, dedicated game servers and more).

Programming Language

While there are plugins that allow using other languages to script in Unity, the default programming language used in Unity is C#, a high-level object-oriented language that has a lot of features such as automatic memory management (via garbage collection), and is thus the primary selection for the developer to develop the game. C# utilizes a syntax similar to Java which is known for being beginner-friendly due to being more human-readable. C# also comes with some standard libraries that allows developer to work with string, collections, reading and writing files more easily.

2.3.2 Justification

After evaluating both of the options thoroughly and with much consideration, Unity is deemed to be the more fitting game engine for this project. This decision is made after a comprehensive assessment of the pros and cons of both Unity and Godot, taking into account all the goals and requirements of the project to maximize the efficiency in creating the game product.

Unity's extensive set of features as well as it’s community support plays a significant role in helping the decision-making process. It offers a lot of versatility along with its great 3D rendering and physics support, which will be very useful in making the actual game. Unity’s shader graph is also a great tool for making the game’s visuals appeal to players. Developers can also easily reach out to the community for help should there be any specific problems that they are stuck with.

Moreover, Unity’s official asset store offers a lot of free assets that can be easily imported into the project for use. This includes art assets such as 3D models, user interface graphics, music and sound effects, as well as other useful tools such as premade shaders.

All in all, Unity’s versatility, accessibility and robust set of features make it the better fit for the scope of this project and its goals, which will be of great help in the process of creating an enjoyable and replayable game experience for the players.

2.4 Summary

In this chapter, various existing materials regarding game replayability and roguelike games are studied. Preexisting theories and solutions to achieve replayability has been discussed as well as techniques that can be applied in this project to further enhance both the enjoyment and replayability of the game.

While many games utilize procedural content generation, each of them implements the generation algorithm differently to suit their own game and achieve widely different purposes. For instance, Minecraft (2011) is a sandbox game that utilizes procedural terrain generation for the player to walk on, producing different but massive and beautiful worlds. It is of utmost importance to keep in mind that procedural content generation for games can be modified and expanded upon to fit any kind of project and needs, and is in no way a rigid rule that has to be strictly followed in order to achieve replayability.

Aside from generating game levels, procedural generation can also be applied to many other aspects of a video game to further enhance its replayability, such as randomizing the rewards obtained by the player or the placement of enemies to create element of surprise. This is essential in keeping the player motivated to repeat the same task over and over again without diminishing in interest.

In conclusion, this literature review section has been of great help in obtaining professional knowledge regarding the roguelike genre as well as helping the developer understand the common problems faced in procedural content generation as well as their respective solutions, which will contribute towards creating a better end result by the end of this project.

Chapter 3: Methodology

3.1 System Development Methodology

To ensure the smooth development of the game project, a variety of system development methodologies has been explored and considered in search for the best fit for the project. In order to fulfill the project's goals as well as manage and allocate resources as efficiently as possible, it is crucial to choose the right approach.

Out of all the options, the Agile model and Waterfall model stood out as great choices as they both provide unique benefits and fit this project's setting in different ways. In order to properly compare and assess Agile and Waterfall approaches, this chapter lays the foundation for the future justification of the selected technique for the project's advancement.

3.1.1 Agile Methodology

The Agile methodology is a flexible and iterative methodology, which divides development into manageable chunks known as sprints. This makes it one of the most popular options for dynamic projects such as game development projects. Given how complicated the process of game development can be, the ability to quickly respond to evolving requirements and gather continuous feedback in order to make enhancements incrementally makes the Agile methodology stand out even more. In addition, it also emphasizes close collaboration between the developer and the target users, promoting continuous feedback so that developers can perform adjustments based on real-time feedback. The advantages of Agile Methodology lie in its ability to accommodate changing design ideas as it mostly focuses on completing sprints, which are small but functional increments to the project.

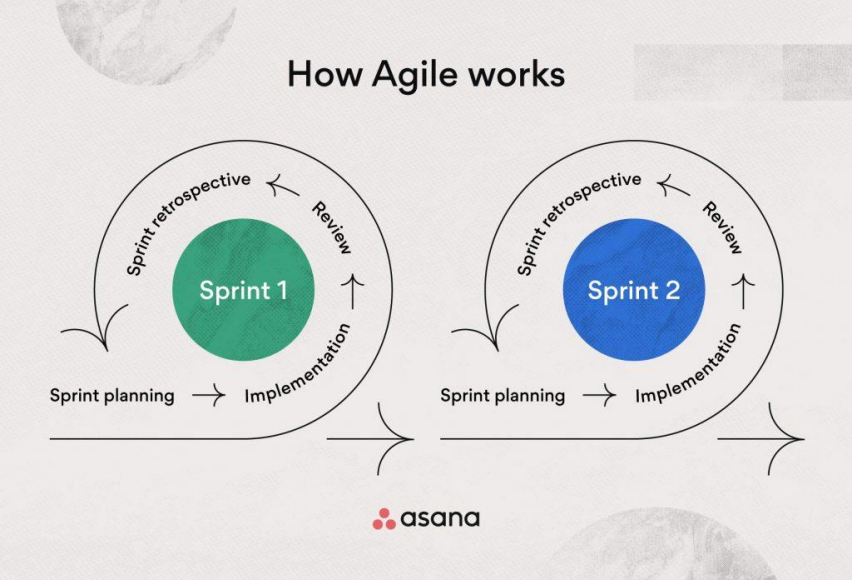


Figure 3 Agile methodology

3.1.2 Waterfall Methodology

The Waterfall methodology follows a linear progression through a fixed number of phases, which are requirement analysis, design, implementation, verification, and maintenance. It is known for its structured and comprehensive planning upfront, making it stand out for projects that already has well-defined requirements and stable objectives. By providing clear milestones and documentation at each stage, this makes it suitable for projects that requires specific regulation actions or contractual requirements.

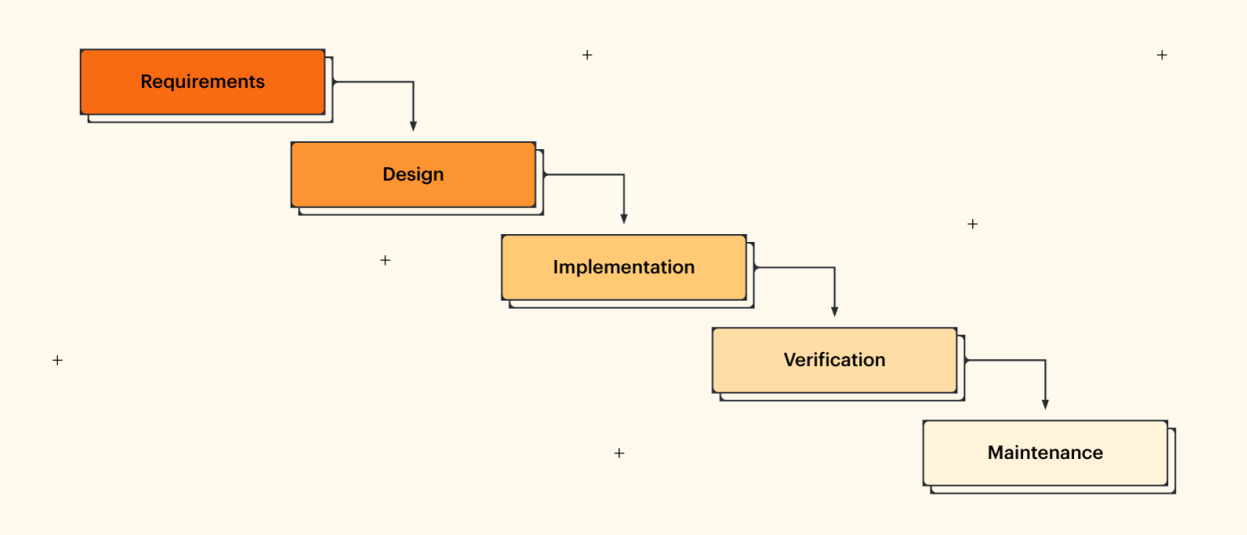


Figure 4 Waterfall Methodology (LucidChart, 2022)

3.1.3 Justification

Given the dynamic and creative nature of solo game development, the Agile methodology appears to be the most fitting approach for Curse Breaker: Awakening. Creating a game project involves a fluid design process that requires a lot of responsiveness to evolving design ideas and the ability to iterate on features incrementally. The Agile methodology's iterative cycles, emphasis on continuous feedback, and focus on delivering value align seamlessly with these development requirements, especially since this project is being worked on solo.

While Waterfall's structured approach has its merits in certain contexts, its linear progression may hinder the project's responsiveness to evolving gameplay elements and design changes. The project's goal of crafting an engaging and ever-evolving game experience necessitates a methodology that embraces change and iteration, making Agile the ideal choice. The subsequent sections elaborate on how the Agile methodology will be implemented, its associated diagrams, and an overview of the project's iterative phases, all contributing to the creation of a captivating and engaging roguelike game experience.

3.1.4 Description of System Development Methodology

For the development of Curse Breaker: Awakening, an Agile methodology will be adopted, with a focus on an iterative and incremental approach.

The project will be broken down into several **sprints**, each lasting approximately two to three weeks. During each sprint, specific tasks and goals will be identified, prioritized, and implemented. This iterative process allows for continuous feedback and adaptation, enabling flexibility to accommodate changes and improvements.

In the planning stage of each sprint, the core objectives and features for that iteration will be outlined. This ensures that development efforts are focused on delivering incremental value to the game. Design and development will occur in subsequent sprints, with an emphasis on core functionalities and features. This approach aligns with the Agile methodology's emphasis on incremental development and customer-centricity.

The implementation of the Agile methodology will incorporate the principles of iterative development, collaboration, and regular feedback. This approach not only allows for the integration of player feedback to refine the game's mechanics but also promotes a dynamic and adaptive development process that can respond effectively to evolving ideas and requirements.

By adopting an Agile methodology, a balance between structure and creativity is maintained, enabling the project to evolve in according to player needs and preferences. This iterative and flexible approach will contribute to the development of a game product that engages players, offers continuous improvement, and delivers an enjoyable gameplay experience.

Below are the development cycles (sprints) that the project is expected to go through, but are subject to future changes.

Sprint 1: Conceptualization and Planning

- Define the project goals, scope, and target audience.
- Create a high-level overview of the game's narrative and mechanics.
- Identify key features and functionalities that will be included in the game.
- Identify the tools needed to develop the project.

- Build an asset library with external assets.

Sprint 2: Narrative Design and Preparation

- Develop a basic storyline for the game.
- Outline the game world's lore and characters.
- Create mockups for UI design.
- Finish the design of the level generation algorithm.

Sprint 3: Prototyping

- Implement the basic movement and controls of the player character.
- Create a testing environment using placeholder assets.
- Develop the core combat mechanics and enemy AI.
- Create a prototype of the level generation algorithm.

Sprint 4: Playtesting and Refinement

- Conduct thorough playtesting of the game's prototype level.
- Gather feedback from play testers on gameplay, narrative, and overall experience.
- Identify areas of improvement and implement refinements.

Sprint 5: Unique Features Integration

- Start integrating the game's narrative.
- Implement character interactions such as receiving blessings and buying items.
- Implement secondary mechanics that makes the game unique.

Sprint 7: Game Polishing

- Add post-processing to the game scene to make the visuals more appealing.
- Use visual effects to increase player feedback for various game events.
- Implement audio into the game.
- Implement other changes and bug fixes based on playtesting feedback.

Sprint 8: Optimization and Release Preparation

- Address and fix any bugs, glitches, or technical issues identified.
- Optimize performance to ensure smooth gameplay on the targeted platforms (Windows).
- Set up a proper store page for the game (Itch.io), create simple marketing materials such as posters.

3.2 Data Gathering Design

3.2.1 Identifying Research Method

This chapter focuses on gathering data that will help improve the development process of the game project as well as increase the quality of the result, as well as explain the techniques used to analyze the collected data to extract useful information.

A few research methods are considered to be used for this project. One of them is using a questionnaire to gather public input. A questionnaire can be easily distributed to the public to receive public feedback, allowing the developer to understand their target audience, gaming habits and preferences, thoughts on game replayability and more.

The other possible option is by conducting an interview with one or more professional in the game industry. This is a great way to receive high-quality advices from an industry professional, but the feedback received might be subjective due to only having less participants compared to other research methods such as questionnaire.

After through consideration, the developer decided to use the **questionnaire** method for this project. This is because given the context of this project, which aims to develop a user-centric game product, it is crucial to gather insights directly from potential users. The chosen research methods should able to effectively collect data such as user preferences, feedback, and player expectations to improve the development process effectively.

The questionnaire is also a valuable data collection method because of its versatility and effectiveness in gathering both qualitative and quantitative data from a wider audience. It provides a structured approach towards collecting standardized data, allowing the comparison and analysis

of responses. The use of closed-ended and open-ended questions facilitates the collection of quantitative data, such as user preferences and ratings, as well as qualitative data, including suggestions and feedback.

3.2.2 Design

The questionnaire is designed to understand the target audiences of the game as well as receiving their thoughts on how to enhance game replayability. It is divided into 3 sections: **Demographics**, **Gaming Habits and Preferences** and lastly, Game **Replayability**.

Below is the overview of the entire questionnaire.

Section A: Demographics

1. Age
    - 18 – 22
    - 23 – 27
    - 28 – 32
    - 33 – 37
    - 38 - 42
    - 42 – 47
    - 48 or above

2. Gender
    - Male
    - Female
    - Other / Prefer not to disclose

3. Occupation
    - Student
    - Employed
    - Self Employed
    - Freelancer
    - Unemployed
    - Retired
    - Other

4. Educational Background
    - Primary
    - Secondary
    - High School
    - Bachelor’s Degree
    - Master’s Degree
    - Other

Section B: Gaming Habits and Preferences

This section aims to understand the users' gaming habits and preferences, which plays a crucial role in helping the developer understand public preferences and gaming activities.

1. On average, how many hours do you play video games every day?
   - Below 1 hour
   - 1 – 2 hours
   - 2 – 3 hours
   - 3 – 4 hours
   - 4 – 5 hours
   - 5 hours and above

2. What is your favorite video game genre?
   - Action
   - Role-playing
   - First Person / Third Person Shooter
   - Adventure
   - Puzzle
   - Strategy
   - Simulation
   - Sports
   - Horror
   - Others

3. Do you find yourself playing more casual games (low difficulty, short game sessions) or hardcore games (high difficulty, takes more time to complete)
   - Casual Games
   - Hardcore Games

4. Do you prefer games that promotes strategic thinking (turn-based, puzzle etc.), or more action-oriented mechanics that test your reflexes (action, horror)?
    - Strategic Thinking
    - Action-Oriented

5. Do you find yourself more inclined towards indie games (games developed by independent companies, often with smaller-scale productions) or 3A games (games with top-tier graphics and high production values created by larger, well-established companies)?
    - Indie Games
    - 3A Games

6. On average, how many games do you finish regardless of completion rate every month (eg. only getting one out of many endings)?
    - None
    - 1 – 2
    - 2 – 3
    - 3 – 5
    - More than 5

7. How many games have you completed with 100% completion (eg. Getting all endings in a game, unlocking all unlockables)?
    - None
    - 1 – 2
    - 2 – 3
    - 3 – 5
    - More than 5

8. How important are the following factors in determining your enjoyment of a video game?
    - Graphics and Visuals
    - Story and Narrative
    - Gameplay Mechanics
    - Character Customization
    - Replayability
    - Difficulty Level
    - Other

Section C: About Game Replayability

Game replayability refers to the extent and appeal of a video game's ability to be played multiple times without losing its entertainment value. It is a measure of how engaging and enjoyable a game remains even after the player has completed it or experienced its content several times. A highly replayable game offers fresh and varied experiences with each playthrough, encouraging players to return to the game for additional sessions. This is achieved through various design elements and features.

1. How often do you replay a video game you have completed before?
   1 (Never) – 5 (Very Often)

2. What do you think motivates you to replay a game multiple times?
   - Exploration and Discovery
   - Procedurally Generated Levels and Loots
   - Unlockables and Achievements
   - Mastery over the Game
   - New Game Plus (Carrying over progress into a new playthrough)
   - Optional Side Quests
   - Branching Storylines (Multiple endings etc.)
   - Competitive Aspects (Multiplayer, Ranking, Leaderboards etc.)
   - Emotional Engagement
   - Other

3. On a scale from 1 to 5, how often do you play roguelike games?
   *Note: Roguelike is a type of video game known for its challenging and procedural (randomly generated) gameplay, often with the concept of permanent character death.
   1 (Never) – 5 (Very Often)

4. **If you have played a roguelike game before:**
   On a scale from 1 to 5, how much do you prefer roguelike games?
   1 (Strongly Dislike) – 5 (Strongly Like)

5. Which of the following elements do you think contributes the most in game replayability?
    - Procedural Content Generation (Randomly generated content such as levels and loots)
    - Permanent Character Death (Progress reset on death)
    - Metaprogression (Getting new upgrades after every playthrough)
    - Discovering New Elements (NPCs, Bosses etc.)
    - Unlockables and Collectibles
    - Resource Management
    - Different Playstyles in each Playthrough / Build Diversity
    - Rare or Special Occurrences (Secret Items, Quests etc.)
    - High Difficulty
    - Other

Question 6 – 11 asks about the respondent's opinion on very specific features, and how important they think it is in increasing replayability.

6. How important do you think randomized level layouts and item placements are in enhancing replayability?
   1 (Not important) – 5 (Very important)

7. How much does the presence of unlockables or achievements affect your motivation to replay a game?
   1 (Not at all) – 5 (Extremely)

8. How do you feel about permanent character death (permadeath) in a game?
   1 (Strong Dislike) – 5 (Strongly Like)

9. How important do you think is a diverse selection of character / classes in increasing game replayability?
   1 (Not important) – 5 (Very important)

10. To what extent does the presence of a narrative or lore impact your willingness to replay a game?
    1 (Not at all) – 5 (Extremely)

11. How much do online leaderboards and rankings influence your decision to replay a game?
    1 (Not at all) – 5 (Extremely)

Question 12 – 14 allows the respondent to input some text to provide their own suggestions on how to increase game replayability.

12. Based on your experiences, suggest a game that you have played before and what improvements can be made to enhance its replayability. (Input Text)

13. Are there any specific features you believe would make a game more replayable? (Input Text)

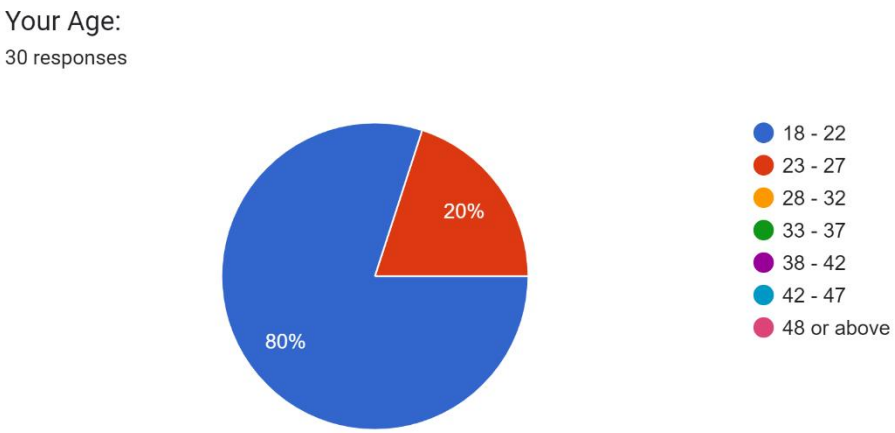
14. Any additional comments or insights you'd like to share about roguelike games and their replayability? (Input Text)

3.3 Analysis

In this section, the analysis of data collected through the questionnaire is presented. The questionnaire was distributed to gather public input and opinions on various aspects of the game project. A total of 30 responses has been received from the questionnaire.

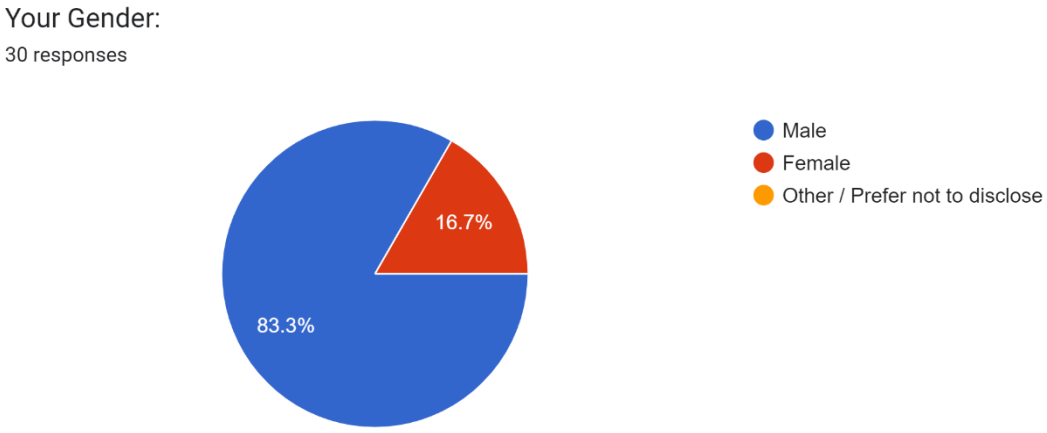
3.3.1 Section A: Demographics

Question 1: Age of Respondent



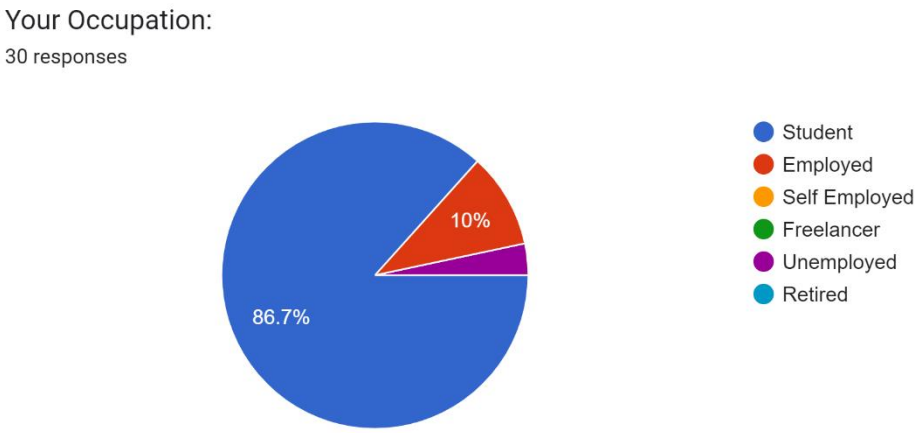
The age of the respondents range between the age of 18 and 27, with the majority of them (24 individuals) with their age between 18 - 22. This is most likely because the questionnaire is targeted towards video game players, which apparently belongs to younger age groups.

Question 2: Gender of Respondent



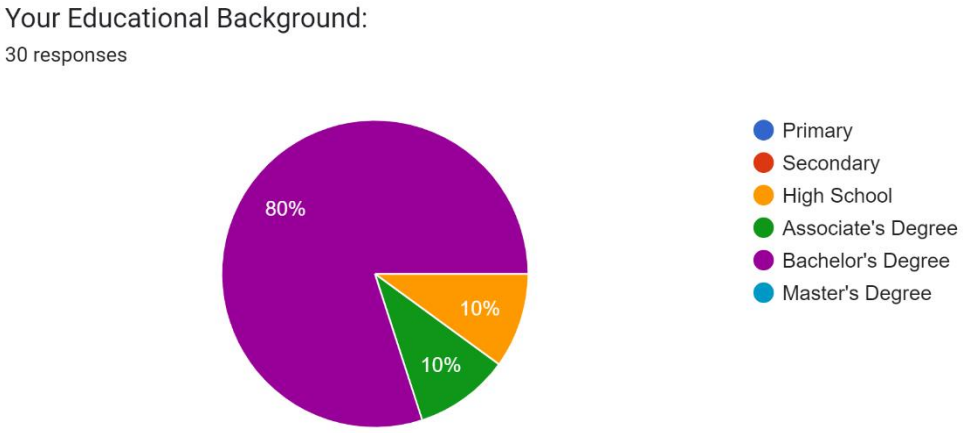
According to the chart, most of the respondents that showed interest in completing the form are male (25 individuals).

Question 3: Respondent's Occupation



Out of the 30 respondents, 26 of them are students who are still undergoing studies. There are also 3 respondents who are currently employed as well as 1 who is unemployed.

Question 4: Respondent's educational background



Most of the respondents of the questionnaire appear to be undergoing their studies in bachelor's degree (24 individuals). There are also 3 who are still in high school and 3 who are undergoing their study in associate's degree.

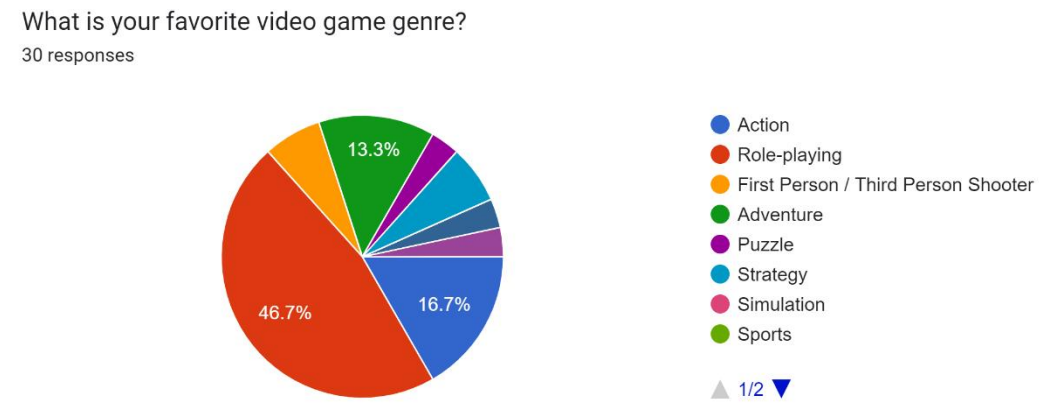
3.3.2 Section B: Gaming Habits and Preferences

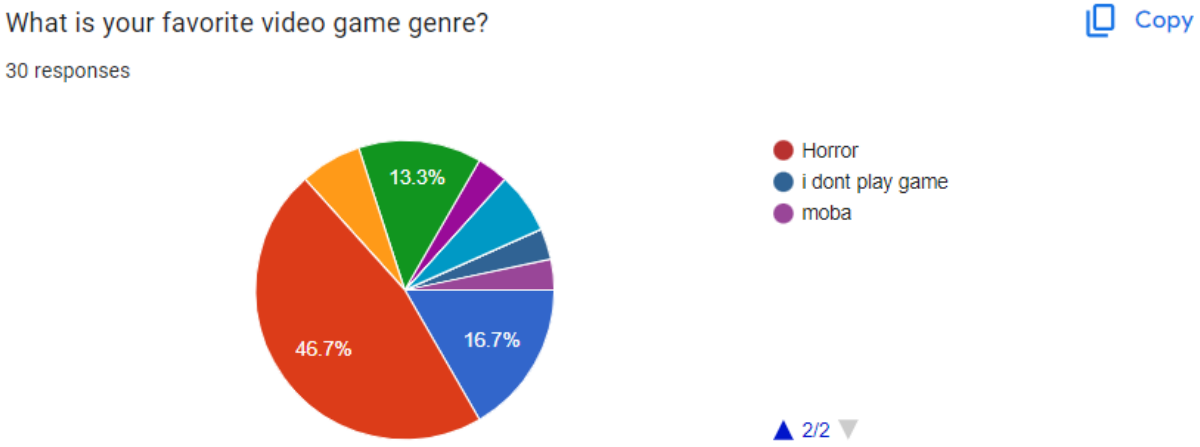
Question 1



According to the questionnaire, the daily gaming time of respondents vary greatly. However, 1-2 hours of playtime every day seems to be the most common among the others (8 participants). There are also 5 respondents who play less than 1 hour of video games every day, 7 respondents who play for 2 – 3 hours every day, 4 respondents who play 3 – 4 hours every day, 3 respondents who play for 4 – 5 hours every day as well as 3 respondents who play video games for more than 5 hours every day.

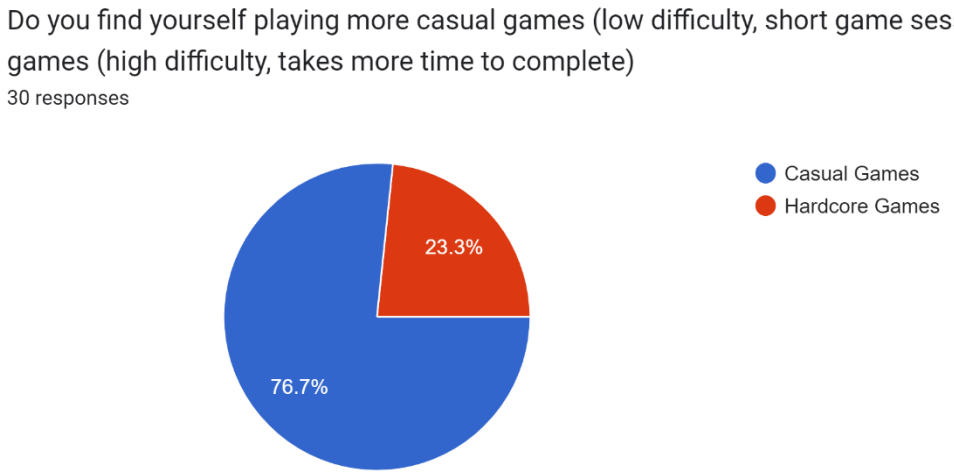
Question 2





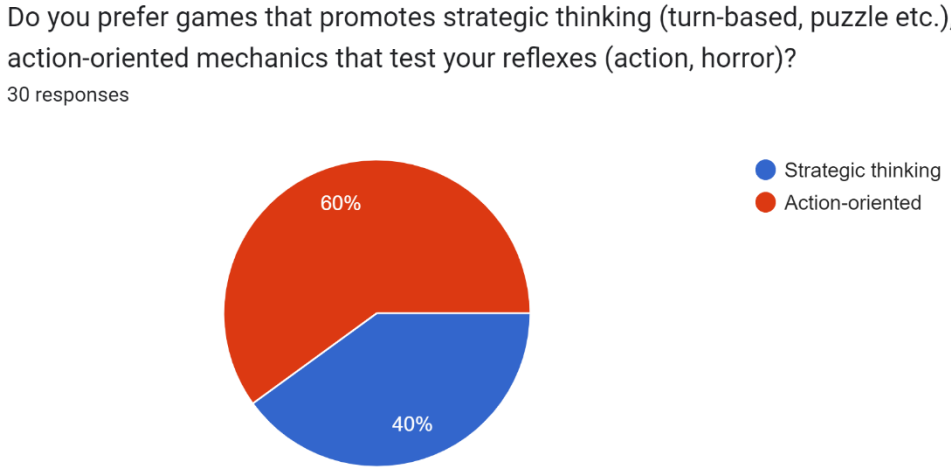
This question inquires about the respondent's favorite video game genre. Role-playing games turned out to be the favorite of the majority (14 respondents). Some of the respondents also chose to add their own option, such as the Multiplayer Online Battle Arena (MOBA) genre.

Question 3



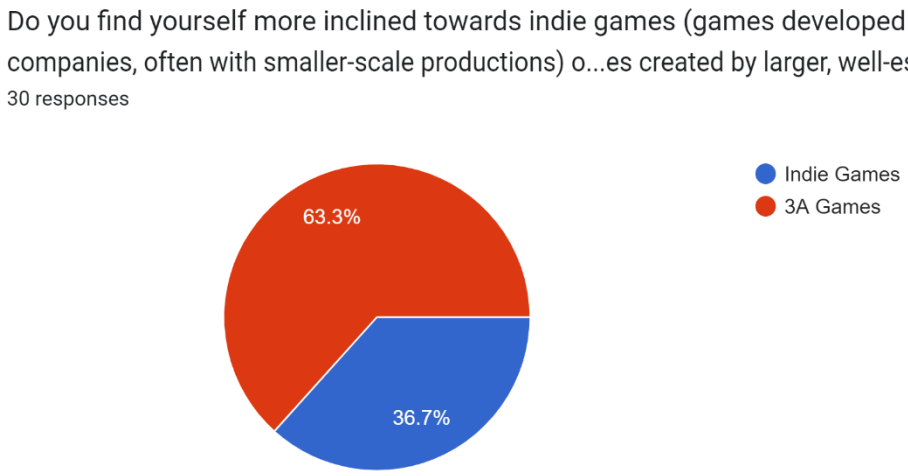
The third question found out that most of the respondents (23 respondents) consider themselves as casual gamers, meaning that they prefer games with shorter gameplay sessions and lesser difficulty. There are also 7 respondents who prefer hardcore games that put their abilities such as strategic thinking and reflexes to test.

Question 4



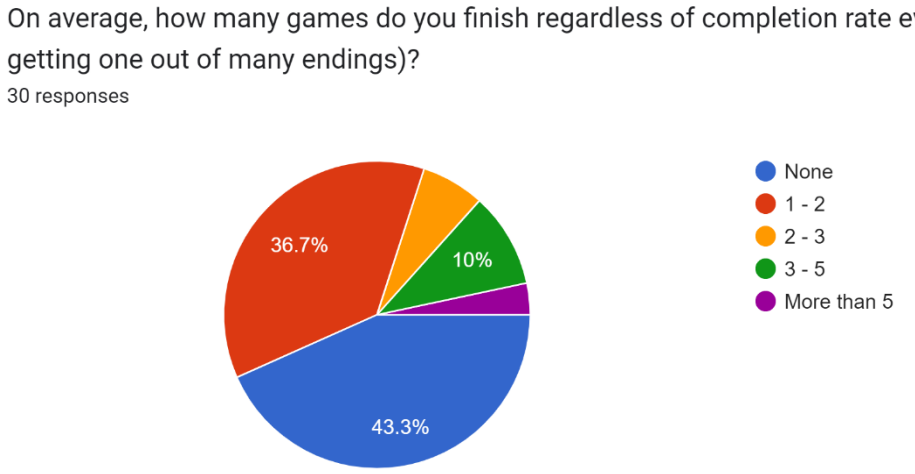
Question 4 shows that 18 of the participants prefer action-oriented gameplay that test their real-time reflexes, while 12 of them prefer games that promote strategic thinking and allows them to take their time to figure out the best course of action.

Question 5



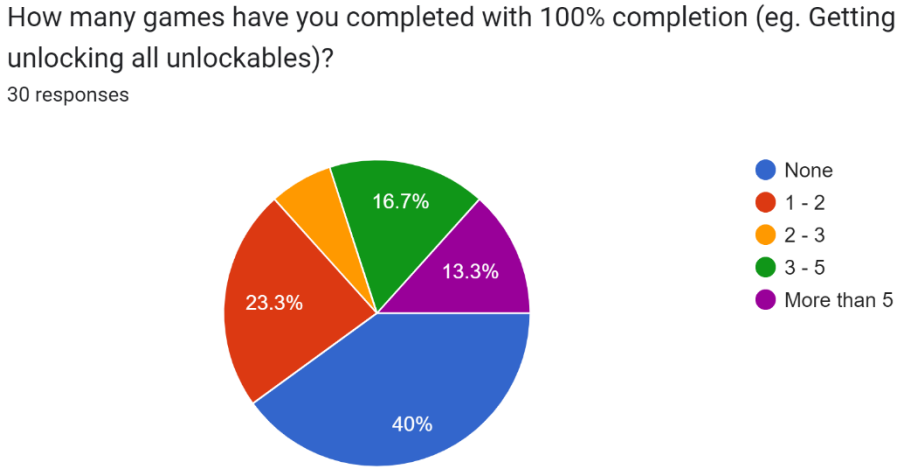
This question asks the respondent if they find themselves more inclined towards games developed by small, independent teams that usually promotes creativity or 3A games developed by large companies that utilizes top-tier graphics and technology. Among them, 19 prefer games made by 3A companies while 11 chose indie games over 3A games.

Question 6



According to the chart, out of 30 respondents, 13 of them do not complete any video games per month. This might be due to the younger generation's shorter attention span, constantly trying new games instead of spending an extended amount of time finishing one; or it could be because of most of the respondents having to focus on their studies.

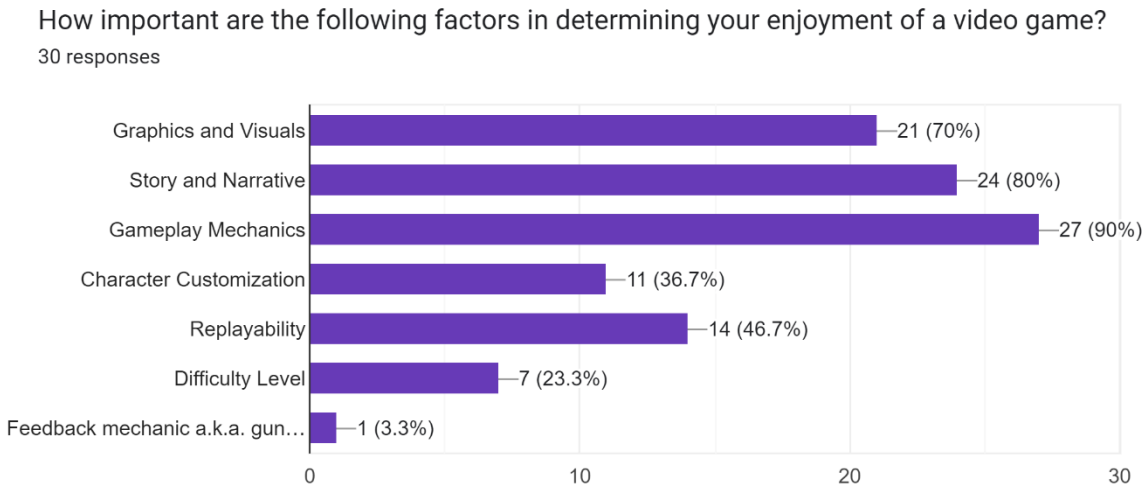
Question 7



12 of the respondents have never completed a game with 100% completion rate before. This is likely because most game have hidden achievements and unlockables that are hard to be obtained

under normal circumstances. However, 60% of the respondents have finished at least one game with 100% completion rate.

Question 8



Question 8 inquires the respondent about what they enjoy the most about a video game. The majority of the respondents think that gameplay mechanics are the most important aspects to bring enjoyment when playing a game, while game story and narrative is a close second. A respondent also pointed out game feedback is an important aspect to consider

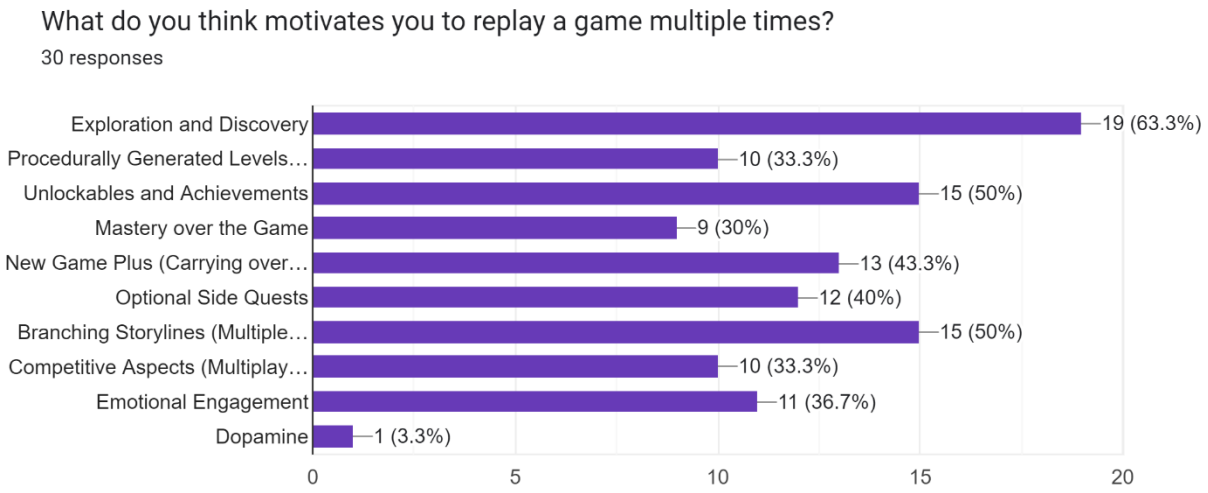
3.3.3 Section C: About Game Replayability

Question 1



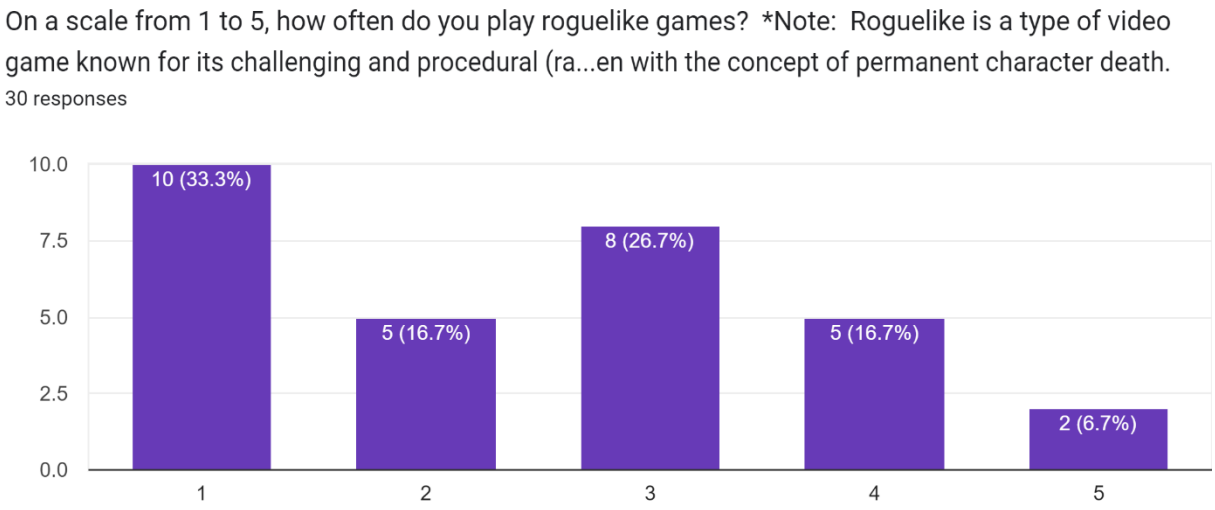
The first question in this section asks the respondent if they often replay a game they have played before. It appears that most players do not replay a game they have completed, which is likely due to the most of the modern games lacking new content to motivate players to replay them.

Question 2



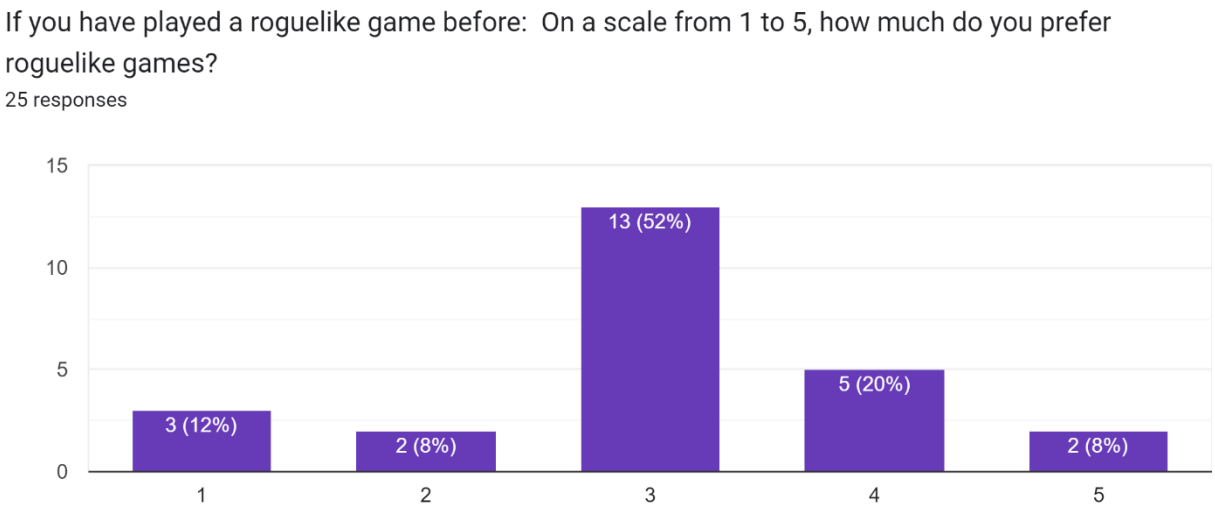
The second question asks the respondent to select aspects of a video game that they think motivates them to replay the game (multiple answers can be selected). The majority of them suggest that exploration and discovery is the most important aspect to motivate players to return to a game, which is one of the crucial elements of a roguelike game.

Question 3



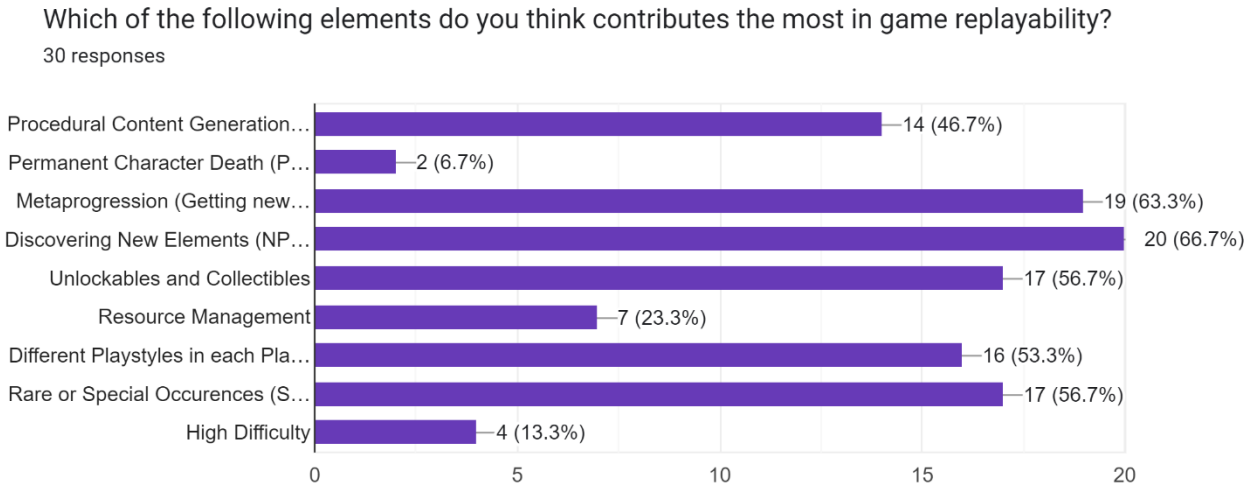
Out of the 30 respondents, 20 of them have played a roguelike game before, but most of them do not consider themselves playing roguelike games as often. There are also 10 respondents who never / almost never played a roguelike game.

Question 4



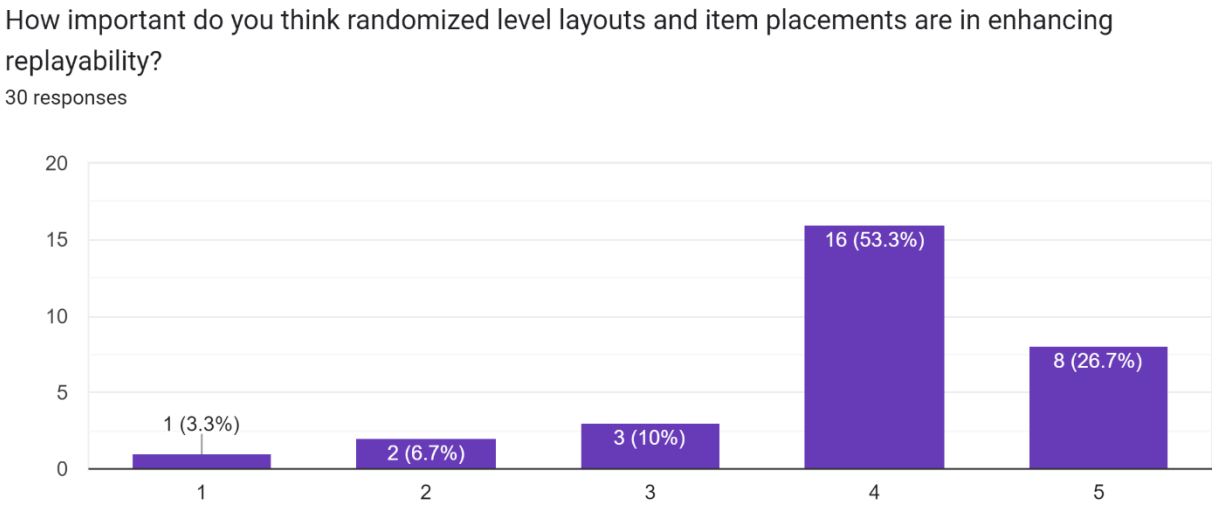
This is an optional question for respondents who have played roguelike games before. Most of these respondents have a neutral attitude towards roguelike games, while 7 of them do prefer roguelike games and 5 of them dislike roguelike games.

Question 5



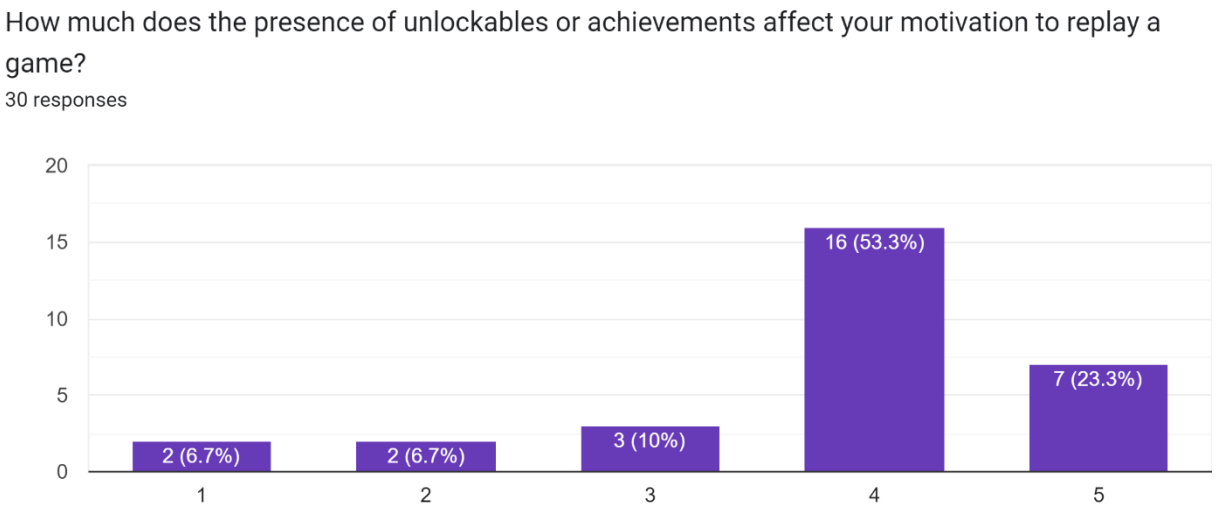
This question is similar to question 2, except the aspects included are the common elements that appear in roguelike games instead of generic elements of video games. Most respondents find discovering new game elements to be the most amusing. The respondents also showed great interest in finding other rare elements such as rare events and unlockables. Metaprogression (character growth carried across gameplay, usually exist in roguelite games) is also one of the aspects that the respondents think contribute towards replayability. Unlike traditional roguelikes, roguelite games with metaprogression helps with the sense of loss when the player loses, as they still got some reward out of the failed run.

Question 6



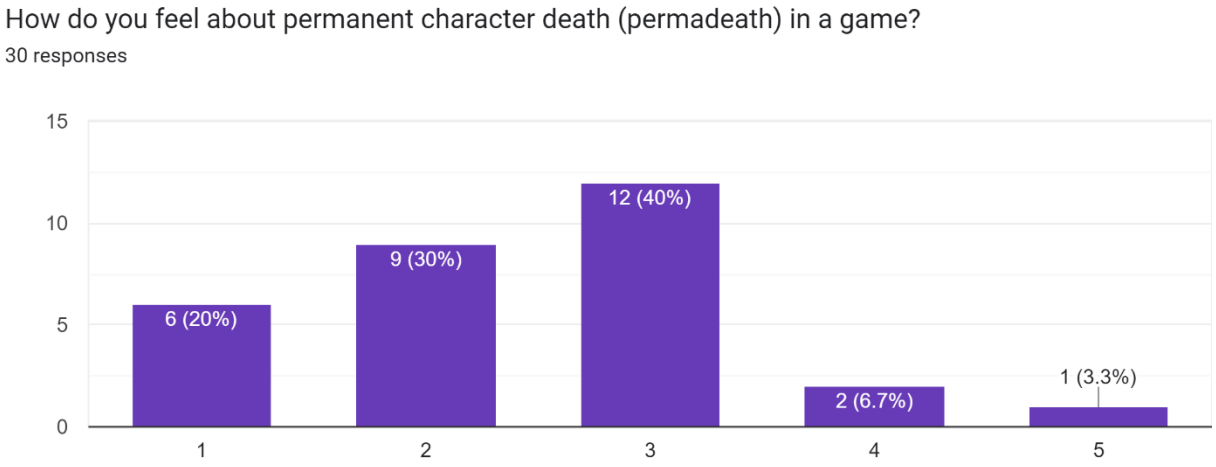
This question asks about how important the respondents think randomized level layouts and item placements are in terms of increasing game replayability. Most of the respondents (24 individuals) think that it contributes greatly towards game replayability, 3 suggest that the influence is neutral while 3 thinks that it does not contribute much.

Question 7



The majority of the respondents think that adding unlockables and achievements greatly increase their motivation to replay a video game, while 4 thinks otherwise.

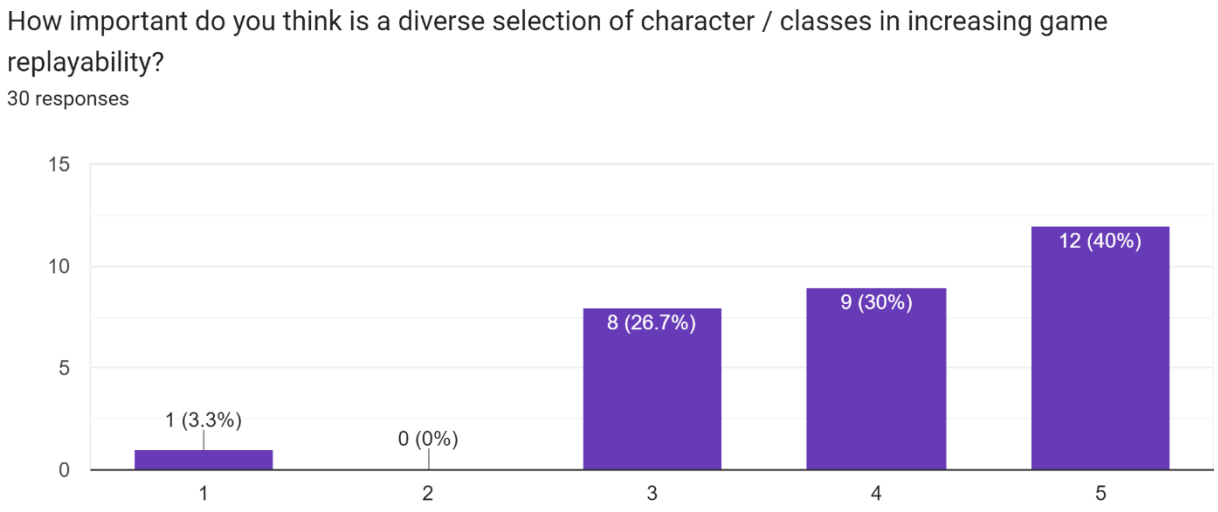
Question 8



Most respondents are either neutral or dislike the concept of permanent character death in video games, even though it is one of the most common elements found in roguelike games. This might

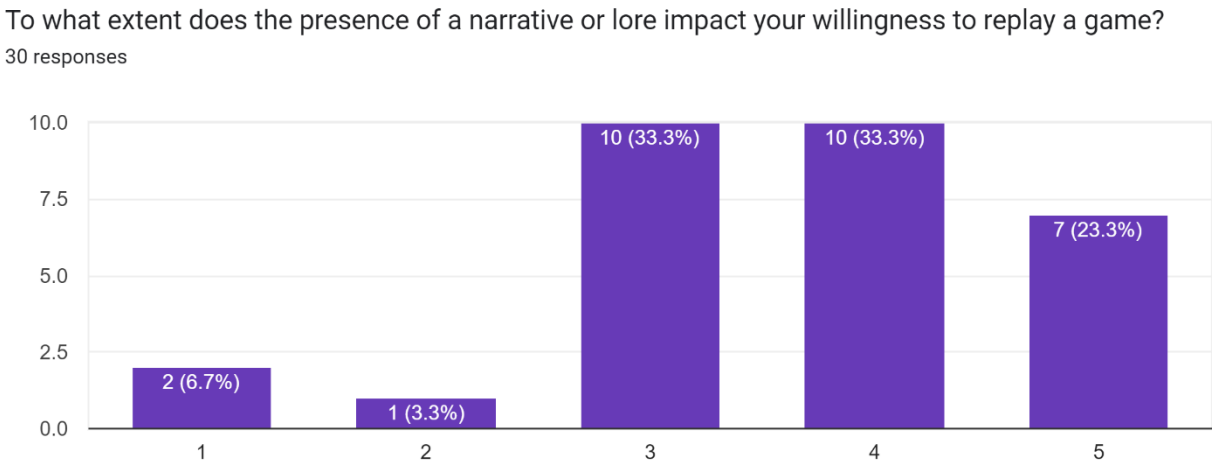
be due to the respondents' lack of interest in roguelike games, or they feel demotivated when their character dies. Implementation of metaprogression might be able to help making them feel better about permanent character death.

Question 9



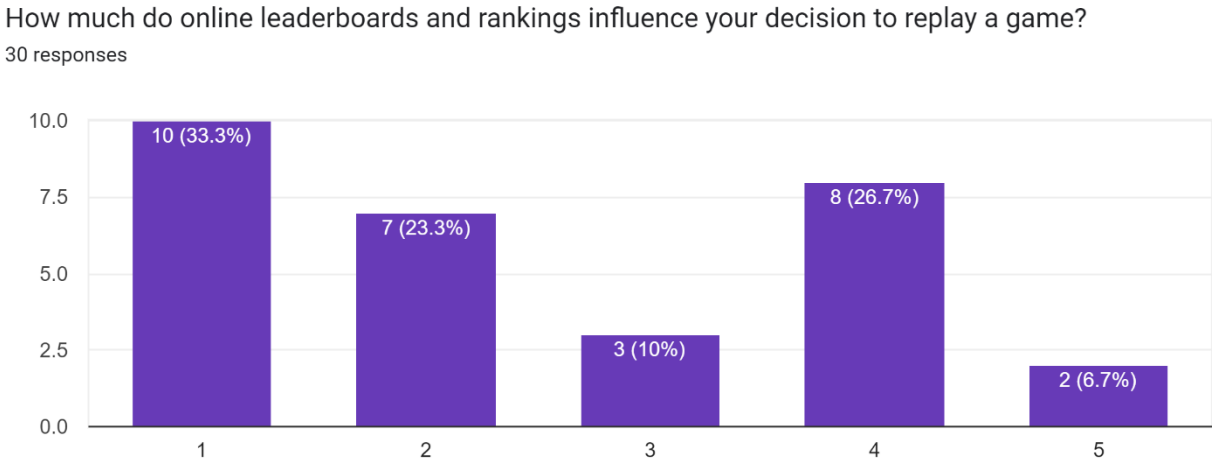
The majority of respondents believe that by offering a diverse selection of character and classes in a game, the game's replayable value will be greatly increased, as it allows them to attempt the game with different character builds and playstyles.

Question 10



According to the graph, 17 respondents think that by adding good narrative and lore into a video game, they will become more motivated to replay it. However, 10 of them remain neutral regarding the influence of game narrative towards replayability while 3 of them disagree.

Question 11



Most respondents do not think that having online leaderboards and rankings will influence their decision to replay a game. This might indicate compared to comparing with other players, what matters to them is that the game itself has something that attracts them to replay it.

Question 12: Based on your experiences, suggest a game that you have played before and what improvements can be made to enhance its replayability.

(Note: Blank responses are removed.)

1	Phasmophobia - Gameplay feels almost identical every round, the ghost AI could be more unique.
2	Risk of Rain 2, more new updates are needed, games with high replayability will eventually get bored as the players will eventually finish the game's progression.
3	Minecraft Dungeons, the game has secret levels but sometimes the RNG for the level to appear is too low which can be frustrating after numerous playthroughs of the same levels although each they are procedurally generated
4	Devil may cry , good storyline and gameplay mechanic
5	Fate Grand Order. Enhance battle system and banner.
6	Don’t Starve, you can try to add "Sanity", which is as it falls, the screen will begin to distort and shake
7	Returnal(ps5). the game may be difficult for new players therefore a difficulty level can be used.
8	Slay the Spire: Have some metaprogression
9	GTA 5, sometimes the storyline is too boring, better to enhance the storyline to make sure it is attractive enough
10	FF7R - Levels are very important in Hard Mode, causing players returning to play the second time requiring to grind at least a few days to max level, not to mention the weapons also require levelling using the Save and Load Method. This is rather tedious especially

	on a PS4 that doesn't have an SSD since you would have to go through those load screens repeatedly. What they could do is give us an arena or mode that allows an extra XP gain so they we ca continuously grind without needing to reload save files repeatedly.
11	cyberpunk2077, no need to replay whole story to unlock secret ending after they miss the opportunity
12	Middle-Earth: Shadow of War, a game that require you to build your own army by recruit(brainwash) enemy orc and attack enemy fortress. As the game goes, the number of fortress that you have to conquer increase. Which makes the process of recruiting enemy orcs become boring. I think can let player recruit the enemy orcs in different way instead of unlimited fighting, like utilize your army to pursue enemy orcs to join you, have a small war with several enemy orcs to recruit multiple orcs at the same time, and others.
13	Harvest Moon: Trees of Tranquility, would prefer to extend the storyline more even after quests are all completed.
14	Trails series, secret additional story content, able to unlock previous aspects of the game without grinding all over again
15	Kirby's Dream Land
16	hollow Knight
17	20 minutes till dawn, enemy scla3 based on performance, i.e. rubberbanding. Mostly for pacing issues, weak early game strong late game builds are horribly balanced. Adding more synergy for more on the fly choice instead of predetermined builds.
20	Clash of clans. None.
21	Witcher 3; enhance the players mobility to move on the map, it is the major reason that stopping me to replay it again and again.
23	Plant vs Zombies, availability of cosmetic items
24	LOL，increase game mode

26	vampire survivors with its different skill builds
27	No Man's Sky. I put a lot of hours there. But I hope more things can be done in a certain place, rather than going aimlessly without goal discovering a planet that will have similar attributes between each other. The uniqueness of the activity also adds a lot of points.
28	Risk of Rain 2, random map generation (layout)
29	Genshin impact
30	Through the ages. A good turn based strategic board games. But if playing as video games instead of physical board games, I would suggest to add more graphically actions for the war.

Question 13: Are there any specific features you believe would make a game more replayable?

(Note: Blank responses are removed.)

1	DLC
2	Meme feature, new playstyle system and impressive storyline
3	After every time dead, will keep something to let next time gameplay better and easier. For example, talents point
4	add more features that will be unlocked after completing like a new gun, weapon, skills, etc. that players can use in combat. Players will have the motivation to want to continue on to try out these weapons or skills.
5	Let me customize the randomness setting so I can choose to get the build I want more easily (e.g. More rerolls)
6	Player has their own freedom to explore inside the game. Such as Watch Dog, player can hack whatever inside the game
7	Game Mechanics that introduce new ways of gameplay or different endings.
8	multiplayer features, play with friends or family will increase the replayabilitity
9	various of character playstyle
10	Graphics and storyline
11	Higher difficulty+ new game plus
12	Character cuteness
13	settings of the world of the story
14	Community mod support, games such as gmod, koikatsu!, rivals of aether, ultimate marvel vs capcom 3 lived w/o dev support via community mods.

15	Different side stories which are affected by different playthrough.
16	Deeper lores
17	Marketplace for trading rare loots
18	diverse of character，big skill trees
19	Unique experience
20	A balance between a randomly generated levels and unique activities within. Because games either have a full on randomly generated level with same activities or full-on various activity with no chance of it's randomly generated.
21	Variety in viable strategies (e.g. different builds or gameplay styles)
22	Joy of draw
23	Randomize it in suitable way, avoid pay to win

Question 14: Any additional comments or insights you'd like to share about roguelike games and their replayability? (Optional)

(Note: Blank comments are removed)

1	Honestly more roguelike games should have metaprogression, that will make it more fun
2	I don’t play much roguelike games so no idea.
3	smooth action
4	Left 4 dead 2. There needs to always be some predictability. Its weapon supply, item spawn is determined by how good the players are. Also I've never seen a roguelite tower defense game :)

5	Some essential parts should be fixed, but some parts such as equipment or skills can be randomized to fulfill different skill combination in every new game.

3.3.4 Summary

This chapter revealed and analyzed the data collected via distributing a questionnaire to the public.

Through Section A, it is found out that most of the respondents are between 18 – 22 years old in terms of age. It is also discovered that most of the respondents are male. The majority of the respondents are currently students, while there are some who are employed and unemployed. Their educational background comprises of associate's degree, bachelor's degree and high school, with bachelor's degree being the predominant background.

Section B has successfully found out most of the respondents' gaming habits and preferences. Most respondents play around 1 – 3 hours of video games on a daily basis. Role-playing games is the majority's favorite video game genre, as it allows a lot of freedom for building characters and provides immersion. Most of the respondents also consider themselves casual gamers, preferring short, easy game sessions over games that test their limits with high difficulty. They also prefer games that promote action-oriented gameplay mechanics over games that emphasize on strategic-thinking. 3A games also seems to be the more popular choice within the respondents compared to indie games. It is also apparent that not many respondents finish the games they purchased. Lastly, the respondents seem to agree that gameplay mechanics is the most important aspect that contributes toward their enjoyment of a video game.

Section C asks the respondents various questions regarding game replayability. It appears that most of the players seldom play roguelike games, despite it being one of the genres with rapidly rising popularity. They also agreed that the sense of exploration of discovery, whether it be new features, events, characters, to be the greatest way to increase a game's replayability. The respondents' thoughts on individual elements that are usually found in roguelike games are also inquired, helping the developer to understand how important each of them is in terms of increasing game replayability. Last but not least, the respondents also shared their thoughts on how to increase replayability of existing game products that they have played before.

Overall, the questionnaire proved to be of great help to the develop to understand the needs and preferences of the players, and will greatly contribute towards shaping a better game product as a result of this project.

Chapter 4: Design and Implementation

4.1 Introduction

In this chapter, the develop will walk through all the designing process of the core systems that forms the backbone of the final game and make it what it is. This includes important systems such as the custom character controller, level generation algorithm and its implementation, combat design, artificial intelligence for enemies as well as various other gameplay systems such as audio and interaction systems.

Various design techniques as well as specific ways of implementation will be discussed in depth.

4.2 Design

4.2.1 Level Generation Algorithm

For a roguelike game, a good implementation of a complex but customizable level generation algorithm is crucial as it directly affects player experience in a lot of ways.

To start it off, the method of procedural generation has to be decided. After much consideration, this is how the final design of the generation method comes out:

The generation method used will be a modified version of a **Depth-First Search (DFS)** algorithm, also known as the **backtracking algorithm**. The DFS algorithm is often used to create maze or find the shortest path between two points (Chen & Wu, 2020). To put in it simple terms, a 2-dimensional grid (referred to as the **board** afterwards) is first generated. The individual grid in the board will be referred to as **cells**. The board can be in any shape and size. The algorithm then utilizes a stack, and as the algorithm starts running, it takes a cell as its starting position on the board and take a step in the board in a random direction while storing the visited cell into the stack. The exit direction of the previous cell and the entry direction of the next visited cell will also be stored, marking these directions of the respective cells to be **open**. This process is repeated until it reaches any end of the maze. It will **not** visit cells that have already been visited. Once it reaches any end, it will start backtracking, removing elements from the stack and searching for alternative

paths. The algorithm ends when the stack is emptied, but the developer is free to add their own conditions to stop the algorithm halfway once certain conditions has been met.

After the result is generated, each cell will represent a **room** that will be generated, and the **opened direction** of each cell represents a **connection** between the rooms. However, using the algorithm as is comes with a few problems:

- Since each cell can only be visited once when not backtracking, the generated result will be a tree-like structure where it is impossible to have loops (where later visited cells connect back to earlier cells via a different path). This is useful for pathfinding but not particularly exciting for generating a robust level.
- The board will always be filled with cells, so unless the board itself is also procedurally generated there will always be the same number of rooms in the end.

To suit the game' needs, the developer modified the algorithm as follow:

- Instead of strictly restricting the algorithm from visiting cells that has been previously visited, the developer added the option to remove this restriction, allowing the algorithm to visit the same cells repeatedly. This allows the algorithm to repeatedly visit the same cells from different directions, creating more connections in the generated result.
- A customizable max room count and max step count parameters is implemented, ensuring that the number of rooms is customizable and the algorithm does not get stuck in an endless loop due to the removal of the previous restriction. The max room count parameters also makes it so that not all cells on the board will always be visited, creating more variation in shape.

The result is a **graph** with a number of nodes and their respective connections that can be used to spawn rooms or perform any other actions that the developer desires (which will be explained later in the report). It is important to note that the generation result does not exist physically.

After this graph is successfully generated, the developer utilizes a **Room Spawner** script that reads the generated graph. This script is responsible for not only spawning the actual rooms and levels that the game will take place in but also for deciding which kind of rooms will be spawned and their rotation.

A number of **room prefabs** has to be provided to the room spawner for it to spawn the physical rooms. These prefabs have to be tagged as follow:

- 1 Exit,
- 2 Exits arranged in I shape,
- 2 Exits arranged in L shape,
- 3 Exits (T-shaped),
- 4 Exits (Cross shaped),

Which are all the possible shapes that a room can exist in a 2d grid.

After reading the generated graph, the room spawner decides which of these prefab types match each of the cells on the board and what rotation they need to be in. Each prefab types can also consist of more than one prefabs, which if true the room spawner will pick a random one to spawn upon request.

Following are examples prefabs used for visualization:

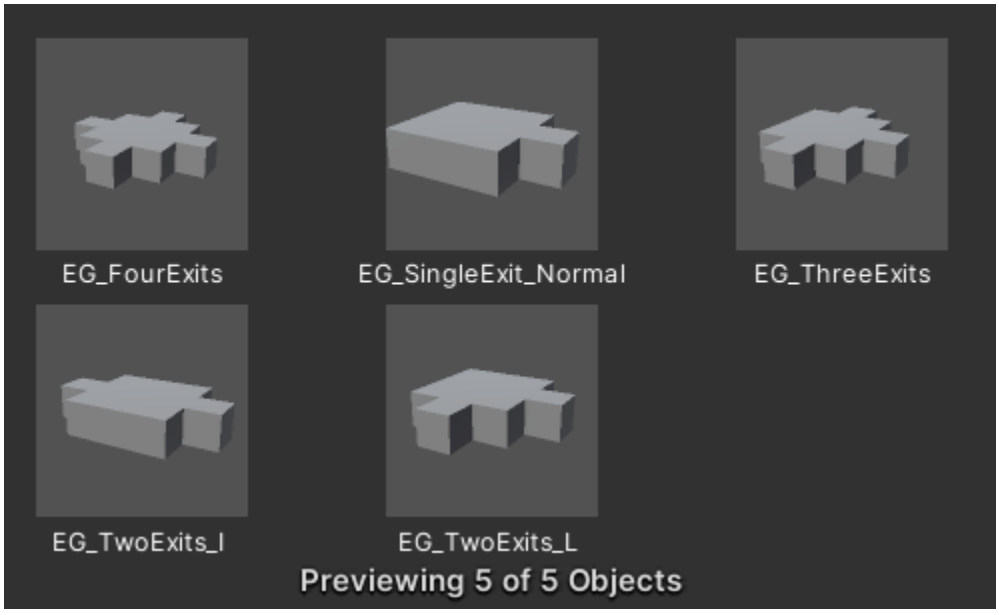


Figure 5 Example Prefabs

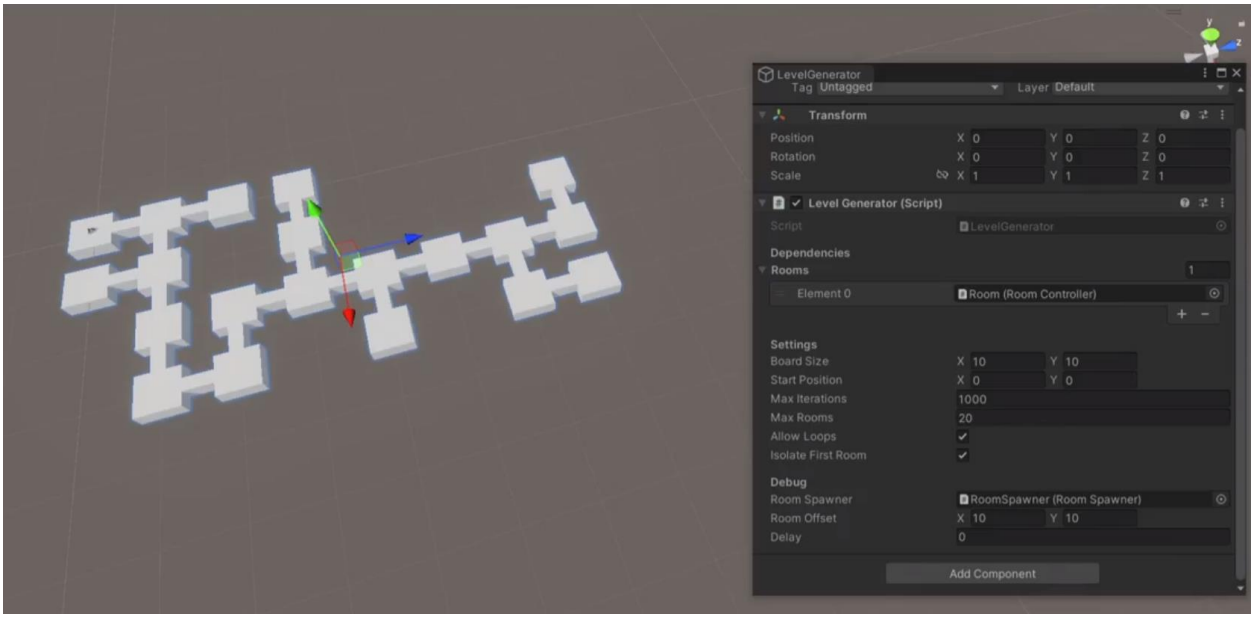


Figure 6 Example Visualization

Aside of deciding which room to spawn from a list of prefabs, the developer can also impose custom **rules** to the room spawner for gameplay purposes.

```
namespace CBA.LevelGeneration
{
    [Serializable]
    3 references
    public class RoomSpawnerRule
    {
        [HideInInspector] public string ruleName;

        public ERoomType roomType;
        public int MinSpawnIndex;
        public int MaxSpawnIndex;
        [Range(0, 1)]
        public float SpawnChance = 1;

        1 reference
        public void UpdateName()
        {
            ruleName = roomType.ToString();
        }
    }
}
```

Figure 7 Custom Rules for Room Spawner

- The roomType variable is an enum that defines the gameplay purpose of a room, such as being a normal room, entrance room of the level, treasure room, or boss room.
- The minimum and maximum spawn indices decide in which indices this type of room can exist. For example, the entrance room should always be the first room, and therefore should have a minimum and maximum spawn indices of 1. Assuming that there are 20 total rooms, and the first room should be an entrance room and the final room should be the boss room, then the normal rooms can have a minimum spawn index of 2 and maximum index of 19 so that they do not occupy those positions.
- The spawn chance is an extra parameter in case there are 2 types of rooms that are desired to spawn within the same indices range. For example, just like normal rooms, treasure rooms can spawn in anywhere between indices 2 and 19, but they have a much lower chance to spawn.

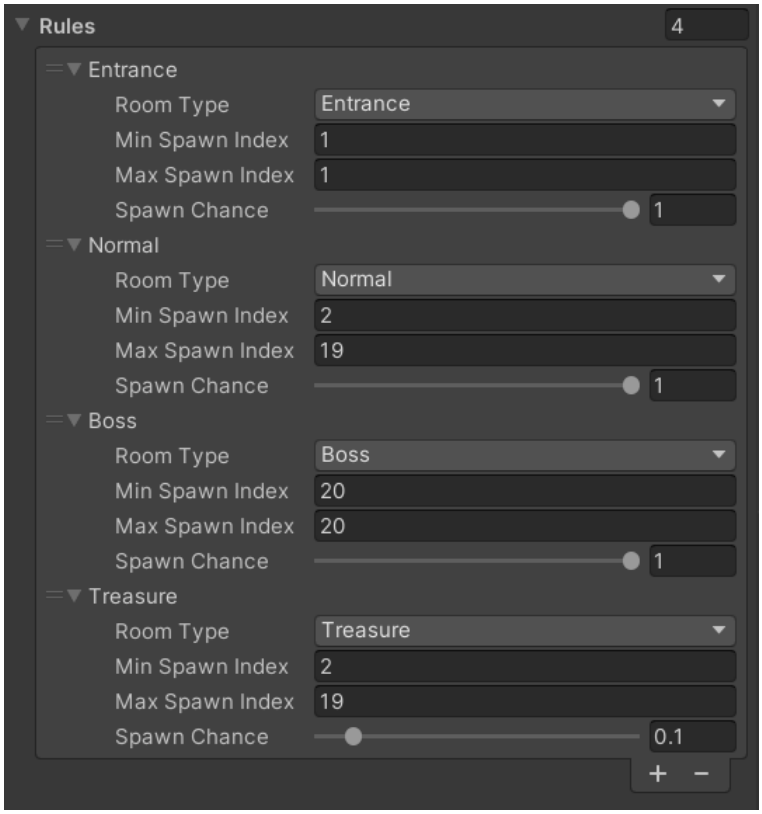


Figure 8 Example rules set used in the first level of the game

In the example above, the first room will always be the entrance room while the last room will always be the boss room. Rooms 2 – 19 will be either normal or treasure rooms, where treasure rooms have a much lower chance to spawn compared to normal rooms in order to create an element of surprise for the player.

When the spawning of rooms is completed, the room spawner will invoke a **RoomSpawned event**. When the **Level Manager** receives this event, it knows that the level is ready to be played and will spawn the player into the first room.

Although the visualization above showcases the generated rooms being physically connected, having the rooms connected physically comes with some major downsides: each room must be in the same size and the position of the exits should match each other. To overcome this issue to create more variety in level design, the developer decided that the rooms will not be connected physically. All the rooms spawned will exist at the world origin (0,0,0), but only the current room the player is in will be activated in the game world. A **shader** is used to darken the exit ensuring that the player cannot see what lies beyond each exit.



Figure 9 Exit Shader

When the player exits a room, the Level Manager **despawns** the current room, and teleports the player to the exit of the respective entrance of the next room before activating said room. A **screen fadeout** effect is also used to hide the teleportation, creating the illusion that they progressed in the same direction they exited from.

Last but not least, each room contains a **Room** script that records the direction of each exit in the prefab. This is important for the Level Manager to dictate which direction each leads to and which room to activate when the players leave through any exit. The rooms also call a **OnPlayerEnter** event and an **OnPlayerExit** event that can be subscribed by other scripts for extra behaviour. For example, the **Enemy Room** script subscribes to the **OnPlayerEnter** event, so that when the player enters the room all exits will be locked and the player has to defeat all enemies within the room to unlock the exits and progress forward.

The generation speed of the level generator is carefully optimized. Figure below showcases the speed of 10 generation attempts of the first level in the game.



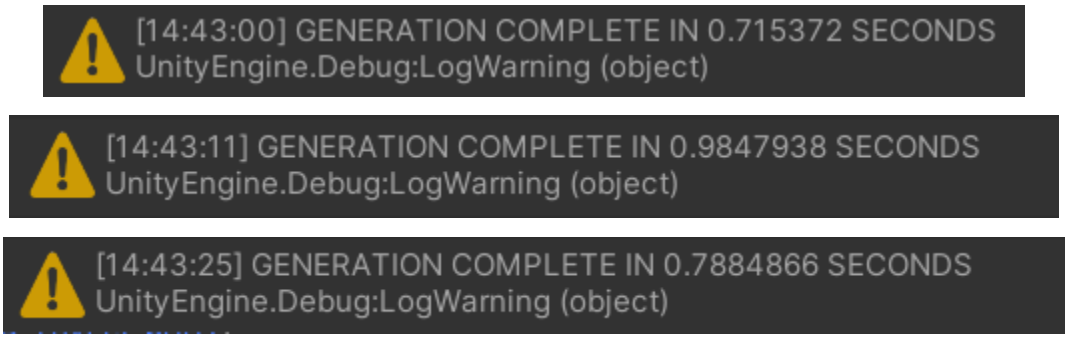


Figure 10 Level Generator Speed Test

Judging from these tests, the generator is able to generate a level with 20 rooms with hundreds of connections with the average time of less than one second. This speed can be further optimized if the generator does not use a coroutine as it a coroutine limits the algorithm to run only one iteration per frame, but the developer decided to keep the coroutine to enable other features such as adding an artificial delay between each step to visualize the generation.

4.2.2 Player Controller

One of the design pillars of Curse Breaker: Awakening is to create a fast-paced, movement-based combat system. To achieve this, the developer spent quite some time creating, testing and refining the custom player controller made for the game.

The player controller utilizes a state machine to manage each states the player can be in and their respective behaviours. The final states that the player has are:

1. Walk State
    - The default movement state of the player. This applies to both when the player is grounded or airborne, where the player rigidbody moves with a force depending on the player's input direction.
2. Sprint State
    - Activated when the player holds the sprint button and has enough stamina. The movement behaviour in this state behaves similarly to the Walk State, except that it gives the player a boost in the movement force added to the rigidbody at the cost of stamina.

3. Wall Run State
    - Triggered when the player is airborne and there is a wall near either side of the player (detected with a raycast). Provides a slight speed boost to the player as long as they stay close to the wall and face the respective direction at the cost of stamina. The camera also tilts to indicate when a wall run is successful. This is one of the most important feature that makes the movement fast and satisfying.
4. Crouch State
    - Entered when the player is grounded and holds the crouch button. Lowers the player's collider as well as slightly reducing applied movement force. While the levels currently do not introduce any tight spaces that can only be navigated through when crouching, it might be useful in some specific scenarios, such as dodging incoming attacks.
5. Charged Attack State
    - Triggered when the player releases a charged attack (the charging of said attack is managed by a non-movement related script). Overrides the player rigidbody and camera movement temporarily and sets the player rigidbody's velocity to the forward direction of the camera, creating a rapid dash movement. Some collision layers of the player are also temporarily disabled to allow the player to dash through certain objects such as enemies.

All of these states are managed by the **Player Controller** script, which serves 3 main purpose:

1. Read input from another script responsible for receiving input, which is the PlayerInputHandler script, and triggers the respective actions accordingly
2. Provides customizable parameters in the inspector, such as movement force of each state, jump force, stamina limit and consumption and many more.
3. Receives input from the current active state and apply the actual effect via the **Movement Module** script, which is simply a custom wrapper script made for Unity's rigidbody component with extra features.

The detailed inner working of the Player Controller script can be examined within Appendix F.

4.2.3 Combat System

As mentioned before, the combat in Curse Breaker Awakening is designed to be fast and movement-based. All combat actions of the player are supported by the PlayerCombatManager script, which is responsible for receiving combat-related input, and handling the weapon.

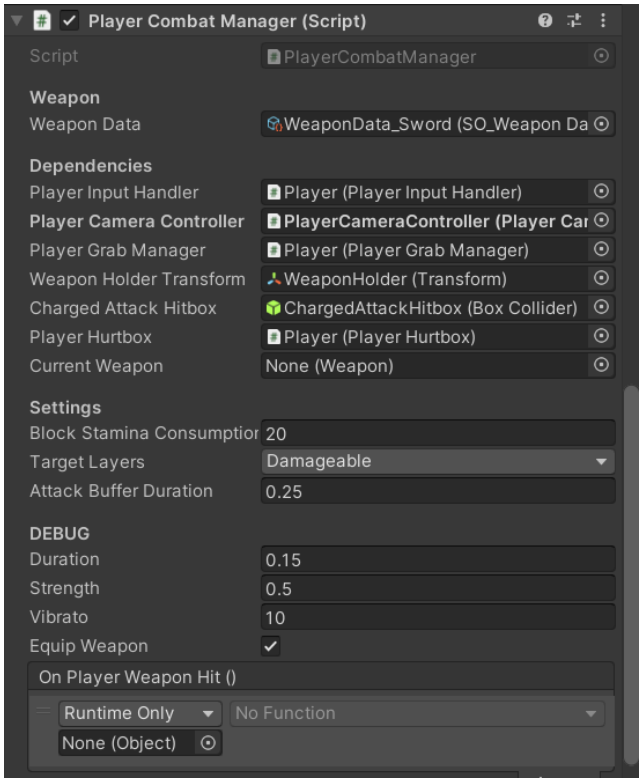


Figure 11 PlayerCombatManager Inspector View

When the game begins, this script will equip the player with the corresponding weapon provided in the **Weapon Data** field. Weapon data are scriptable objects that hold basic information for the weapon, such as its name, rarity, type, prefab reference and other parameters such as damage and combos.

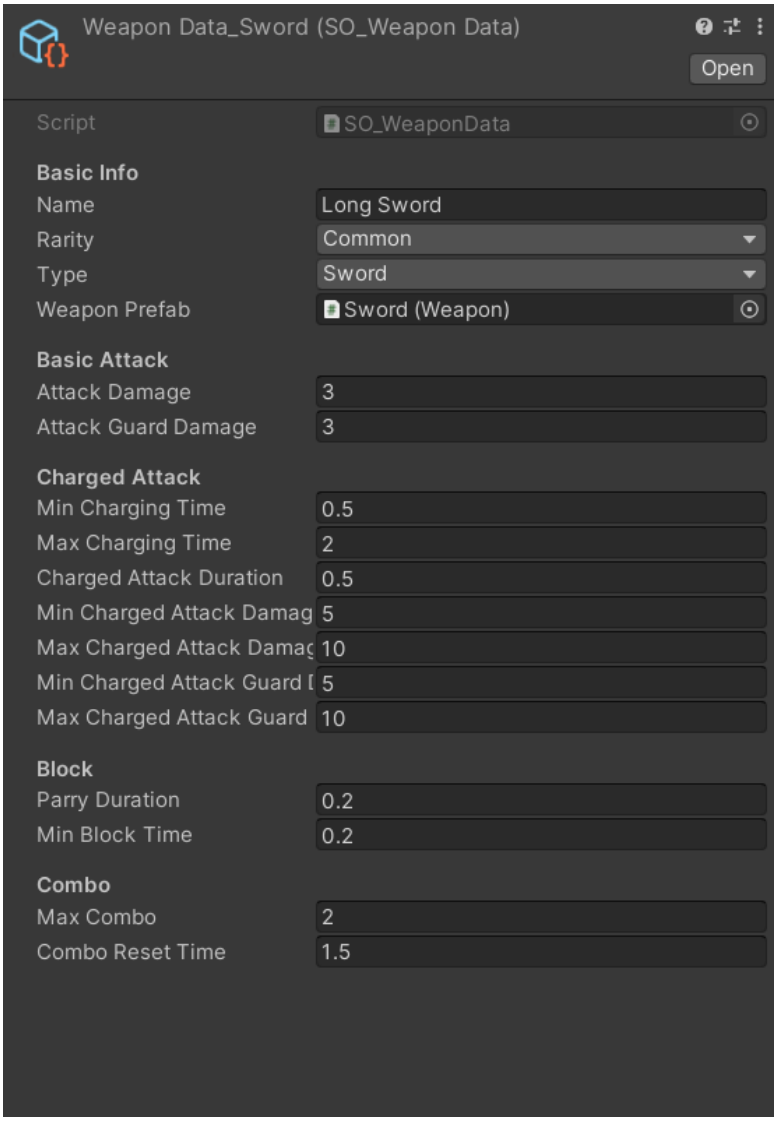


Figure 12 Weapon Data Scriptable Object

The weapon prefab will have a **Weapon** script attached, which is responsible for the actual actions the weapon itself can perform. The combat manager is only responsible for informing the weapon that an input is performed as well as equipping weapons, but what the weapon does is entirely dependent on the weapon itself. For instance, the sword weapon is able to perform the following actions:

1. Basic attack: Swings the sword with a maximum combo count of 2.
2. Charged attack: Charges up a powerful attack, activating a separate hitbox. It also informs the Combat Manager that charging is performed, allowing other scripts to subscribe to a various charged attack related events (charging start, maxed, released and ended). This is how the PlayerController was able to know about the charged attack and transition into the **ChargedAttackState** and perform the dash movement accordingly.
3. Block: Activates the block state, preventing the player from taking damage at the cost of stamina.
4. Parry: If the player blocks just before getting hit, a parry will be triggered, negating all damage without consuming any stamina. This creates a risk-and-reward situation where the player can try to time their block carefully to save up on stamina to perform their movement actions.

Additionally, the weapon script features input buffering, meaning that input received from the combat manager can be stored temporarily. This is useful when for example, the player presses attack when their current attack has not yet finished, instead of not executing the next attack at all there is a short duration that the second input can be stored. If the ongoing attack finishes before the duration expires, this input will be automatically consumed to perform the next attack.

```
10  namespace CBA.Entities.Player.Weapons
11  {
        Unity Script (1 asset reference) | 3 references
12      public class Weapon : MonoBehaviour
13      {
14          [Header(GameData.DEPENDENCIES)]
15          [FormerlySerializedAs("_weaponData")][field: SerializeField] public SO_WeaponData WeaponData;
16          [SerializeField] private CombatAnimationEventHander _weaponAnimationEventHander;
17          [SerializeField] private Animator _weaponAnimator;
18          [SerializeField] private BoxCollider _hitbox;
19
20          [Header(GameData.SETTINGS)]
21          [SerializeField] private float _attackBufferDuration;
22
23          private PlayerCombatManager _playerCombatManager;
24          private BoxCollider _chargedAttackHitbox;
25
            9 references
26          public bool IsPerformingCombatAction { get; private set; } = false;
27
28          [Header("Effects")]
29          [SerializeField] private AudioEmitter _weaponAudioEmitter;
30          [SerializeField] private GameObject _chargingVFX;
31          [SerializeField] private GameObject _fullyChargedVFX;
32          [SerializeField] private GameObject _hitVFX;
33          [SerializeField] private VisualEffect _blockSuccessVFX;
34          [SerializeField] private VisualEffect _parrySuccessVFX;
35          [SerializeField] private MeshRenderer _weaponMeshRenderer;
36          [SerializeField] private Material _chargedAttackMaterial;
37
            0 references
38          public CombatAnimationEventHander WeaponAnimationEventHander => _weaponAnimationEventHander;
39
40          private LayerMask _targetLayers;
41          private List<Collider> _hitTargetCache = new List<Collider>();
42
43          private int _currentCombo = 0;
44          private bool _hitboxEnabled;
45          private bool _chargedAttackHitboxEnabled;
46          private bool _normalAttackBuffer;
47          private bool _isBlockPressed = false;
48
49          private float _currentBlockTime = 0f;
50          private float _currentChargeTime;
51          private float _currentChargedAttackDamage;
```

```
53      private float _blockAndParryAnimationCooldown = 0f;
54
55      private Material _originalMaterial;
56
        3 references
57      public bool NextComboInputAllowed { get; private set; } = true;
58
59      private Coroutine _normalAttackBufferCO = null;
60      private Coroutine _chargingCO = null;
61      private Coroutine _chargedAttackCO = null;
62      private Coroutine _minBlockTimeCO = null;
63
64      //Events
65      public event Action OnWeaponHit;
66
        1 reference
67      public Weapon Initialize(PlayerCombatManager playerCombatManager)
68      {
69          _playerCombatManager = playerCombatManager;
70          _targetLayers = _playerCombatManager.TargetLayers;
71          _chargedAttackHitbox = _playerCombatManager.ChargedAttackHitbox;
72
73          _playerCombatManager.PlayerHurtbox.OnParrySuccess.AddListener(OnParrySuccess);
74          _playerCombatManager.PlayerHurtbox.OnBlockSuccess.AddListener(OnBlockSuccess);
75
76          return this;
77      }
78
        Unity Message | 0 references
79      private void Awake()
80      {
81          _chargingVFX.SetActive(false);
82          _fullyChargedVFX.SetActive(false);
83
84          _originalMaterial = _weaponMeshRenderer.material;
85      }
86
        Unity Message | 0 references
87      private void OnEnable()
88      {
89          _weaponAnimationEventHander.OnActivateHitboxEvent += ActivateHitbox;
90          _weaponAnimationEventHander.OnDeactivateHitboxEvent += DeactivateHitbox;
91          _weaponAnimationEventHander.OnAllowNextComboEvent += AllowNextComboInput;
92      }
```

```
94      private void OnDisable()
95      {
96          _weaponAnimationEventHander.OnActivateHitboxEvent -= ActivateHitbox;
97          _weaponAnimationEventHander.OnDeactivateHitboxEvent -= DeactivateHitbox;
98          _weaponAnimationEventHander.OnAllowNextComboEvent -= AllowNextComboInput;
99      }
100
        Unity Message | 0 references
101     private void Update()
102     {
103         _blockAndParryAnimationCooldown -= Time.deltaTime;
104
105         if (_isBlockPressed)
106         {
107             _currentBlockTime += Time.deltaTime;
108         }
109         else if (_normalAttackBuffer)
110         {
111             Attack();
112         }
113
114         if (_hitboxEnabled)
115         {
116             AttackHitboxDetection();
117         }
118         else if (_chargedAttackHitboxEnabled)
119         {
120             ChargedAttackHitboxDetection();
121         }
122     }
123
```

```
124     private void AttackHitboxDetection()
125     {
126         Collider[] colliders = Physics.OverlapBox(_hitbox.transform.position, _hitbox.size * 0.5f, _hitbox.transform.rotation, _targetLayers);
127         foreach (var collider in colliders)
128         {
129             if (_hitTargetCache.Contains(collider))
130             {
131                 continue;
132             }
133
134             _hitTargetCache.Add(collider);
135
136             collider.GetComponentInChildren<IDamageable>()?.TakeDamage(WeaponData.AttackDamage);
137
138             OnWeaponHit?.Invoke();
139
140             _weaponAudioEmitter?.PlayOneShotSfx("Sword_Hit");
141
142             if (_hitVFX != null)
143             {
144                 Destroy(Instantiate(_hitVFX, _hitbox.transform.position, Quaternion.identity), 1f);
145             }
146         }
147     }
148
```

```
149     private void ChargedAttackHitboxDetection()
150     {
151         Collider[] colliders = Physics.OverlapBox(_chargedAttackHitbox.transform.position, _chargedAttackHitbox.size * 0.5f, _chargedAttackHitbox.transform.rotation, _targetLayers);
152         foreach (var collider in colliders)
153         {
154             if (_hitTargetCache.Contains(collider))
155             {
156                 continue;
157             }
158
159             _hitTargetCache.Add(collider);
160
161             collider.GetComponentInChildren<IDamageable>()?.TakeDamage(_currentChargedAttackDamage);
162
163             OnWeaponHit?.Invoke();
164
165             _weaponAudioEmitter?.PlayOneShotSfx("Sword_Hit");
166
167             if (_hitVFX != null)
168             {
169                 Destroy(Instantiate(_hitVFX, _hitbox.transform.position, Quaternion.identity), 1f);
170             }
171         }
172     }
173
```

```
174     #region Normal Attack
        1 reference
175     public void PrepareAttack()
176     {
177         _normalAttackBufferCO = StartCoroutine(AttackBufferCO());
178     }
179
        1 reference
180     private IEnumerator AttackBufferCO()
181     {
182         _normalAttackBuffer = true;
183
184         yield return WaitHandler.GetWaitForSeconds(_attackBufferDuration);
185
186         _normalAttackBuffer = false;
187     }
188
        1 reference
189     public void Attack()
190     {
191         if (!NextComboInputAllowed)
192             return;
193
194         IsPerformingCombatAction = true;
195
196         _weaponAnimator.SetInteger(GameData.COMBO_HASH, _currentCombo);
197         _weaponAnimator.SetTrigger(GameData.ATTACK_HASH);
198
199         _weaponAudioEmitter?.PlayOneShotSfx("Sword_Attack");
200
201         NextComboInputAllowed = false;
202         _currentCombo = (_currentCombo + 1) % (WeaponData.MaxCombo);
203
204         _normalAttackBuffer = false;
205         StopCoroutine(_normalAttackBufferCO);
206     }
207
```

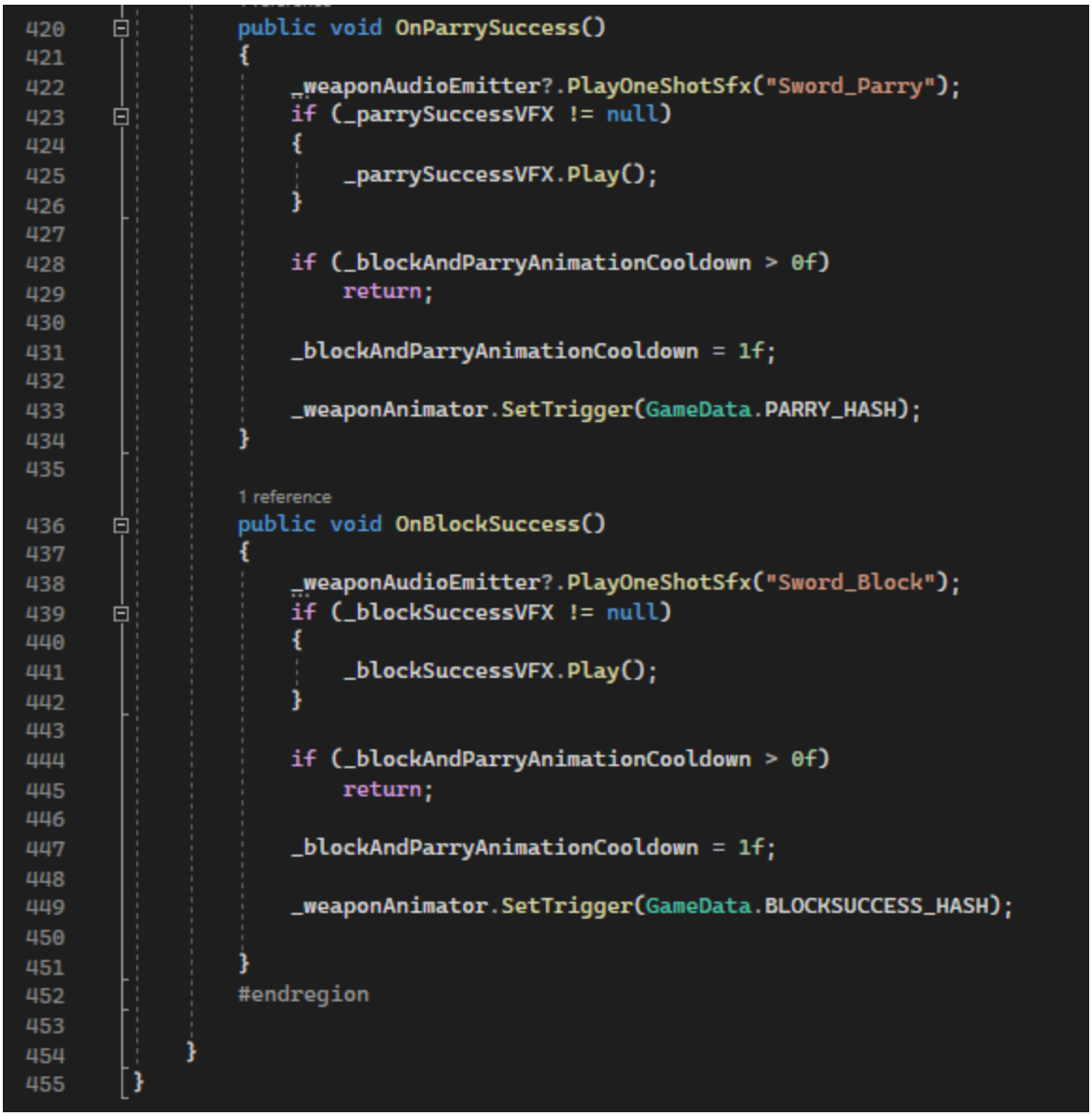
```
208         public void ActivateHitbox()
209         {
210             _hitboxEnabled = true;
211         }
212
                2 references
213         public void DeactivateHitbox()
214         {
215             _hitboxEnabled = false;
216
217             //Reset cache
218             _hitTargetCache.Clear();
219         }
220
                2 references
221         public void AllowNextComboInput()
222         {
223             NextComboInputAllowed = true;
224
225             IsPerformingCombatAction = false;
226         }
227
                0 references
228         public void ResetCombo()
229         {
230             _currentCombo = 0;
231         }
232
233         #endregion
234
235         #region Charged Attack
                1 reference
236         public void PrepareCharge()
237         {
238             //Don't allow charging if is blocking
239             if (_isBlockPressed)
240                 return;
241
242             _chargingCO = StartCoroutine(ChargingCO());
243         }
244
```

```
245         public void ReleaseCharge()
246         {
247             if (_chargingCO == null) //Not charging but attack key released
248                 return;
249 
250             StopCoroutine(_chargingCO);
251             _chargingCO = null;
252 
253             if (_currentChargeTime < WeaponData.MinChargingTime)
254             {
255                 //Cancelled due to not enough charge time
256                 return;
257             }
258             else if (_currentChargeTime >= WeaponData.MinChargingTime)
259             {
260                 //Do Charged Atatck
261                 PerformChargedAttack(_currentChargeTime - WeaponData.MinChargingTime / WeaponData.MaxChargingTime - WeaponData.MinChargingTime);
262             }
263         }
264 
                1 reference
265         public void InterruptCharging()
266         {
267             if (_chargingCO == null) //Not charging thus nothing to interrupt
268                 return;
269 
270             StopCoroutine(_chargingCO);
271             _chargingCO = null;
272 
273             _currentChargeTime = 0f;
274 
275             _weaponAnimator.SetBool(GameData.ISCHARGING_HASH, false);
276 
277             _weaponMeshRenderer.material = _originalMaterial;
278             _chargingVFX.SetActive(false);
279             _fullyChargedVFX.SetActive(false);
280 
281             IsPerformingCombatAction = false;
282         }
283 
```

```
284         private IEnumerator ChargingCO()
285         {
286             _currentChargeTime = 0f;
287 
288             while (_currentChargeTime < WeaponData.MaxChargingTime)
289             {
290                 _currentChargeTime += Time.deltaTime;
291 
292                 if (_currentChargeTime >= WeaponData.MinChargingTime)
293                 {
294                     OnChargedAttackReady();
295                 }
296 
297                 yield return null;
298             }
299 
300             //Fully Charged
301             _currentChargeTime = WeaponData.MaxChargingTime;
302             OnChargedAttackFull();
303         }
304 
                1 reference
305         public void OnChargedAttackReady()
306         {
307             IsPerformingCombatAction = true;
308 
309             _playerCombatManager.OnChargingStarted?.Invoke();
310 
311             _weaponAnimator.SetBool(GameData.ISCHARGING_HASH, true);
312 
313             _chargingVFX.SetActive(true);
314         }
315 
                1 reference
316         public void OnChargedAttackFull()
317         {
318             _playerCombatManager.OnChargingMaxed?.Invoke();
319 
320             _weaponAnimator.SetTrigger(GameData.FULLYCHARGED_HASH);
321 
322             _chargingVFX.SetActive(false);
323             _fullyChargedVFX.SetActive(true);
324 
325             _weaponMeshRenderer.material = _chargedAttackMaterial;
326         }
327 
```

```
328         public void PerformChargedAttack(float chargedPercentage)
329         {
330             _weaponAnimator.SetTrigger(GameData.CHARGERELEASED_HASH);
331             _weaponAnimator.SetBool(GameData.ISCHARGING_HASH, false);
332             _chargingVFX.SetActive(false);
333 
334             _chargedAttackHitboxEnabled = true;
335             _currentChargedAttackDamage = Mathf.Lerp(WeaponData.MinChargedAttackDamage, WeaponData.MaxChargedAttackDamage, chargedPercentage);
336 
337             _playerCombatManager.OnChargedAttackReleased?.Invoke(chargedPercentage);
338             _chargedAttackCO = StartCoroutine(ChargedAttackCO());
339         }
340 
                1 reference
341         private IEnumerator ChargedAttackCO()
342         {
343             float timeElapsed = 0f;
344 
345             while (timeElapsed <= WeaponData.ChargedAttackDuration)
346             {
347                 timeElapsed += Time.deltaTime;
348                 yield return null;
349             }
350 
351             EndChargedAttack();
352         }
353 
                1 reference
354         public void EndChargedAttack()
355         {
356             _playerCombatManager.OnChargedAttackEnded?.Invoke();
357 
358             _chargedAttackHitboxEnabled = false;
359 
360             //Reset cache
361             _hitTargetCache.Clear();
362 
363             _weaponAnimator.SetTrigger(GameData.CHARGEDATTACKENDED_HASH);
364 
365             _weaponMeshRenderer.material = _originalMaterial;
366 
367             _fullyChargedVFX.SetActive(false);
368 
369             IsPerformingCombatAction = false;
370         }
371         #endregion
```

```
373     #region Blocking
        1 reference
374     public void StartBlocking()
375     {
376         //Cancel charging if is charging
377         if (_chargingCO != null)
378         {
379             InterruptCharging();
380         }
381
382         IsPerformingCombatAction = true;
383
384         _isBlockPressed = true;
385         _currentBlockTime = 0f;
386         _weaponAnimator.SetBool(GameData.ISBlOCKING_HASH, true);
387
388
389         if (_minBlockTimeCO != null)
390         {
391             StopCoroutine(_minBlockTimeCO);
392         }
393     }
394
        2 references
395     public void StopBlocking()
396     {
397         if (_currentBlockTime < WeaponData.MinBlockTime)
398         {
399             StartCoroutine(MinBlockTimeCO());
400         }
401         else
402         {
403             _isBlockPressed = false;
404             _weaponAnimator.SetBool(GameData.ISBlOCKING_HASH, false);
405
406             IsPerformingCombatAction = false;
407         }
408     }
409
        1 reference
410     private IEnumerator MinBlockTimeCO()
411     {
412         yield return WaitHandler.GetWaitForSeconds(WeaponData.MinBlockTime - _currentBlockTime);
413
414         _isBlockPressed = false;
415         _weaponAnimator.SetBool(GameData.ISBlOCKING_HASH, false);
416
417         IsPerformingCombatAction = false;
418     }
```



```
420         public void OnParrySuccess()
421         {
422             _weaponAudioEmitter?.PlayOneShotSfx("Sword_Parry");
423             if (_parrySuccessVFX != null)
424             {
425                 _parrySuccessVFX.Play();
426             }
427
428             if (_blockAndParryAnimationCooldown > 0f)
429                 return;
430
431             _blockAndParryAnimationCooldown = 1f;
432
433             _weaponAnimator.SetTrigger(GameData.PARRY_HASH);
434         }
435
            1 reference
436         public void OnBlockSuccess()
437         {
438             _weaponAudioEmitter?.PlayOneShotSfx("Sword_Block");
439             if (_blockSuccessVFX != null)
440             {
441                 _blockSuccessVFX.Play();
442             }
443
444             if (_blockAndParryAnimationCooldown > 0f)
445                 return;
446
447             _blockAndParryAnimationCooldown = 1f;
448
449             _weaponAnimator.SetTrigger(GameData.BLOCKSUCCESS_HASH);
450
451         }
452         #endregion
453
454     }
455 }
```

Figure 13 Weapon.cs

The other important part of the combat system is the high flexible state machine system implemented for the enemy AIs. This is achieved by thoroughly **decoupling of the state actions and conditions**, making them easily replaceable. For example, the skeleton enemy continues pursuing the player after performing an attack while the wizard enemy will temporarily retreat after performing an attack. This also works with multiple conditions, such as the boss entering the second phase when its health is depleted for the first time but dies when the health is depleted the second time. It is worth noting that although the player controller also utilizes a state machine, it is a different implementation compared to the enemies because the flexible state machine is developed during the creation of the enemies.

To start implementing an enemy, the developers has to create classes with their respective action. The states used in this implementation are merely a set of actions that will be performed when it is entered, exited or during update or fixed update. Since states are not inheriting from Monobehaviour, the state machine is responsible for triggering these callbacks. The state machine is a Monobehaviour class that stores the current state to run these said callbacks, as well as storing a list of special transitions that any state can transition to.

Next, the developer must create a new class inheriting from the StateMachine class. The creation of the AI takes 4 steps:

1. State initialization, where states are created using their respective constructors and given their parameters. For instance, when an idle state is created, the following code is written:

   ```
   _idleState = new IdleState(entity, this);
   ```

   Where entity is an extra parameter that allows the state to access the entity class attached on the game object to read certain data. Other types of parameters can also be passed during construction of the states, for example:

   ```
   _chaseState = new ChaseState(entity, this, _chaseSpeed);
   ```

   The chase state can receive a speed parameter, determining how fast the enemy is able to chase the player. The parameter can also be references, such as the grabbed state,

   ```
   _grabbedState = new GrabbedState(entity, this, _grabbableObject);
   ```

   Where the respective grabbable object class is passed into the state so that it knows which object to call grab-related functions on.

2. Condition Initialization, where condition classes are created. Conditions are simple classes that has a single purpose: return a boolean. If this boolean is true, that means that a transition needs to take place. This will be further explained in the next step. The conditions can return the boolean in many ways, for example, the timer class starts a timer and always return false, only to return true when the timer has expired.

```
public class Condition_Timer : Condition
{
    private float _startTime;
    public readonly float Duration;

    private Action _startCallback = null;
    private Action _endCallback = null;


    10 references
    public Condition_Timer(float duration, Action startCallback = null, Action endCallback = null)
    {
        this.Duration = duration;

        this._startCallback = startCallback;
        this._endCallback = endCallback;
    }

    12 references
    public override void Enter()
    {
        base.Enter();

        _startTime = Time.time;
        _startCallback?.Invoke();
    }

    10 references
    public override void Exit()
    {
        base.Exit();

        _endCallback?.Invoke();
    }

    3 references
    public override bool Evaluate()
    {
        return Time.time > _startTime + Duration;
    }
}
```

Figure 14 Timer Condition

On the other hand, Condition_HealthDepleted holds a reference to the entity's health module and only returns true when the health is depleted, which is useful for triggering entity death.

```
public class Condition_HealthDepleted : Condition
{
    private HealthModule _healthModule;
    private bool _healthDepleted;

    5 references
    public Condition_HealthDepleted(HealthModule healthModule)
    {
        this._healthModule = healthModule;
    }

    12 references
    public override void Enter()
    {
        base.Enter();

        this._healthModule.OnHealthDepleted.AddListener(OnHealthDepleted);

        _healthDepleted = false;
    }

    10 references
    public override void Exit()
    {
        base.Exit();

        this._healthModule.OnHealthDepleted.RemoveListener(OnHealthDepleted);

        _healthDepleted = false;
    }

    2 references
    private void OnHealthDepleted()
    {
        _healthDepleted = true;
    }

    3 references
    public override bool Evaluate()
    {
        return _healthDepleted;
    }
}
```

Figure 15 Health Depleted Condition

3. Setting up transitions, where the previously constructed states and conditions are connected together to form transitions. For example, idle state transitions to the chase state when the player is in range when the following transition is added:

```
_idleState.AddTransition(_chaseState, _playerInDetectionRangeCondition);
```

This is also where AnyTransitions can be added to the state machine, where any state can transition into the given state when the condition is reached.

```
this.AddAnyTransition(_deathState, _healthDepletedCondition);
```

4. And last but not least, initializing the state machine. This simply tells the state machine to set the given state as the first state to be in and call the respective callbacks.

```
Initialize(_idleState);
```

Figure below shows the state machine flow applied on the skeleton enemy, which is one of the most common enemies in the game.

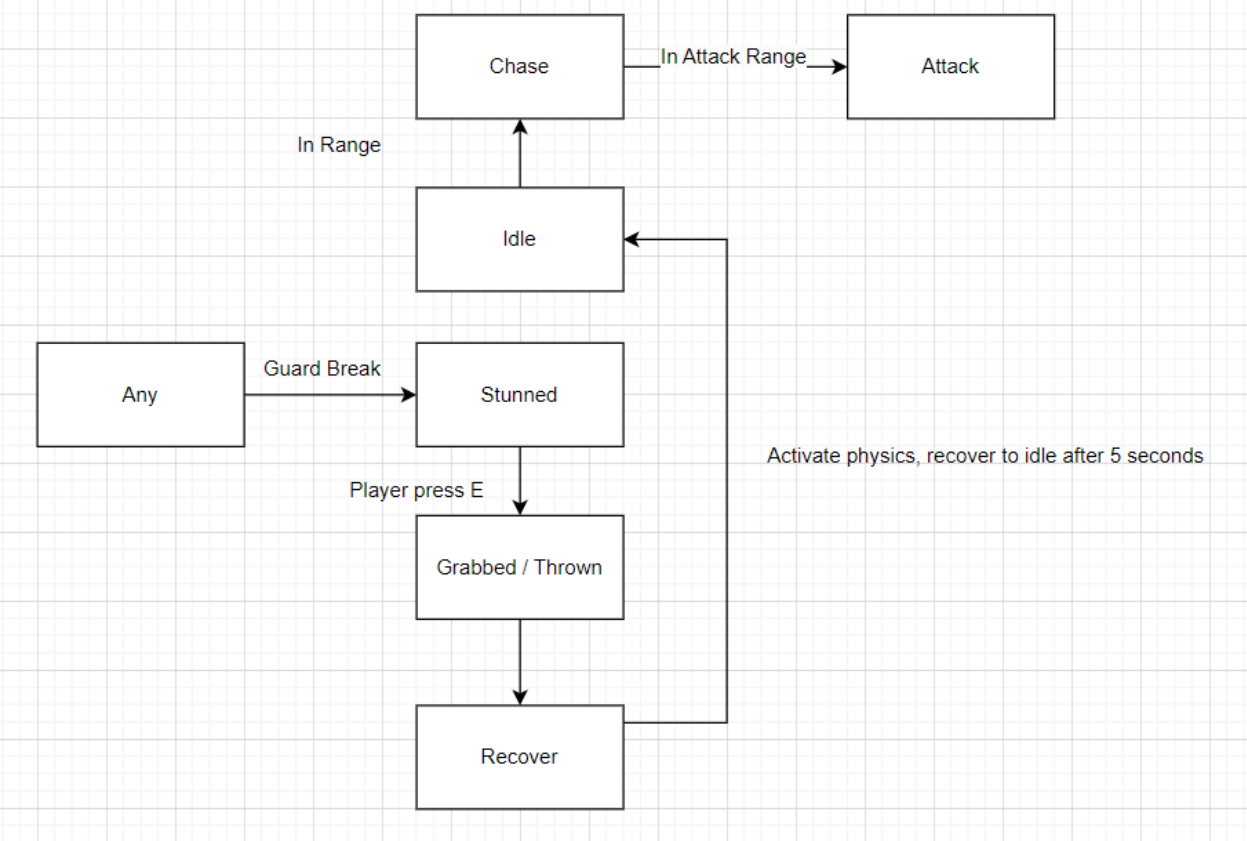


Figure 16 Skeleton State Machine Flow

The state machine of the skeleton is set up this way:

```
private void Awake()
{
    //State Machine Initialization
    //1. State Initialization
    _idleState = new IdleState(entity, this);
    _chaseState = new ChaseState(entity, this, _chaseSpeed);
    _meleeAttackState = new MeleeAttackState(entity, this, meleeAttack);
    _stunnedState = new StunnedState(entity, this, _grabbableObject);
    _grabbedState = new GrabbedState(entity, this, _grabbableObject);
    _recoverState = new RecoverState(entity, this, _grabbableObject);
    _deathState = new DeathState(entity, this, _ragdollController);

    //2. Condition Initialization
    _playerInDetectionRangeCondition = new Condition_PlayerInRange(_playerDetector);
    _playerOutOfDetectionRangeCondition = new Condition_PlayerOutOfRange(_playerDetector);
    _playerInAttackRangeCondition = new Condition_PlayerInRange(_attackRangeDetector);
    _meleeAttackTimerCondition = new Condition_Timer(_meleeAttackDuration);
    _guardBrokenCondition = new Condition_GuardBroken(this.ModuleManager.GetModule<GuardModule>());
    _stunTimerCondition = new Condition_Timer(entity.EntityData.BaseStunDuration);
    _grabbedCondition = new Condition_Grabbed(_grabbableObject);
    _thrownTerrainCollisionCondition = new Condition_ThrownTerrainCollision(_grabbableObject);
    _recoverAnimationFinishedCondition = new Condition_OnAnimationFinished(Animator);
    _healthDepletedCondition = new Condition_HealthDepleted(this.ModuleManager.GetModule<HealthModule>());

    //3. Setting up transitions
    _idleState.AddTransition(_chaseState, _playerInDetectionRangeCondition);

    _chaseState.AddTransition(_idleState, _playerOutOfDetectionRangeCondition);
    _chaseState.AddTransition(_meleeAttackState, _playerInAttackRangeCondition);

    _meleeAttackState.AddTransition(_chaseState, _meleeAttackTimerCondition);

    _stunnedState.AddTransition(_idleState, _stunTimerCondition);
    _stunnedState.AddTransition(_grabbedState, _grabbedCondition);

    _grabbedState.AddTransition(_recoverState, _thrownTerrainCollisionCondition);

    _recoverState.AddTransition(_idleState, _recoverAnimationFinishedCondition);

    this.AddAnyTransition(_stunnedState, _guardBrokenCondition);
    this.AddAnyTransition(_deathState, _healthDepletedCondition);

    Initialize(_idleState);
}
```

Figure 17 Skeleton enemy state machine

To sum it up, this creates an extremely flexible system where each states can have various different condition. For example, an enemy might transition into a retreat state instead of chase state when the player is nearby, and the boss can transition into a new phase instead of dying when their health is depleted. Multiple conditions can also be passed in as an array of condition, and it will act like a and operation where all of the conditions needs to be fulfilled for the transition to happen.

4.3 Interface Design

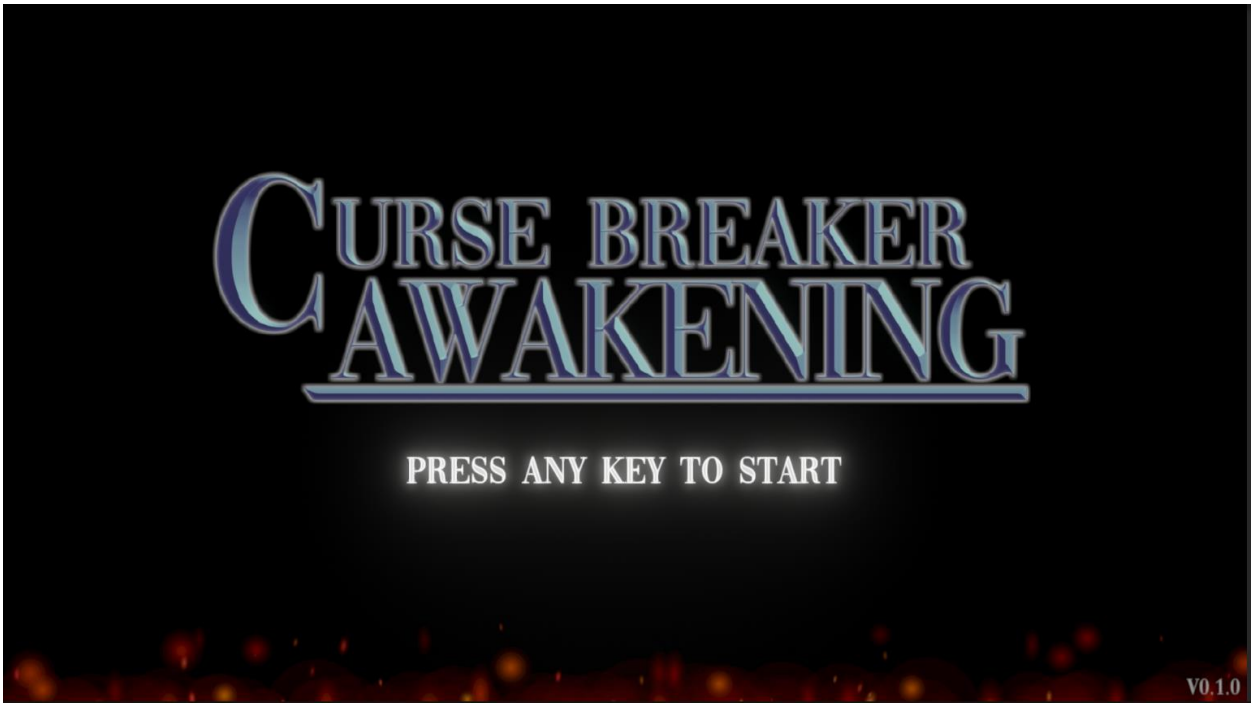


Figure 18 Startup Screen

When the player boots up the game, they will first be greeted by this startup screen. It reveals the game title as well as the version number (lower right corner), as well as prompting the player to press any key to start the game. The title shines occasionally and a fire particle effect is also added at the bottom of the screen to add a sense of dynamic to the otherwise static screen. The key prompt is also animated to fade in and out repeatedly as it is the most important element in the screen besides the title, prompting the player what to do to continue the game.

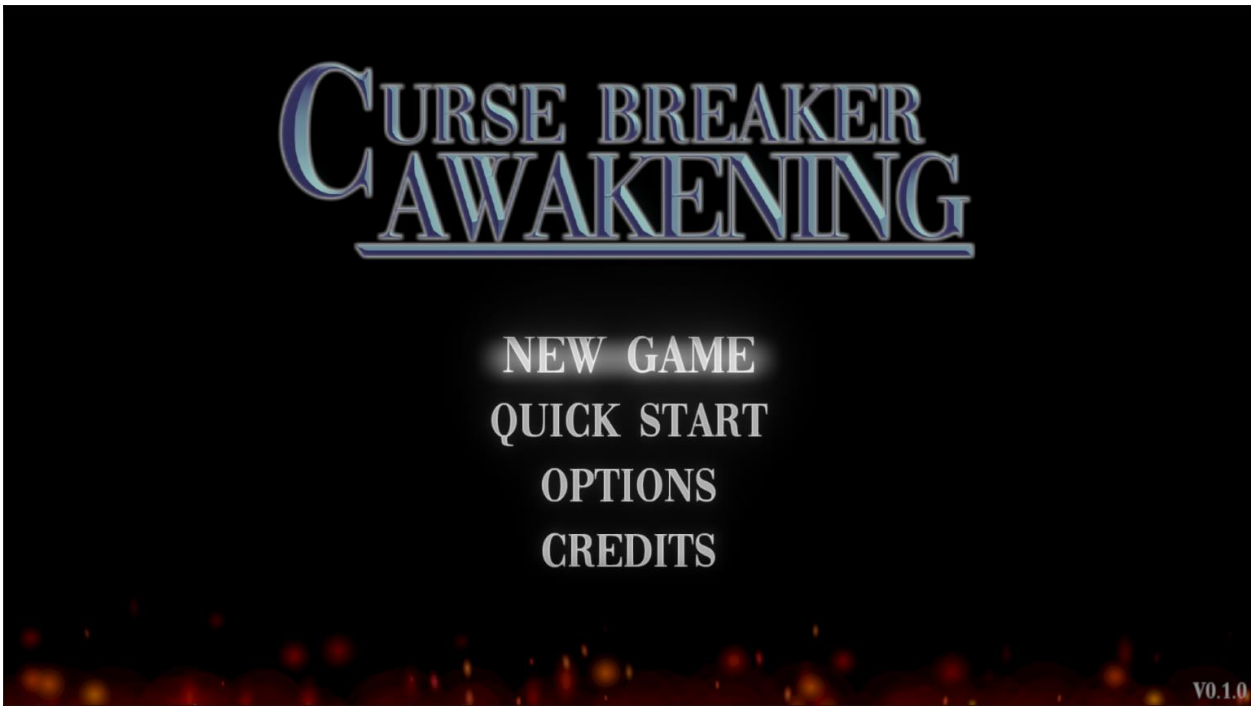


Figure 19 Main Menu

After pressing any key as prompted, the player will enter the main menu and be given 5 options. In this menu, they can choose to:

1. New Game: Start a new game, entering the cutscene sequence.
2. Quick Start: Immediately start a new run, skipping the cutscene.
3. Options: Open the options menu.
4. Credits: Show credits.
5. Exit: Quit the game application.

The exit button is not visible in the figure above, but as the player navigate through the buttons the buttons will be scrolled automatically to reveal more options bellow thanks to a custom script. It is possible to navigate through the UI both by using a mouse or keyboard/gamepad buttons, thanks to the built-in input system module provided by Unity.

Selected buttons will also play a sound as well as being animated to grow slightly in size which is done by adding a simple script that utilizes the DOTween tweening library to dynamically animate the scale of the buttons.

```
Unity Script (2 asset references) | 2 references
public class ButtonScrollList : MonoBehaviour
{
    [SerializeField] private List<CustomButton> _buttons;
    [SerializeField] private CanvasGroup _buttonsCanvasGroup;
    [FormerlySerializedAs("_scrollRect")][SerializeField] private ScrollRect _scrollRect;
    [SerializeField] private RectTransform _contentRect;

    private const float _buttonsTweenDuration = 1f;

    Unity Message | 0 references
    private void OnEnable()
    {
        foreach (var button in _buttons)
        {
            button.OnSelected += () => ScrollToButton(button);
        }

        EventSystem.current.SetSelectedGameObject(_buttons[0].gameObject);
    }

    Unity Message | 0 references
    private void OnDisable()
    {
        foreach (var button in _buttons)
        {
            button.OnSelected += () => ScrollToButton(button);
        }
    }

    2 references
    public void ShowButtons()
    {
        _buttonsCanvasGroup.DOFade(1, _buttonsTweenDuration).OnComplete(() => _buttonsCanvasGroup.interactable = true);
    }

    4 references
    public void SetInteractable(bool interactable)
    {
        _buttonsCanvasGroup.interactable = interactable;
    }

    public void DisableButtons()
    {
        _buttonsCanvasGroup.alpha = 0f;
        _buttonsCanvasGroup.interactable = false;
    }


    private Tween _scrollTween = null;
    2 references
    public void ScrollToButton(CustomButton button)
    {
        float targetValue = 1 - (float)_buttons.IndexOf(button) / _buttons.Count;

        if (_scrollTween != null)
            _scrollTween.Kill();

        _scrollTween = DOTween.To(() => _scrollRect.verticalNormalizedPosition, x => _scrollRect.verticalNormalizedPosition = x, targetValue, 0.2f);

    }
}
```

Figure 20 ButtonScrollList.cs

```
public class CustomButton : MonoBehaviour, ISelectHandler, IDeselectHandler
{
    private TextMeshProUGUI _textMeshProUGUI;
    1 reference
    public RectTransform RectTransform {  get; private set; }

    public const float _tweenDuration = 0.2f;
    public const float _selectedScale = 1.1f;
    public const float _selectedAlpha = 1f;
    public const float _deselectedAlpha = 0.5f;

    public event Action OnSelected;

    Unity Message | 0 references
    private void Awake()
    {
        RectTransform = GetComponent<RectTransform>();
        _textMeshProUGUI = GetComponentInChildren<TextMeshProUGUI>();
    }

    0 references
    public void OnSelect(BaseEventData eventData)
    {
        _textMeshProUGUI.DOKill();
        _textMeshProUGUI.rectTransform.DOScale(_selectedScale, _tweenDuration);
        _textMeshProUGUI.alpha = _selectedAlpha;

        AudioManager.Instance.PlayGlobalSFX("Menu_Select");

        OnSelected?.Invoke();
    }

    0 references
    public void OnDeselect(BaseEventData eventData)
    {
        _textMeshProUGUI.DOKill();
        _textMeshProUGUI.rectTransform.DOScale(1f, _tweenDuration);
        _textMeshProUGUI.alpha = _deselectedAlpha;
    }
}
```

Figure 21 CustomButton.cs



Figure 22 In Game HUD

Figure above showcases the in-game Head-Up-Display (HUD), which is the UI element that the player will be seeing the most. In the developer's opinion, it is extremely important to keep this simplistic and prevent obstructing the player's vision at all cost considering this is a fast-paced action game, as such design can directly affect the players' performance in game and cause their frustration.

The final design of the HUD features simple health and stamina bars on the top left of the screen with an avatar image of the player. The health and stamina are crucial information that the player should keep track of at all times. To make it more obvious when the player gets hit (especially from behind), the avatar and the health bar will also be shaken using DOTween and flashes red in color. When the player is on low health, the avatar will keep flashing in red color, signaling the player to be more careful and search for a healing source as soon as possible.

On the top right corner is a **minimap** that displays the current room that the player is currently within. The green arrow represents the player, and it is important that the player is represented by something that can indicate direction, which helps a lot in helping the player differentiating their current facing direction. The edges of the minimap are also blurred to reduce obstruction so that

the player will not easily be obstructed, yet maintaining a reasonable size so that the player can easily see their whereabouts when they intend to.

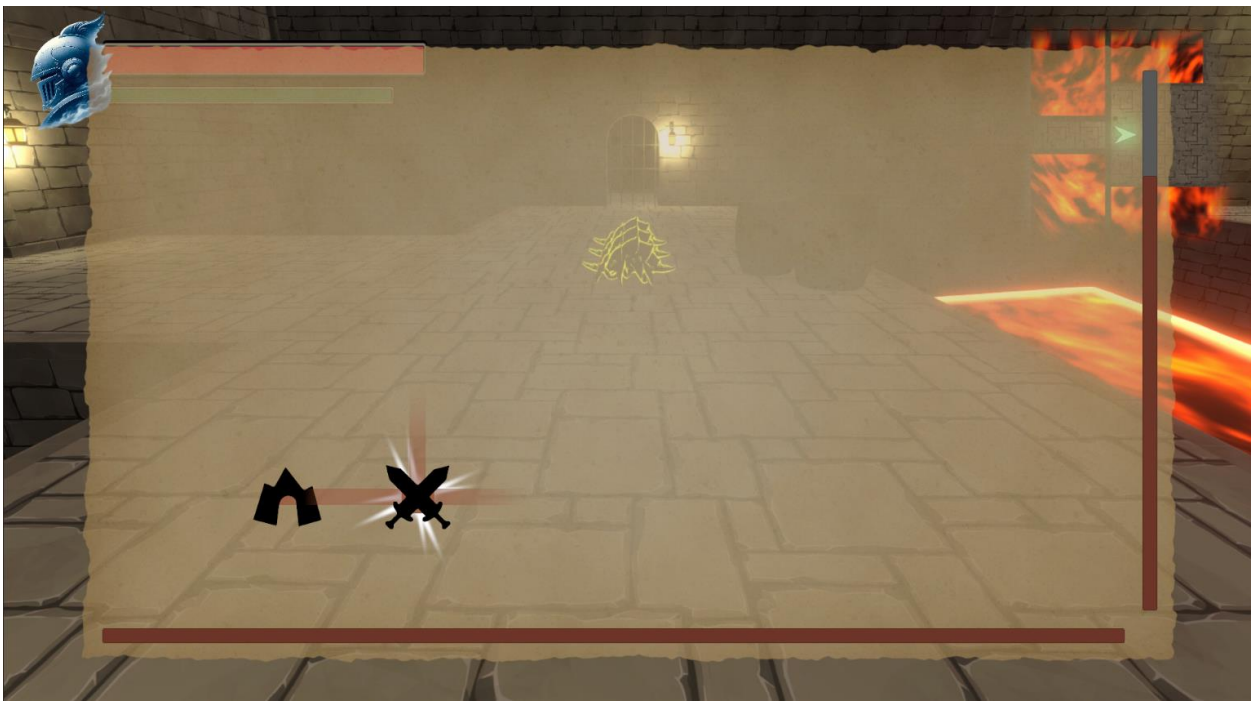


Figure 23 Map (Partially Revealed)

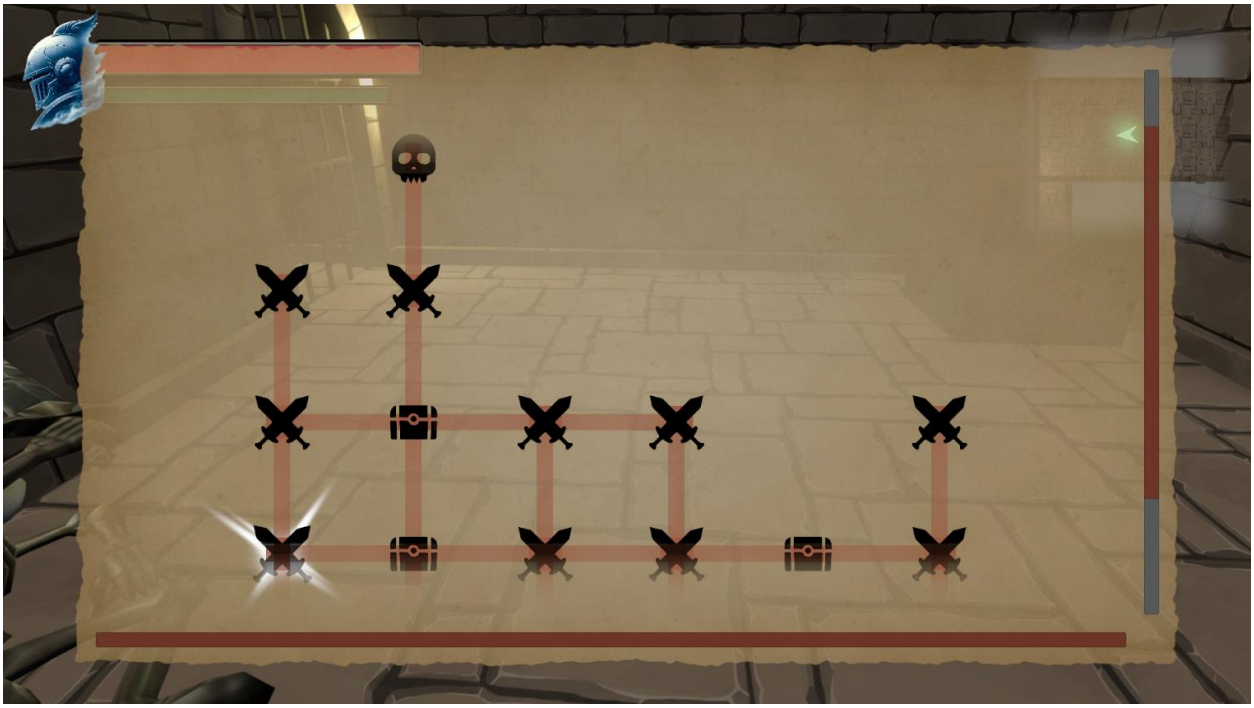


Figure 24 Map (Fully Revealed)

Pressing M in game opens up the map, showing the all of the discovered rooms and the current room that the player is in. The current room is indicated with the white star behind the room icons. Each type of rooms also has a different icon to make them distinguishable at first glance. Sword icons represent normal rooms, treasure chest represents treasure, and the skull icon represents the boss room. It is worth noting that opening the map does not pause the game and the map is semitransparent, which is a design choice to not interrupt the movement flow of the player, allowing them to move around even with the map opened.

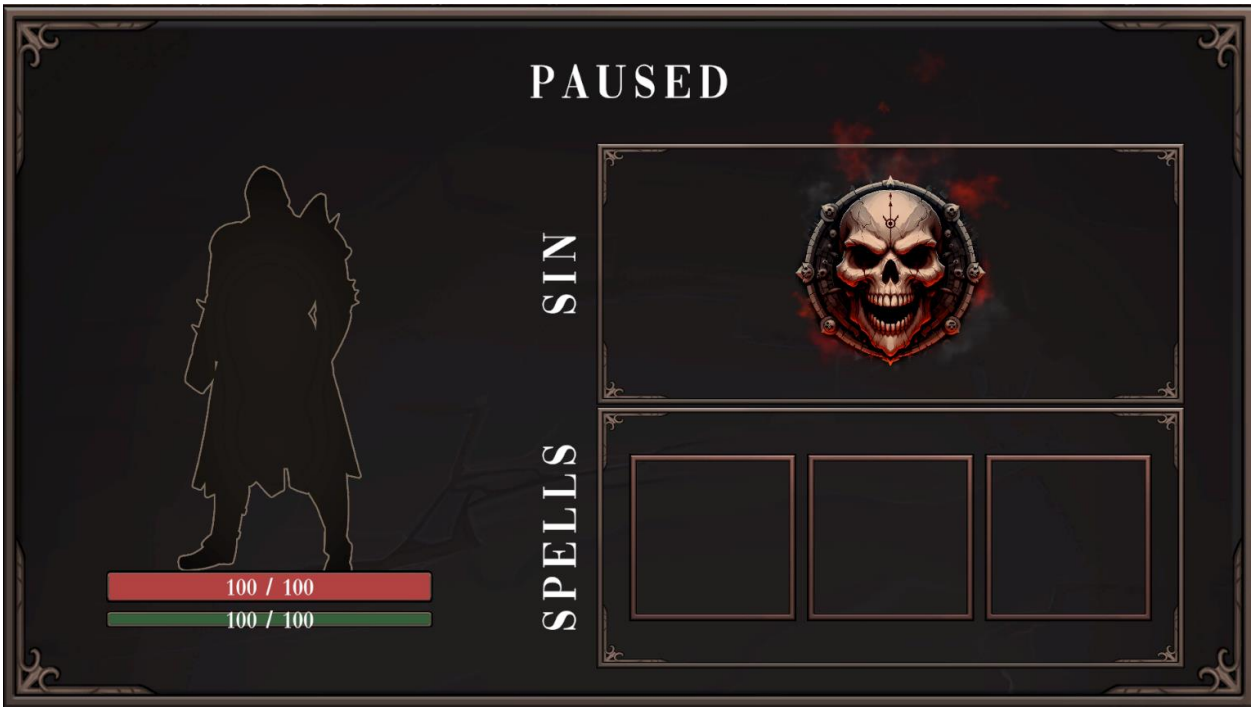


Figure 25 Pause Screen

The pause screen features a simple status section on the left, showing the actual values of the current health and stamina of the player. The right side showcases a sin meter and a spells inventory, which their corresponding systems has not yet been implemented.



Figure 26 Death Screen

Last but not least, the death screen shows up when the player's health reaches zero. The buttons only appear after a short animation to prevent players from not having reacted to their death and choosing a button by accident. Three simple options are provided, which are either to quickly restart a new run, return to the main menu or quit the game application.

4.4 Execution

After developing the level generation algorithm mentioned above, the developer created a number of room prefabs to be generated via the algorithm. Follow figures showcase all the rooms made to be generated into the actual game. Enemies (the blue rigs) and props are also placed into the levels to add challenges and rewards for the player to experience.

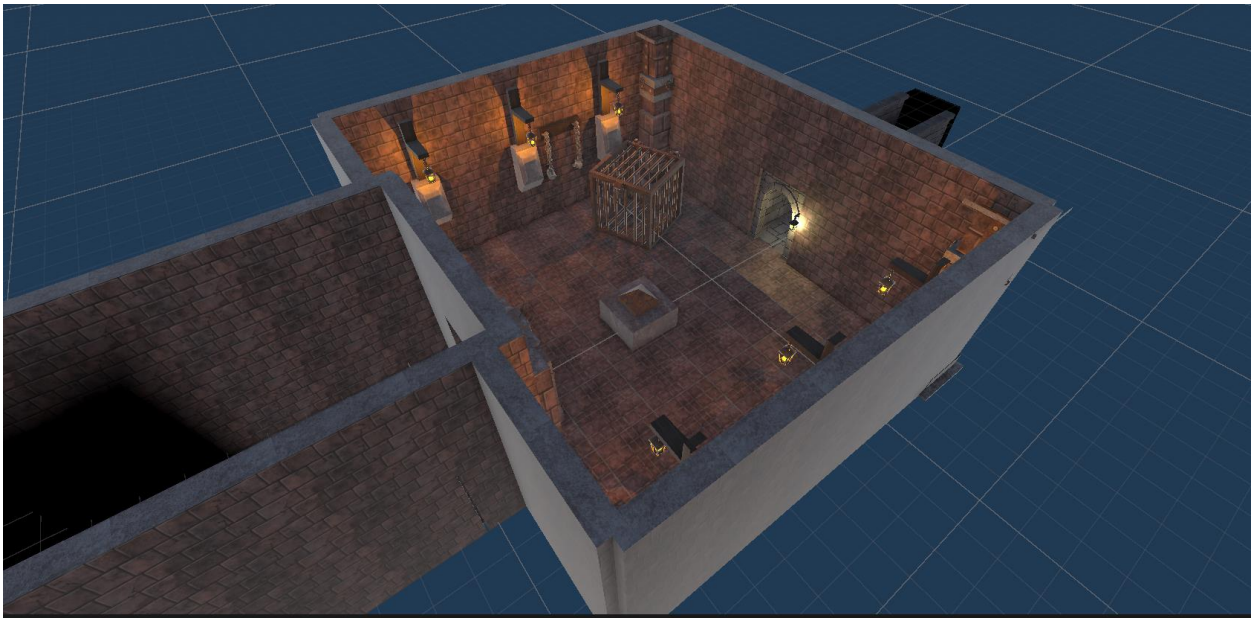
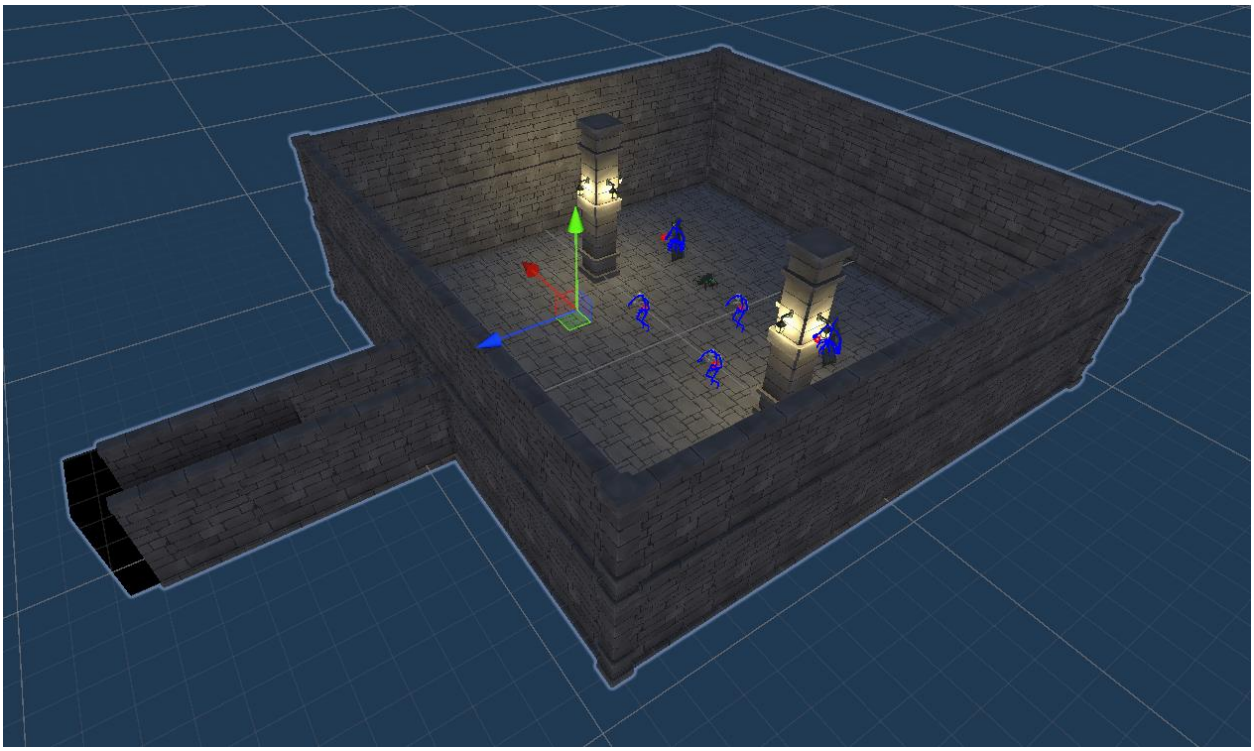
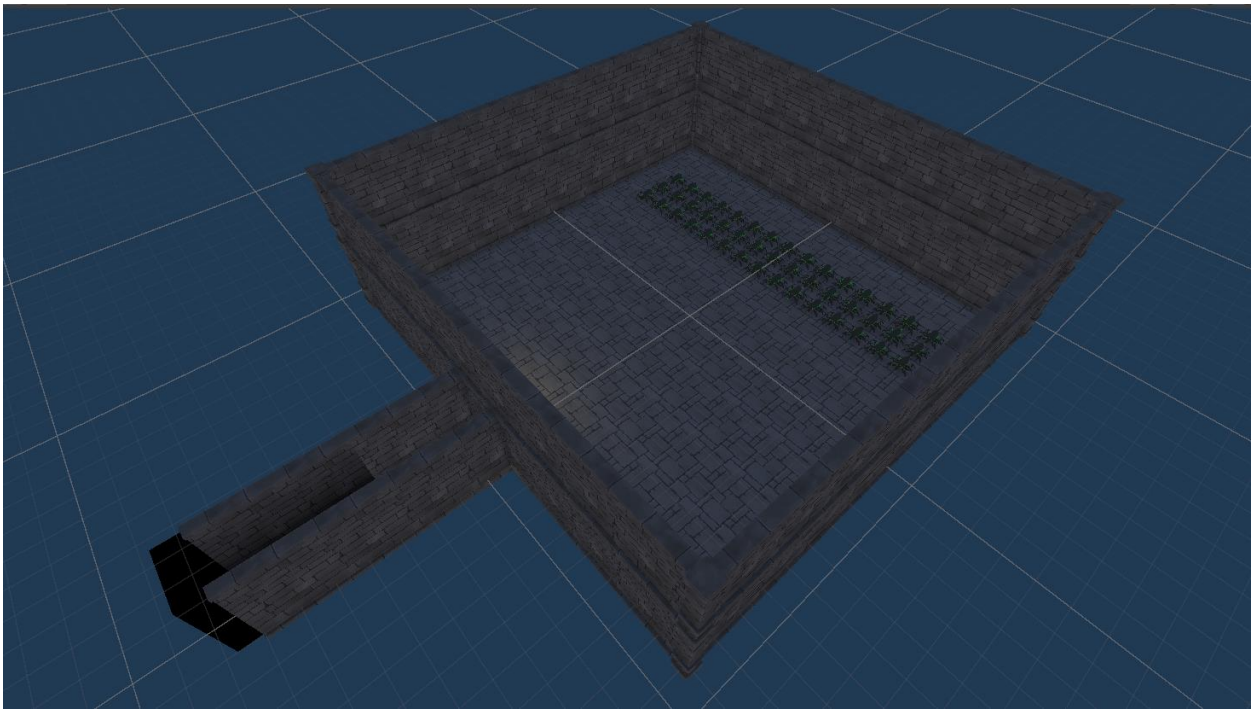


Figure 27 Entrance / Boss room





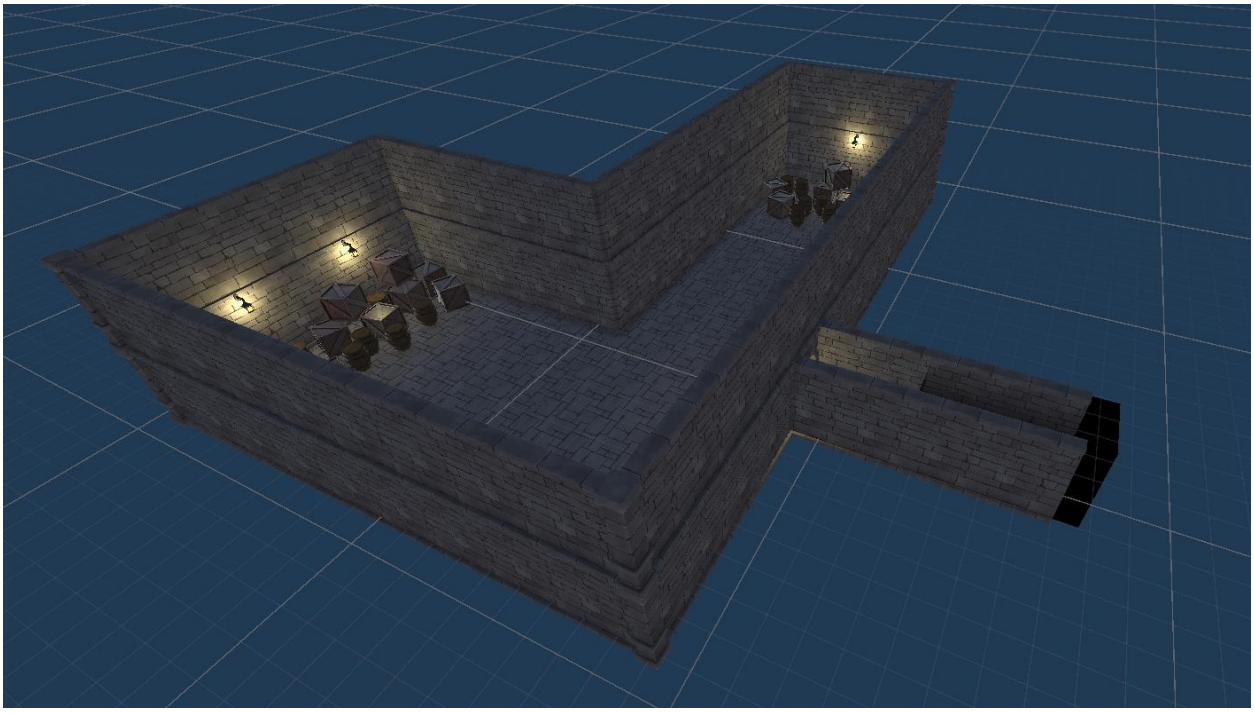


Figure 28 Single Exit Rooms

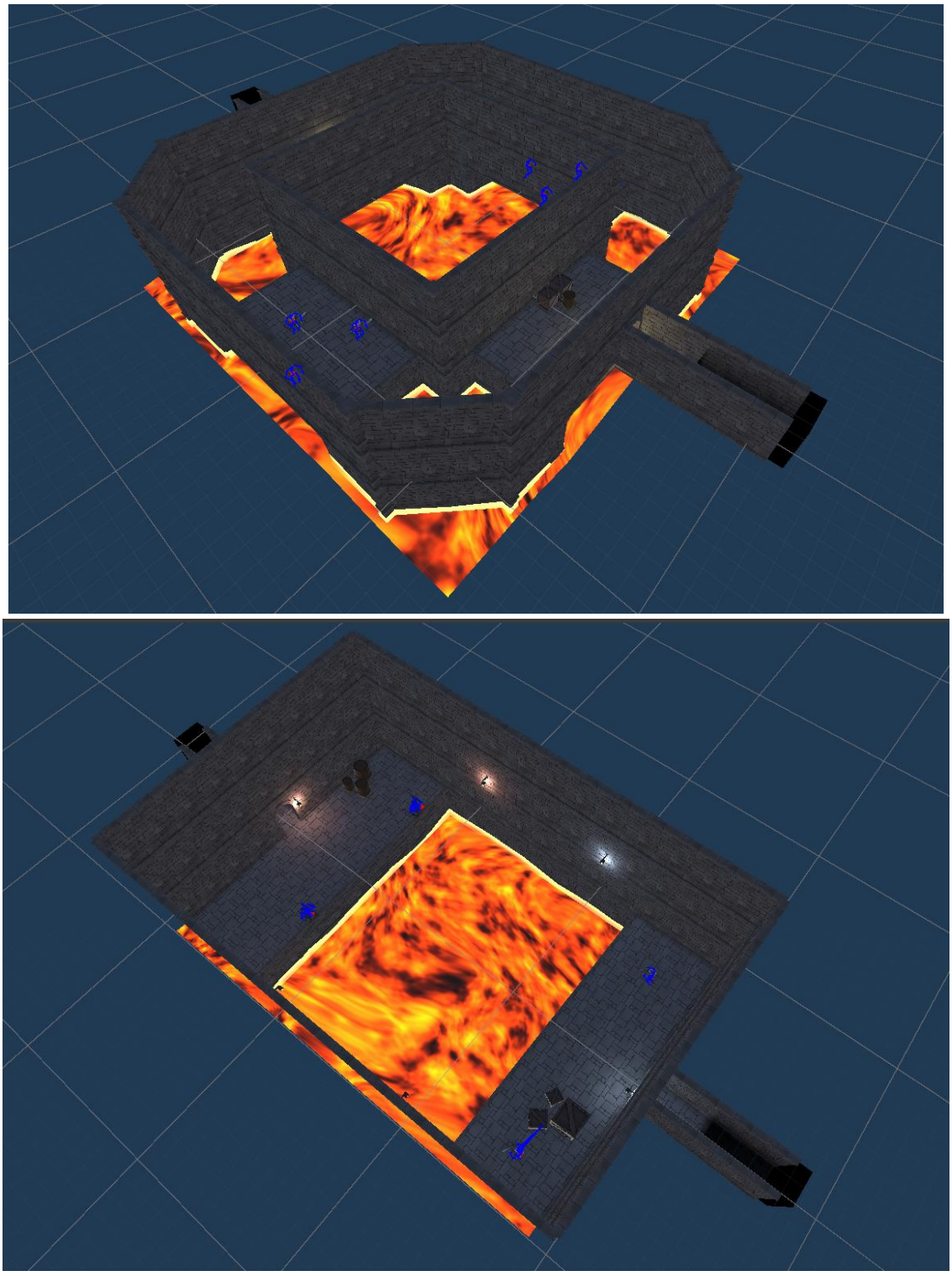


Figure 29 Two Exits (I-Shaped) Rooms

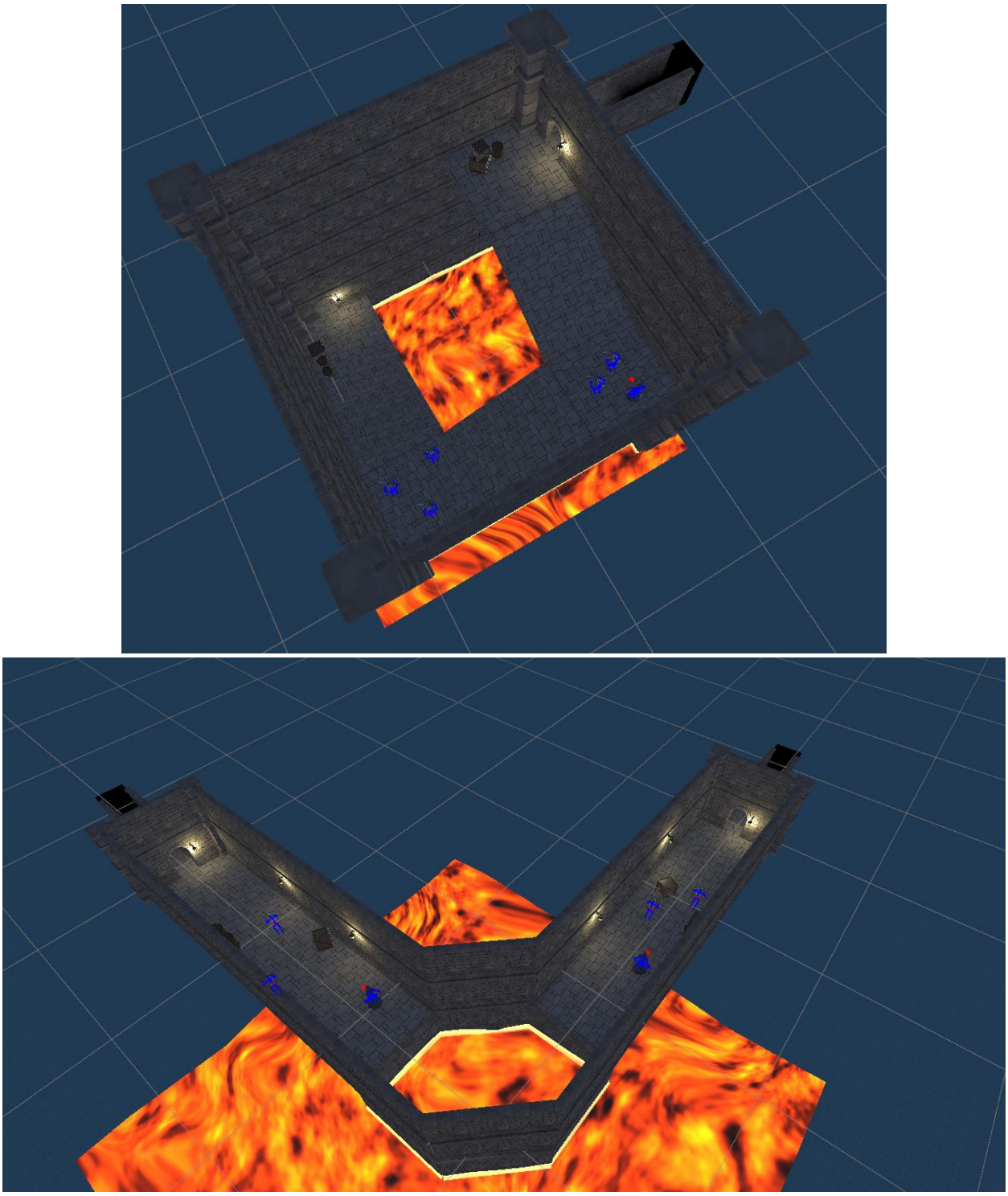


Figure 30 Two Exits (L-Shaped) Rooms

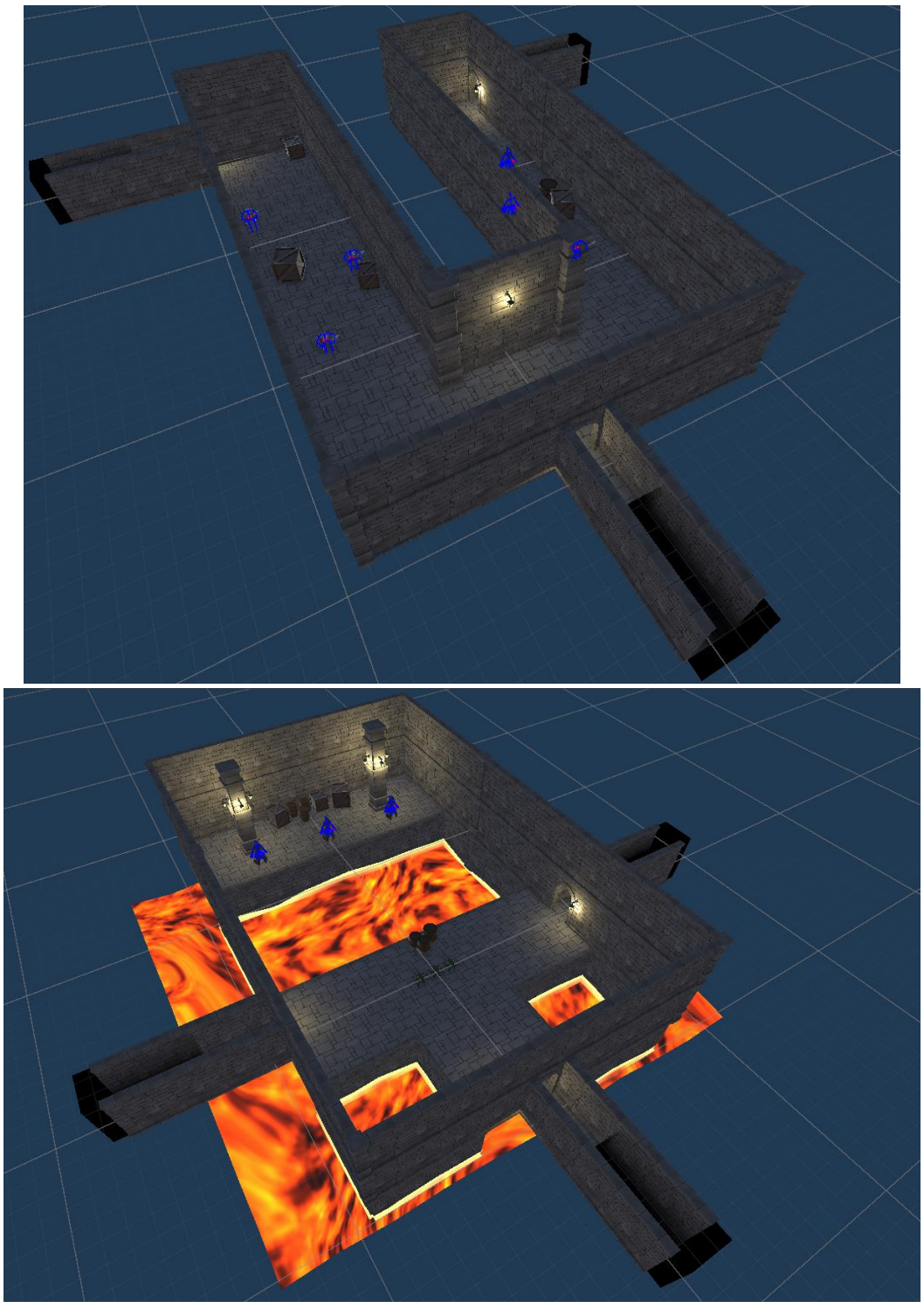


Figure 31 3 Exits (T-Shaped) Rooms

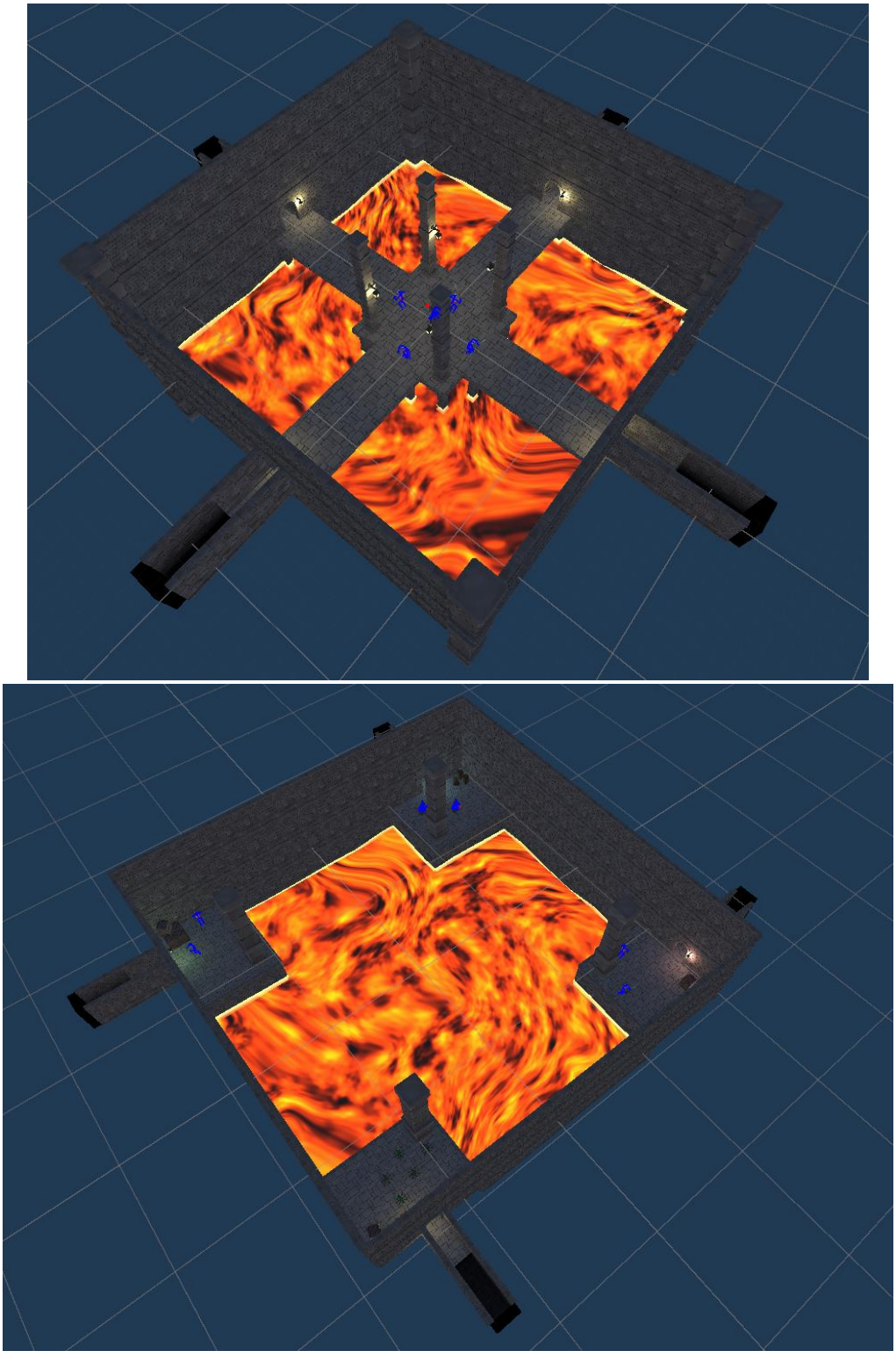


Figure 32 Four Exits Rooms

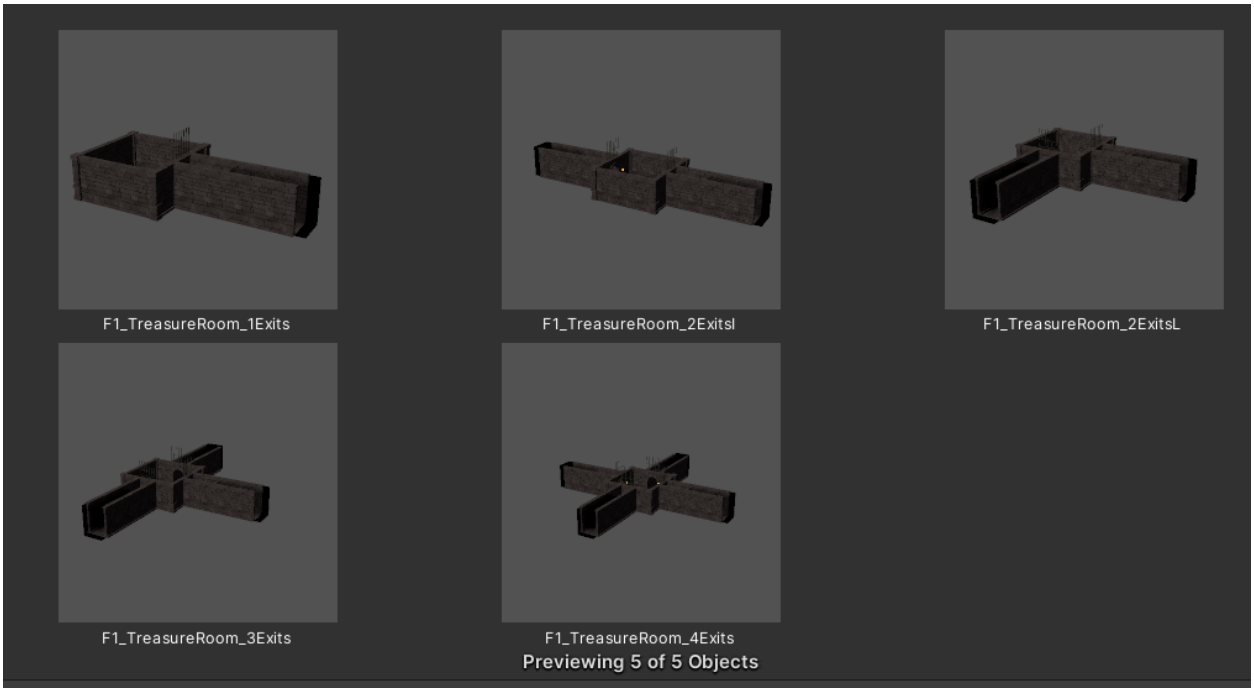


Figure 33 Treasure Rooms

4.5 Screenshots



















Figure 34 Screenshots of the Games

4.6 Summary

This chapter gives a thorough overview on how various systems within the game is designed, implemented and lastly executed, the thought process as well as reasons behind each design decisions and choices. This includes the designing process of core systems such as the level generation algorithm, the player controller and more.

The level generation algorithm is undoubtedly the most important part among all of the systems but also the hardest to implement. It has undergone several iterations and compromises before settling as what it now is. To put the final result in simple words: it uses a depth-first search algorithm to generate paths on a board, marking each of the cells and recording data such as whether they have been visited and which direction do they have openings connecting to other cells. These cells are then used to generate rooms while the connections are converted into information to determine the exits. Physical rooms are then spawned in via the Room Spawner script, creating the actual level that the game will take place in.

The player controller is also a core part of the game considering the game is focused on fast paced combat. It took a lot of careful design choices and refinement to create the desired controllability. The developer hopes to continue improving the controller to make it even more satisfying to play with in the future.

The combat system revolves around two parts: the player's combat manager system and the enemies flexible AI state machine system. The player combat manager is able to generate and equip the player with the correct weapon, handle input, as well as broadcasting weapon events such as charged attack related events for the player controller to react accordingly. The enemy AI on the other hand utilizes a flexible AI state machine that allows for easily replaceable conditions, and the modular spell system further adds to the convenience of using this system.

In conclusion, it has taken a lot of technical research as well as repeated iterations of implementation to develop all the core systems needed for the game. The developer is satisfied with the outcome, although there are definitely a lot of parts that can be further improved upon.

Chapter 5: Result and Discussion

5.1 Introduction

This chapter focuses on explaining how the developer of this project perform testing on the game project based on a number of different ways through both internal and external assistance. Testing is extremely important for such a large project in order to prevent critical bugs and errors that can potentially render the game unplayable and break player immersion, making the experience terrible for the players. To prevent this, excessive amount of testing is performed, and external help is also acquired by the developer to ensure a more subjective view of the problems that exist within the project and to modify the project according to user expectations.

5.2 Testing Design / Plan

Two different testing methods has been designed and put into use for this project, one internal and one external. The internal testing method used is Unit Testing, while the external testing method used is User Acceptance Testing (UAT).

Unit testing is a common way of testing software or games, where developers break down the systems into smaller pieces and perform testing on each of these individual pieces and writing test cases (BasuMallick, C, 2022). It is one of the most common and effective quality assurance techniques as it excels in breaking down huge systems into smaller chunks that can be easily performed analyzation.

The unit test case form used to perform testing on various aspects in the game are as follow:

Step	Test Steps	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1					
2					
3					
4					
5					

On the other hand, User Acceptance Testing is usually performed at the final stages of the development of a software or game, where a pre-release build of the project is distributed to a smaller group of users in order to collect their feedback (Eyal Elazar, 2023). The purpose of performing a user acceptance test is to ensure that the product work as intended as if it is distributed into the market. It is chosen for this project as the developer thinks that it is crucial that the experience provided by the game is as expected and there are no serious bugs that might require blind-testing to discover, considering that it is easy for developers to not notice these errors due to having a subjective view of the game's inner working.

The user acceptance testing form is designed and implemented as below:

User Acceptance Testing for Curse Breaker: Awakening						
Tester name:						
Occupation:						
Testing date:						
Start time:						
End time:						
Components	Grade (1 – Poor, 5 - Outstanding)					
	1	2	3	4	5	
1. Fulfilling objectives						
a. Overall fun factor of the game						
b. Successful implementation of meaningful randomization to create replayability in the game						
c. The cohesiveness / immersiveness created by the graphics / audio						
d. Positive feeling of unexpectedness within the game, including both randomized / non-randomized elements						

2. Bugs free					
3. Text legibility					
4. Graphics quality					
5. Audio quality					
6. Story quality					
7. Gameplay quality					
8. Performance / Optimization					
9. Overall user experience					
Feedback from tester:					
Action(s) to be taken by developer:					

This form is designed to focus on acquiring feedback on how the testers feels towards the concrete implementation of randomization systems within the game. The randomization is aimed towards creating fun, unexpectedness and replayability, and thus should receive a positive response, or it will mean that the randomness introduced to the game is either unnecessary or frustrating, causing the opposite effect from what is desired.

The additional feedback section below also allow the user to provide specific feedback that does not fall into any of the categories above or if there is anything they wish to add as well as how they expect the game to be improved, and the actions to be taken by developer section is where the developer will write down how they plan to make changes to the project according to the feedback received.

5.3 System Testing and Discussion

5.3.1 Unit Testing

Test Case 1: Validating the level generation algorithm

Step	Test Steps	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Run the game	The level generation algorithm start running on game start with an artificial delay added between each iteration of the algorithm	The algorithm starts generating the level	The algorithm starts generating the level	Pass
2	Wait for the algorithm to reach the end of path for the first time	The level generation algorithm continues until it reaches the end of a path	The algorithm starts backtracking	The algorithm starts backtracking	Pass
3	Observe the how the algorithm creates further connections	The level algorithm tries another path while generating new connections along the way	New rooms and connections are successfully generated	New rooms and connections are successfully generated	Pass
4	Observe the algorithm until it ends	The algorithm will end either when it reaches its starting point (stack emptied) or when the max iteration count is reached	The algorithm ends either by reaching starting point or by reaching max iterations	The algorithm ends upon reaching starting point	Pass

5	Observe the updating of the room shapes and rotations via the generation of testing prefabs	Custom data such as shape and rotation of the rooms are decided after the algorithm ends	Testing prefabs are generated with the correct shape and rotation	Testing prefabs are generated with the correct shape and rotation	Pass

Test Case 2: Testing the parry mechanic of the player

Step	Test Steps	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Start the game and locate the enemy	Locate any enemy to test the parry mechanic	Enemy engaged in combat upon detecting player	Enemy engaged in combat upon detecting player	Pass
2	Wait for enemy to perform attack	The parry can be performed only when the player blocks right before getting hit	Enemy performed an attack after closing in on the player	Enemy performed an attack after closing in on the player	Pass
3	Press the block button to perform a parry	Prepare to parry by blocking right before getting hit	Enemy attack parried successfully	Enemy attack parried successfully, but when attacked but more than one enemy only the first attack is parried	Pass, but require further observation and improvements

Test Case 3: Testing the BGM crossfading feature of the Audio Manager

Step	Test Steps	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Start the game and listen to the audio	The exploration theme should be played by the Audio Manager when the player is not in combat	The exploration theme is played	The exploration theme is played	Pass
2	Enter a room with enemies	The battle theme should be transitioned into via crossfading when entering combat	The exploration theme smoothly transitions into the combat theme	The exploration theme smoothly transitions into the combat theme	Pass
3	Defeat all enemies and observe the change in audio	Once all enemies in the room are cleared, the music should crossfade back into the exploration theme and continue where it left off	The combat theme smoothly transitions into the exploration theme and continues from where it left off	The combat theme smoothly transitions into the exploration theme and continues from where it left off	Pass
4	Return to the main menu	The current music should fade away, and the main menu theme should play upon entering main menu	The main menu theme is played successfully	The main menu theme is played successfully but is partly skipped	Fail, as the main menu theme being skipped partially is due to the crossfading using two audio sources but the wrong audio source's time is being reset. This issue is fixed by the developer after this test,

Test Case 4: Testing the modular spell system of the boss enemy

Step	Test Steps	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Set up the spell manager and enter the boss fight	The boss is given 4 different spells, including shield, summon, multi fireball and single fireball	The boss starts by casting shield spell upon engaging in combat with the first spell	The boss starts by casting shield spell upon engaging in combat with the first spell	Pass
2	Observe when the boss casts the next spell	An interval is set to prevent the boss from immediately casting the next spell upon finishing the previous	The boss waits for the duration of the interval before casting the next spell	The boss waits for the duration of the interval before casting the next spell	Pass
3	Wait for the boss to cast the rest of the spells	The boss should cast the spells according to their order in the hierarchy	The boss finish casting all spells in the correct order	The boss finish casting all spells in the correct order	Pass
4	Observe the next spell casted after casting all the spells in the hierarchy	After all spells are casted, the next spell to be casted should be the that has finished its cooldown	The single fireball spell is casted due to having the lowest cooldown	The single fireball spell is casted due to having the lowest cooldown	Pass
5	Wait until the boss casts the summon spell repeatedly	Some spells are intended to only be available in certain conditions. In this case, the summon spell should not be casted when there are more than 4 summoned enemies on the field.	The summon spell will not be casted once there are 4 summoned enemies	The summon spell will not be casted once there are 4 summoned enemies	Pass

Test Case 5: Test the dynamic level loading feature of the Game Manager

Step	Test Steps	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Setup the levels in an array exposed in the inspector	When the game starts, the game manager picks the first level in the array and loads it	The correct level is loaded as the first level	The correct level is loaded as the first level	Pass
2	Complete the first level	Each level element contains a level scene and a boss scene, and the boss scene of that level should be loaded upon completing the level	The correct boss scene is loaded	The correct boss scene is loaded	Pass
3	Defeat the boss and observe if the correct next level is loaded	When the boss is defeated, the next level should be loaded. In this case, the next level is just a duplicated copy of the first to test the system.	The second level is loaded	The second level is loaded	Pass
4	Complete the second level	When all levels and boss scenes are completed and the array has reached its end, the ending scene should be loaded	The ending scene is loaded	The ending scene is loaded	Pass
5	Swap the order of the levels and repeat	Modified the scene name and swapped the scenes around to see if the scenes are still loaded in correct order	The scenes are loaded in the correct order	The scenes are loaded in the correct order	Pass

5.3.2 User Acceptance Testing

User Acceptance Testing for Curse Breaker Awakening						
Tester name:	Khor Chin Yee					
Occupation:	Student (Multimedia)					
Testing date:	9/1/2024					
Start time:	11.00 a.m.					
End time:	12.00 p.m.					
Components	**Grade (1 – Poor, 5 - Outstanding)**					
	1	**2**	**3**	**4**	**5**	
1. Fulfilling objectives						
a. Overall fun factor of the game				✓		
b. Successful implementation of meaningful randomization to create replayability in the game					✓	
c. The cohesiveness / immersiveness created by the graphics / audio				✓		
d. Positive feeling of unexpectedness within the game, including both randomized / non-randomized elements					✓	
2. Bugs free					✓	
3. Text legibility				✓		

4. Graphics quality				✓	
5. Audio quality			✓		
6. Story quality			✓		
7. Gameplay quality					
8. Performance / Optimization				✓	
9. Overall user experience				✓	

Feedback from tester:

- Prefer roguelike levels with unlockable new abilities/power/spell/upgrade to offer to player
- The wallrunning feature is nicely implemented – not causing motion sickness – the field of vision is nicely configured
- Like the charged attack ability – able to dash over the enemies and cause stun, but this required accurate aiming, which is actually contribute to the replayability of the game, because it requires excessive training to pull off successfully, and the action is really powerful which add the user motivation to perform difficult attack
- Can add the enemies attack sound for easier prediction

Action(s) to be taken by developer:

- Development of requested systems such as abilities and upgrades will be added in the future to enrich gameplay
- Add sound before enemies attack to prevent catching players off guard

User Acceptance Testing for Curse Breaker Awakening						
Tester name:	Wong Zi Xuan					
Occupation:	Student					
Testing date:	09/01/2023					
Start time:	11:27AM					
End time:	11:50AM					
Components	Grade (1 – Poor, 5 - Outstanding)					
	1	2	3	4	5	
1. Fulfilling objectives						
a. Overall fun factor of the game				✓		
b. Successful implementation of meaningful randomization to create replayability in the game					✓	
c. The cohesiveness / immersiveness created by the graphics / audio					✓	
d. Positive feeling of unexpectedness within the game, including both randomized / non-randomized elements			✓			
2. Bugs free			✓			
3. Text legibility					✓	
4. Graphics quality					✓	
5. Audio quality					✓	
6. Story quality				✓		
7. Gameplay quality						
8. Performance / Optimization				✓		
9. Overall user experience					✓	

Feedback from tester:
There are some bugs occur during the gameplay. Sometimes it's quite clueless, not sure what to do and how to get the door open. Don't know how to skip the story at first. But overall, the game looks decent and professional, the randomization map makes me want to replay again in the future.
Action(s) to be taken by developer:
- Include a prompt or even a whole tutorial level that teaches the player about the lock and unlock condition of enemy rooms - Add a skip prompt for the story cutscene so that players know how to skip the cutscene

User Acceptance Testing for Curse Breaker Awakening						
Tester name:	Thien Chee Bing					
Occupation:	Student					
Testing date:	9 January 2024					
Start time:	12:40 p.m.					
End time:	12:55 p.m.					
Components	Grade (1 – Poor, 5 - Outstanding)					
	1	2	3	4	5	
1. Fulfilling objectives						
a. Overall fun factor of the game				✓		
b. Successful implementation of meaningful randomization to create replayability in the game					✓	
c. The cohesiveness / immersiveness created by the graphics / audio				✓		
d. Positive feeling of unexpectedness within the game, including both randomized / non-randomized elements					✓	
2. Bugs free					✓	
3. Text legibility					✓	
4. Graphics quality					✓	
5. Audio quality					✓	
6. Story quality					✓	
7. Gameplay quality				✓		
8. Performance / Optimization					✓	
9. Overall user experience				✓		

Feedback from tester:
The overall gameplay experience is enjoyable, and the random refresh of the map is also very good. However, the enemies lack attack sound effects, and there is no tutorial for first-time players, but the operation is still very smooth. And I like the charged dash attack and climb wall walk.
Action(s) to be taken by developer:
- Add more different attack sound effects for the enemies - Implement a tutorial

User Acceptance Testing for Curse Breaker Awakening	
Tester name:	Ng Kah Hou
Occupation:	Student
Testing date:	2-1-2024
Start time:	12:15pm
End time:	12:45pm

Components	Grade (1 – Poor, 5 - Outstanding)				
	1	2	3	4	5
1. Fulfilling objectives					
a. Overall fun factor of the game				/	
b. Successful implementation of meaningful randomization to create replayability in the game				/	
c. The cohesiveness / immersiveness created by the graphics / audio			/		
d. Positive feeling of unexpectedness within the game, including both randomized / non-randomized elements				/	
2. Bugs free			/		
3. Text legibility				/	
4. Graphics quality			/		
5. Audio quality			/		
6. Story quality			/		
7. Gameplay quality					/
8. Performance / Optimization					/
9. Overall user experience					/

Feedback from tester:
Overall is good but has some bugs towards the health and stamina bar. Animation of attack is good but hope got more combo so that others can enjoy
Action(s) to be taken by developer:
- Fix said bugs regarding the health and stamina bars - Enrich the combat system further

User Acceptance Testing for Curse Breaker Awakening	
Tester name:	Nicholas Lim Boon Seng
Occupation:	University Student
Testing date:	9th of January 2024
Start time:	2:00am
End time:	2:30am

Components	**Grade (1 – Poor, 5 - Outstanding)**				
	1	**2**	**3**	**4**	**5**
1. Fulfilling objectives					
a. Overall fun factor of the game					✓
b. Successful implementation of meaningful randomization to create replayability in the game					✓
c. The cohesiveness / immersiveness created by the graphics / audio				✓	
d. Positive feeling of unexpectedness within the game, including both randomized / non-randomized elements					✓
2. Bugs free				✓	
3. Text legibility					✓
4. Graphics quality					✓

5. Audio quality				✓	
6. Story quality				✓	
7. Gameplay quality					✓
8. Performance / Optimization					✓
9. Overall user experience					✓
Feedback from tester:					
Need more variant of procedural generated level layout, but it is already good enough. Fun to play with. Can improve the procedural generated level layout. Maybe add spells too, would be more fun to play.					
Action(s) to be taken by developer:					
- Create more rooms to add more variation to the generated levels - Enrich the combat system further					

5.4 Summary

After performing all of the tests mentioned above, it is evident that these tests have been very crucial to the development project. The developer was able to more quickly and easily address any issues that might have been missed during development thanks to the excessive amount of both internal testing and external testing performed by the testers. If these tests were not performed before concluding the project, the released project might be in a broken state, rendering the game unplayable and leaving a bad impression on the players. There are popular incidents of such a scenario happening even for popular games, such as Cyberpunk 2077 (2020) and The Day Before (2023), which released in a terribly optimized state with lots of bugs and glitches which heavily impacted the game's sales (IGN, 2023).

The unit testing results shows that it is inevitable that during development there will be some bugs that are hard to discovered without breaking down each feature are inspecting each pieces that forms the whole system carefully. By performing unit tests, the developer will have a higher chance of capturing these issues and become able to fix them in time. An example would be the audio issue mentioned above, which is rather hard to notice without performing excessive amount of testing.

On the other hand, the user acceptance tests has been extremely helpful in terms of exposing issues that are more easily found from a player's perspective. As mentioned before, it is easy for developers to miss these issues because of their familiarity and knowledge of the project's structure. The developer has been able to fix this issue thanks to the report provided by external testers. The user acceptance tests also shows that different players might have different opinions towards the same features, for example some find the combat satisfying while some think that the combat system is still lacking features. It is up to the developer to find the balance between differing opinions and take actions accordingly.

All in all, this section showcases how important it is to perform project testing before building the actual product. Had these tests not been performed, the developer believes that the project will not get anywhere near its current state. The developer has managed to fix a great number of issues thanks to these tests, and would therefore like to express their profound gratitude towards the external testers.

Chapter 6: Conclusion

6.1 Critical Evaluation

In the end, the developer would say that the project is quite a great success. The main goals and scope of the project has been successfully achieved, even though there has been some slight deviation from the original plan which are inevitable as some issues are clear only when the project has reached a certain phase. An example of this would be the level generation algorithm which has undergone through quite a number of iterations in order to find the best fit for the project. It was originally planned to be an algorithm similar to that of Enter The Gungeon (Devolver Digital, 2016), but later adapted to create an algorithm that produces a more fitting end result for Curse Breaker: Awakening. The final algorithm uses a modified version of the Depth-First Search algorithm, which offers great flexibility and customizability, though it also has its own limitations such as the maximum of 4 exits for each rooms.

Looking back at the initial scope, all of the targets has been successfully completed in one way or another.

1. All of the **core gameplay systems** such as player controller and player-environment interaction are in place.
2. The **combat system** is successfully completed, where most testers agree that it satisfy the "fast and movement-based" premise but some think that it can still be improved upon.
3. Development of the **procedural level generation system** has also been successful, though it has become quite different from what is initially planned due to changes in requirements and new knowledge acquired during development.
4. A sufficient amount of individual **hand-crafted rooms** (levels) to be generated within the procedural level generation algorithm are created to create variety and ensure that no two runs are the same, but the player might start noticing the same rooms after enough playthroughs.
5. **Loot randomization** is also successfully implemented, although the variety in possible loot itself can be further improved as there are currently only coins and health potions

6. A rather intuitive **navigation system** that helps the player navigate through the randomly generated levels has been successfully implemented, where most testers agree to be quite helpful.
7. A **state machine system** has been developed for the enemy that emphasizes on **high flexibility**. This is achieved by thoroughly **decoupling of the state actions and conditions**, making them easily replaceable, and also works with multiple conditions.
8. All important **UI elements** are in place in the final outcome, including a working main menu, pause menu, death screen and ending screen with some degree of design and polish put into them.
9. A **complete game loop** has been implemented with a proper start and win condition with customizable gameplay length (via changing the number of rooms per level as well as the number of levels) along with an extra cutscene in the beginning to explain the premises of the story, giving the player a purpose to finish the game.
10. A full-fledged **audio system** has been developed, supporting playing background music, global sound effects (eg. menu sounds) and even useful features such as music crossfading, saving previously played music to continue later, volume and pitch variation. It also utilizes **dictionaries** to store the audio files which makes the system more performant.
11. Last but not least, the game has been partially polished with various visual effects and sound effects and post-processing effects to make it more attractive as a demo as well as featuring tweening for UI elements to create performant UI animations. If the game is completely unpolished, it will be nowhere as enjoyable as it currently is.

Besides from the initially defined scope, there are also a number of other extra accomplishments made along the way:

1. **Modules System**

   This system is not particularly developed during this project, but is one of the backbones that make this project possible. It is a system built throughout the past year during the development of the developer's various different projects, initially inspired by Bardent's platformer tutorial series (Bardent, 2024). It follows the principle of **composition over inheritance**, featuring a set of plug-and-play modules that can be attached to game objects requiring the corresponding feature. For example, the audio emitter module is a wrapper for Unity's audio source component that automatically sets up everything needed to play highly customizable sound effects. Combined with the use of the Module Manager, game entities can easily access these modules to achieve various effects. The use of composition ensures that these game entities does not have a direct reference to these modules, for example some entities does not utilize Unity's navigation system, but there will be no errors even if the developer accidentally added an AI navigation module onto the entity.

2. **Spell System for Enemies**

   This is a system created while developing the boss enemy and is an expansion upon the flexible state machine system. While the state machine is highly customizable, it requires the developer to strongly type the concrete conditions and states in the scripts and recompile them in order to apply the changes made. This can be confusing if the enemies have a lot of abilities that they are able to perform and therefore have many states. However, the spell system offers another approach, where the enemy in question will have a central state known as the engaged state. The state machine then accesses the **Spell Manager**, which contains various spells that the enemy can perform. The Spell Manager stores each ability into a dictionary and dynamically create states with their respective conditions based on several parameters such as the interval between spells. This offers further customizability, as the developer can change the cooldown and effect of each spells just in the inspector and even modify the order in which the spells are performed by simply reordering the hierarchy.

However, that is not to say that they project is perfect without any flaws. There are definitely a lot that can be improved upon, which will be discussed in following sections. However, in terms of the main goal of ensuring that the project qualifies as a full-fledged game demo, the developer believes that this goal has been successfully achieved.

6.2 Limitations

While the main goals of this project has been achieved, some of them took some compromises and change in requirements in order to do so.

An example of this would be the implementation of the level generation algorithm. As mentioned before, the algorithm ended up going through several iterations and massive changes due to the developer's limited knowledge and abilities to fit the requirements of the project and being way different from what was originally envisioned. Although it still serves its purpose, a lot of other aspects are affected by this change, for example the rooms cannot be physically connected like originally envisioned and can only have a maximum of 4 exits each.

Another limitation is that although the target of having each generated level being completely different has been achieved, this does not necessarily mean that each playthrough offer unique challenges. In most cases, the tasks of the players remain the same throughout subsequent playthroughs: defeating enemies to progress towards the boss room. This can be fixed in the future by either creating more rooms to be spawned and also by implementing more different room types such as NPC rooms where players can conduct conversations with other characters, shops to buy upgrades and other types of rewards.

The combat system of the game is also somewhat lacking, whereas the player is offered with movement abilities to navigate through the levels but the enemies themselves does not necessary require these abilities to be defeated. Other upgrades and abilities can be also added to the player to further enrich the combat, allowing more options in combat such as using ranged attack as their main offensive option for a more safety-focused strategy.

All in all, the developer hopes to continue improving the game and addressing these problems in the future even after the submission is completed, in hope that it qualifies as a game product with commercial potential that can be enjoyed by a wider audience of players.

6.3 Recommendation

In the future, the developer will continue improving the level generation algorithm to hopefully overcome the respective limitations imposed. This is to create more customizability to the generation result, such as rooms with more than 4 exits and physically connected rooms.

Besides that, the developer will also work on new room types such as NPC rooms. In the developer's opinion, NPCs are crucial in telling the game's story, as the story should not be told in a cutscene alone. The existence of NPCs that fits within the story can massively improve player immersion, making the player feel as if the game world actually exist and works as a setting. More gameplay elements will also be added, such as player upgrades and items with the corresponding methods to acquire these upgrades. For example, a shop room can be implemented for the player to buy items, or an alternative reward room other than the treasure room can be implemented, such as a blessing room that the player can acquire free upgrades within.

The developer also hopes to further enrich the combat system, adding even more movement options to the player's skillset as well as making the combat system resonate well with the movement system. For example, there could be flying enemies that can only be reached by performing a charged attack. More combat options such as spells can also be implemented, encouraging more strategic gameplay where the player has to improvise based on whatever random spells they managed to acquire.

The game can also be further polished, adding more visual effects, UI and audio wherever needed. It is also possible to swap assets sourced from the internet with custom-made assets created by the developer or professional individuals in order to fit the game's style better.

While the recommendations above are the most obvious improvements that can be made, the developer is certain that there are a lot more new creative ideas that can be discovered by simply

continue working on the project, and hopefully these changes will push the project closer towards qualifying as a commercial product.

References

Chen, Y.H., & Wu, C.M (2020). An improved algorithm for searching maze based on Depth-First Search. IEEE Conference Publication | IEEE Xplore. https://ieeexplore.ieee.org/abstract/document/9258170

Zhenyuan Shen. (2022). Procedural generation in games: Focusing on dungeons. https://www.shs-conferences.org/articles/shsconf/pdf/2022/14/shsconf_stehf2022_02005.pdf

The New York Times. (2023). Don't be scared. play a roguelike. The New York Times. https://www.nytimes.com/wirecutter/reviews/what-are-roguelike-games/

Bill Loguidice. (2009). The history of rogue: Have @ you, you deadly zs. Game Developer. https://www.gamedeveloper.com/design/the-history-of-rogue-have-you-you-deadly-zs

Unity Game Development Engine: A technical survey - researchgate. (2022). https://www.researchgate.net/publication/348917348_Unity_Game_Development_Engine_A_Technical_Survey

Omtera. (2022). What is agile methodology? Sprint Backlog and link to Scrum. Omtera. https://omtera.com/en/what-is-agile-methodology-sprint-backlog-and-link-to-scrum/

LucidChart. (2018).What the Waterfall Project Management Methodology can (and can't) do for you. https://www.lucidchart.com/blog/waterfall-project-management-methodology

Boris. (2022). Dungeon generation in enter the gungeon. BorisTheBrave.Com. https://www.boristhebrave.com/2019/07/28/dungeon-generation-in-enter-the-gungeon/

Griffin, S. M. (2022). Cult of the lamb review – grow your own cult in darkly cute game. The Guardian. https://www.theguardian.com/games/2022/sep/02/cult-of-the-lamb-review-grow-your-own-cult

Unity Technologies. (2019). Input System 1.6. https://docs.unity3d.com/Packages/com.unity.inputsystem@1.6/manual/index.html

Unity Technologies. (2022). Navmesh obstacle. https://docs.unity3d.com/Manual/class-NavMeshObstacle.html

Unity Technologies. (2022). Inner workings of the navigation system. Unity. https://docs.unity3d.com/Manual/nav-InnerWorkings.html#:~:text=When%20the%20obstacle%20becomes%20stationary,carves%20holes%20into%20the%20NavMesh.

Swan, C. (2022). Elden Ring makes the most of reused assets. Game Rant. https://gamerant.com/elden-ring-reused-assets-dungeons-bosses-perfectly-used/

Peters, J. (2023, March 11). It's a great time to start playing Dead Cells. The Verge. https://www.theverge.com/23634280/dead-cells-return-to-castlevania-expansion-dlc-start-playing

BasuMallick, C. (2022, September 20). What is Unit Testing? Top Tools and Best Practices. Spiceworks. https://www.spiceworks.com/tech/devops/articles/what-is-unit-testing/

Appendices

Appendix A: PPF – Title Registration Proposal

Office Record	Receipt
Date Received: Received by whom:	Student name: Student number: Received by: Date:

DRAFT PROJECT PROPOSAL FORM

Proposal ID:

Supervisor:

Student Name : **GOH SHAO HANG**

Student No : **TP063859**

Email Address : TP063859@mail.apu.edu.my

Programme Name : Computer Games Development(CGD)

Title of project : **Making a 3D game with high replayability using procedural level generation**

---

1. Introduction

Assume the reader has very little knowledge of the subject. Introduce the topic, the sector of business/industry concerned and how the project relates to it. Define the context of the problem and identify the research required to solve it.

Video games have become increasingly popular as a source of entertainment in the modern era due to their accessibility and interactive nature. However, with the younger generation's shorter attention spans, it is crucial for game developers to consistently provide new gameplay content and challenges. This poses a challenge considering the expensive and labor-intensive nature of game development (Luiz Rodrigues and Jacques Brancher, 2019). Continuously creating new game content not only strains developers but also increases the risk of introducing bugs and other issues. Balancing content development and quality control becomes a daunting and unmanageable task in the long run. Fortunately, there are several approaches to address this issue, such as employing play-testers or utilizing computer algorithms to streamline the process. Among these solutions, procedural content generation has emerged as a popular choice. This technique involves automating or assisting programmers and designers in creating content, allowing them to generate diverse and substantial amounts of content using a limited number of assets and within a constrained timeframe (Rafael et al., 2022). This paper aims to explore how

developers can optimize the benefits of procedural generation while mitigating potential drawbacks.

2. Problem Statement

Identify past and current work in the subject area. Outline the key references to other people's work, indicate for the most pertinent of these how your proposal relates to the ideas they contain.

1. Time and budget limitations

Due to the increasing popularity and demand of video games, game developers have to constantly produce new and engaging content in order to keep their players interested and willing to try out their products. To do so, they will have to create a large amount of game assets in a limited amount of time in a consistent pace, not to mention the cost of creating these assets. If they fail to produce enough content in this limited time, they might suffer a loss in players and their games will rapidly die out in the intense competition in the market. These companies must grapple with the delicate balance of time, cost, and creativity to ensure their games remain relevant and engaging in the face of intense market competition. This problem is especially apparent for small, independent game development teams or companies, which often have very limited budget.

2. Developer stress and quality of production

Assuming that the company in question has sufficient budget to constantly produce content, the developers might suffer from stress due to having to keep up a consistent pace at developing content. The quality of the product will also very likely decrease due to excessive, nonstop work, as well as creativity exhaustion. Even if developers manage to produce the assets, low product quality that fails to captivate players will cause the game to decline in popularity, causing major loss to the company. Repetitive content will also cause aesthetic fatigue in the players. It is evident that game development is an expensive process that requires multidisciplinary and constant effort (Luiz Rodrigues and Jacques Brancher, 2019). Rather than stressing themselves out, the developers should come up with clever ways to produce content with minimal effort, such as procedural content generation.

3. Project Aim and Objectives

Identify the AIM(s) of the project, i.e. what the overall achievement is intended to be, in terms of both academic and commercial/industrial advances. Identify the particular intellectual difficulties posed by the proposal, the problems to be addressed, and explain how these might be solved. Clearly list individual measurable OBJECTIVES which can be related to the workplan and deliverables.

Aims and objectives are subject to approval from supervisor and students are expected to revise them if deemed inappropriate for a Level 3 project.

The aim of this project is to develop a fully playable demo for a 3D game that emphasizes game replayability, supporting the fact that a game can be enjoyed for an extended period of time without creating a substantial amount of high quality assets by utilizing procedural level generation techniques and algorithms.

The objectives of the project are as follow:

- To investigate various game development and procedural generation techniques that can be used to create an immersive, enjoyable and customizable gameplay experience with minimal effort.
- To develop gameplay and content generation systems that produces varying yet controllable and satisfying results with high efficiency.
- To evaluate the importance of asset variety and gameplay aspects in terms of their contribution towards gameplay experience.

4. Literature Review

According to Newzoo (2020), around 3 billion people play video games in the world, generating a market with the astounding revenue of 160 billion USD. This serves as a concrete proof that the popularity of video games is rising at a rapid pace. However, developers are struggling to develop new and engaging content in order to satisfy the ever rising demands of the players. Procedural content generation serves as one of the popular methods to create game content, which allows for randomized rather than hand crafting levels, reducing the costs and increasing the replayability value of a game (Pereira. L. T. et al., 2021). This allows developers to develop on a algorithm that assists them in producing randomized content quickly, and can be further modified and extended if needed.

Besides, procedural game content generation can also significantly contribute to increasing the enjoyment experienced by players. Procedural generation proves effective in introducing high variation and randomness to the gameplay while minimizing the utilization of extensive game assets. By stimulating curiosity and offering diverse scenarios, procedurally generated content encourages players to replay the game and adapt various strategies in response to counteract varying situations. A study conducted by Luiz Rodrigues and Jacques Brancher (2019) revealed notable distinctions in gameplay behavior between human-designed and procedurally generated content. The introduction of procedurally generated content piques players' curiosity, fostering greater interest and immersion in the gaming experience. However, it is important to acknowledge that their conclusions may require further support through additional comprehensive examples and research.

5. Deliverables

Provide a clear list of the outputs from the project.

The target users are:

1. Young video game players looking for interesting game content
2. Roguelike genre lovers
3. Game developers looking for ways to create game content more efficiently

The project will have the following features that benefits the player:

1. A fully playable game demo of a 3D video game consisting 1 (or more) levels with some amount of polish (visual effects, full-fledged audio system as well as post processing effects)
2. A level generation method that allows for a mix of both randomized and hand-crafted experience to maximize player enjoyment
3. Immersive and engaging gameplay systems to support the generated levels

The project will have the following features that benefits game developers:

1. A flexible, modular and readable codebase for the level generation algorithm that allows for easy modification to suit their needs
2. Various applied techniques to customize the right amount of randomness of the generation algorithm
3. Game design and game polishing techniques that makes up an enjoyable gameplay experience

6. References

Luiz Rodriguse and Jacques Brancer (2019). *The Impact of Procedural Level Generation on Players' Experiences and In-game Behavior*. https://www.researchgate.net/publication/337032924_The_Impact_of_Procedural_Level_Generation_on_Players'_Experiences_and_In-game_Behavior

Rafael Guerra de Pontes, Herman Martins Gomes, Igor Santa Ritta Seabra (2022). *Particle swarm optimization for procedural content generation in an endless platform game.* https://www-sciencedirect-com.ezproxy.apu.edu.my/science/article/pii/S1875952122000209

Pereira. L. T., Prado, P. V. S., Lopes. R. M., Toledo C.F.M. (2021). *Procedural generation of dungeons' maps and locked-door missions through an evolutionary algorithm validated with players.* https://www.sciencedirect.com/science/article/pii/S0957417421004504

Newzoo (2020). *Newzoo global games market report 2020: Light version.* https://newzoo.com/insights/trend-reports/newzoo-global-games-market-report-2020-light-version/

Appendix B: Ethic Forms (Fast Track)

Office Record	Receipt – Fast-Track Ethical Approval
Date Received:	Student name: Goh Shao Hang Student number: TP063859
Received by whom:	Received by: Dr. Lai Ngan Kuen Date: 27/7/2023

APU / APIIT FAST-TRACK ETHICAL APPROVAL FORM (STUDENTS)

Tick one box (level of study):
- ☐ POSTGRADUATE (PhD / MPhil / Masters)
- ☑ UNDERGRADUATE (Bachelors degree)
- ☐ FOUNDATION / DIPLOMA / Other categories

Tick one box (purpose of approval):
- ☑ Thesis / Dissertation / FYP project
- ☐ Module assignment
- ☐ Other: _____

Title of Programme on which enrolled __Computer Games Development_____

Tick one box: ☑ Full-Time Study or ☐ Part-Time Study

Title of project / assignment ___Creating a Highly Replayable Roguelike Game_____

Name of student researcher _____Goh Shao Hang_____

Name of supervisor / lecturer ____Dr Lai Ngan Kuen / Mr. Jacob Sow Tian You_____

Student Researchers- please note that certain professional organisations have ethical guidelines that you may need to consult when completing this form.

Supervisors/Module Lecturers - please seek guidance from the Chair of the APU Research Ethics Committee if you are uncertain about any ethical issue arising from this application.

		YES	NO	N/A
1	Will you describe the main procedures to participants in advance, so that they are informed about what to expect?	✓		
2	Will you tell participants that their participation is voluntary?	✓		
3	Will you obtain written consent for participation?	✓		
4	If the research is observational, will you ask participants for their consent to being observed?	✓		
5	Will you tell participants that they may withdraw from the research at any time and for any reason?	✓		
6	With questionnaires and interviews will you give participants the option of omitting questions they do not want to answer?	✓		
7	Will you tell participants that their data will be treated with full confidentiality and that, if published, it will not be identifiable as theirs?	✓		
8	Will you give participants the opportunity to be debriefed i.e. to find out more about the study and its results?	✓		

If you have ticked **No** to any of Q1-8 you should complete the full Ethics Approval Form.

		YES	NO	N/A
9	Will your project/assignment deliberately mislead participants in any way?		✓	
10	Is there any realistic risk of any participants experiencing either physical or psychological distress or discomfort?		✓	
11	Is the nature of the research such that contentious or sensitive issues might be involved?		✓	

If you have ticked **Yes** to 9, 10 or 11 you should complete the full Ethics Approval Form. In relation to question 10 this should include details of what you will tell participants to do if they should experience any problems (e.g. who they can contact for help). You may also need to consider risk assessment issues.

			YES	NO	N/A
12	Does your project/assignment involve work with animals?			✓	
13	Do participants fall into any of the following special groups? **Note that you may also need to obtain satisfactory clearance from the relevant authorities**	Children (under 18 years of age)			
		People with communication or learning difficulties			
		Patients			
		People in custody		✓	
		People who could be regarded as vulnerable			
		People engaged in illegal activities (eg drug taking)			
14	Does the project/assignment involve external funding or external collaboration where the funding body or external collaborative partner requires the University to provide evidence that the project/assignment had been subject to ethical scrutiny?			✓	

If you have ticked **Yes** to 12, 13 or 14 you should complete the full Ethics Approval Form. There is an obligation on student and supervisor to bring to the attention of the APU Research Ethics Committee any issues with ethical implications not clearly covered by the above checklist.

STUDENT RESEARCHER
Provide in the boxes below (plus any other appended details) information required in support of your application, THEN SIGN THE FORM.

Please Tick Boxes

I consider that this project/assignment has no significant ethical implications requiring a full ethics submission to the APU Research Ethics Committee.	✓
Give a brief description of participants and procedure (methods, tests used etc) in up to 150 words.	
I also confirm that: i) All key documents e.g. consent form, information sheet, questionnaire/interview are appended to this application. Or	✓
ii) Any key documents e.g. consent form, information sheet, questionnaire/interview schedules which need to be finalised following initial investigations will be submitted for approval by the project/assignment supervisor/module lecturer before they are used in primary data collection.	

E-signature… … … … … … … … … … … … … Print Name… … … GOH SHAO HANG… … … Date…24/7/2023
(Student Researcher)

Please note that any variation to that contained within this document that in any way affects ethical issues of the stated research requires the appending of new ethical details. New ethical consent may need to be sought.

The completed form (and any attachments) should be submitted for consideration by your Supervisor/Module Lecturer

SUPERVISOR/MODULE LECTURER
PLEASE CONFIRM THE FOLLOWING:

	Please Tick Box
I consider that this project/assignment has no significant ethical implications requiring a full ethics submission to the APU Research Ethics Committee	✓
i) I have checked and approved the key documents required for this proposal (e.g. consent form, information sheet, questionnaire, interview schedule) Or	✓
ii) I have checked and approved draft documents required for this proposal which provide a basis for the preliminary investigations which will inform the main research study. I have informed the student researcher that finalised and additional documents (e.g. consent form, information sheet, questionnaire, interview schedule) must be submitted for approval by me before they are used for primary data collection.	

SUPERVISOR AND SECOND ACADEMIC SIGNATORY

STATEMENT OF ETHICAL APPROVAL (please delete as appropriate)

1) **THIS PROJECT/ASSIGNMENT HAS BEEN CONSIDERED USING AGREED APU/SU PROCEDURES AND IS NOW APPROVED**

2) **THIS PROJECT/ASSIGNMENT HAS BEEN APPROVED IN PRINCIPLE AS INVOLVING NO SIGNIFICANT ETHICAL IMPLICATIONS, BUT FINAL APPROVAL FOR DATA COLLECTION IS SUBJECT TO THE SUBMISSION OF KEY DOCUMENTS FOR APPROVAL BY SUPERVISOR (see Appendix A)**



E-signature… … … … … … … … … … … … Print Name…Dr Lai Ngan Kuen… … … … Date… … … …
(Supervisor/Lecturer)

OR

E-signature… … … … … … … … … … … … Print Name… … … … … … … … … … … … Date… … … …
(Second Academic Signatory)

Office Record	Receipt – Appendix A (Fast-Track Ethics Form)
Date Received: Received by whom:	Student name: Goh Shao Hang Student number: TP063859 Received by: Dr. Lai Ngan Kuen Date: 27/7/2023

APPENDIX A
AUTHORISATION FOR USE OF KEY DOCUMENTS

Completion of Appendix A is required when for good reasons key documents are not available when a fast track application is approved by the supervisor/module lecturer and second academic signatory.

I have now checked and approved all the key documents associated with this proposal e.g. consent form, information sheet, questionnaire, interview schedule

Title of project/assignment… … …Creating A Highly Replayable Roguelike Game… … … … … … … … … … …

… … … … … … … … … …

APPENDIX B
RESEARCH QUESTIONNAIRE

Introduction

Dear Respondent,

Good day to you, my name is Goh Shao Hang, and I am currently pursuing a degree in Computer Game Development at Asia Pacific University (APU). I am excited to present this questionnaire as an essential part of my final year project titled "Creating a Highly Replayable Roguelike Game."

The purpose of this questionnaire is to gather valuable insights and opinions from video game like yourself on what aspects contribute to the replayability and enjoyment of video games. Whether you have experience with roguelike games or not, your input will be invaluable in shaping the success of my research.

Your participation will involve sharing some information, such as your daily gaming activities, gaming habits, preferences, and more. However, please note that having prior experience with roguelike games is beneficial but not compulsory for participating in this study.

All the information you provide will be strictly confidentiality and used solely for research purposes. Your privacy is of the utmost importance, and you can rest assured that your responses will be handled responsibly and professionally.

Should you have any questions or concerns, please do not hesitate to reach out to me via TP063859@mail.apu.edu.my.

Thank you so much for taking the time to participate in this questionnaire. Your valuable insights will be crucial in shaping the direction of my final year project.

Section A: Demographics

1. Age
   - 18 – 22
   - 23 – 27
   - 28 – 32
   - 33 – 37
   - 38 - 42
   - 42 – 47
   - 48 or above
2. Gender
   - Male
   - Female
   - Other / Prefer not to disclose
3. Occupation
   - Student
   - Employed
   - Self Employed
   - Freelancer
   - Unemployed
   - Retired
   - Other
4. Educational Background
   - Primary
   - Secondary
   - High School
   - Bachelor’s Degree
   - Master’s Degree

- Other

Section B: Gaming Habits and Preferences

This section aims to understand your gaming habits and preferences, which plays a crucial role in helping me understand public preferences and gaming activities. Your input will provide valuable insights into the elements that captivate players and contribute to the appeal of different game genres.

1. On average, how many hours do you play video games every day?
   - Below 1 hour
   - 1 – 2 hours
   - 2 – 3 hours
   - 3 – 4 hours
   - 4 – 5 hours
   - 5 hours and above
2. What is your favorite video game genre?
   - Action
   - Role-playing
   - First Person / Third Person Shooter
   - Adventure
   - Puzzle
   - Strategy
   - Simulation
   - Sports
   - Horror
   - Others
3. Do you find yourself playing more casual games (low difficulty, short game sessions) or hardcore games (high difficulty, takes more time to complete)
   - Casual Games
   - Hardcore Games
4. Do you prefer games that promotes strategic thinking (turn-based, puzzle etc.), or more action-oriented mechanics that test your reflexes (action, horror)?
   - Strategic Thinking
   - Action-Oriented
5. Do you find yourself more inclined towards indie games (games developed by independent companies, often with smaller-scale productions) or 3A games (games with top-tier graphics and high production values created by larger, well-established companies)?
   - Indie Games
   - 3A Games
6. On average, how many games do you finish regardless of completion rate every month(eg. only getting one out of many endings)?
   - None
   - 1 – 2
   - 2 – 3
   - 3 – 5
   - More than 5
7. How many games have you completed with 100% completion (eg. Getting all endings in a game, unlocking all unlockables)?
   - None
   - 1 – 2
   - 2 – 3
   - 3 – 5
   - More than 5

8. How important are the following factors in determining your enjoyment of a video game?
   - Graphics and Visuals

- Story and Narrative
- Gameplay Mechanics
- Character Customization
- Replayability
- Difficulty Level
- Other

Section C: About Game Replayability

Game replayability refers to the extent and appeal of a video game's ability to be played multiple times without losing its entertainment value. It is a measure of how engaging and enjoyable a game remains even after the player has completed it or experienced its content several times. A highly replayable game offers fresh and varied experiences with each playthrough, encouraging players to return to the game for additional sessions. This is achieved through various design elements and features.

1. How often do you replay a video game you have completed before?
   1 (Never) – 5 (Very Often)
2. What do you think motivates you to replay a game multiple times?
   - Exploration and Discovery
   - Procedurally Generated Levels and Loots
   - Unlockables and Achievements
   - Mastery over the Game
   - New Game Plus (Carrying over progress into a new playthrough)
   - Optional Side Quests
   - Branching Storylines (Multiple endings etc.)
   - Competitive Aspects (Multiplayer, Ranking, Leaderboards etc.)
   - Emotional Engagement
   - Other
3. On a scale from 1 to 5, how often do you play roguelike games?
   *Note: Roguelike is a type of video game known for its challenging and procedural (randomly generated) gameplay, often with the concept of permanent character death.
   1 (Never) – 5 (Very Often)
4. **If you have played a roguelike game before:**
   On a scale from 1 to 5, how much do you prefer roguelike games?
   1 (Strongly Dislike) – 5 (Strongly Like)
5. Which of the following elements do you think contributes the most in game replayability?
   - Procedural Content Generation (Randomly generated content such as levels and loots)
   - Permanent Character Death (Progress reset on death)
   - Metaprogression (Getting new upgrades after every playthrough)
   - Discovering New Elements (NPCs, Bosses etc.)
   - Unlockables and Collectibles
   - Resource Management
   - Different Playstyles in each Playthrough / Build Diversity
   - Rare or Special Occurences (Secret Items, Quests etc.)
   - High Difficulty
   - Other
6. How important do you think randomized level layouts and item placements are in enhancing replayability?
   1 (Not important) – 5 (Very important)
7. How much does the presence of unlockables or achievements affect your motivation to replay a game?
   1 (Not at all) – 5 (Extremely)
8. How do you feel about permanent character death (permadeath) in a game?
   1 (Strong Dislike) – 5 (Strongly Like)
9. How important do you think is a diverse selection of character / classes in increasing game replayability?
   1 (Not important) – 5 (Very important)
10. To what extent does the presence of a narrative or lore impact your willingness to replay a game?
    1 (Not at all) – 5 (Extremely)
11. How much do online leaderboards and rankings influence your decision to replay a game?
    1 (Not at all) – 5 (Extremely)

12. Based on your experiences, suggest a game that you have played before and what improvements can be made to enhance its replayability. (Input Text)
13. Are there any specific features you believe would make a game more replayable? (Input Text)
14. Any additional comments or insights you'd like to share about roguelike games and their replayability? (Input Text)

Appendix C: Log Sheets

(APU: Serial Number)

PLS V1.0

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX (6) during the course of the project (SIX mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session.
7. The log sheet is an important deliverable for the project and an important record of a student's organisation and learning experience. The student **must** hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's name: Goh Shao Hang **Date: 27/6/2023** **Meeting No: 1**

Project title: Developing a 3D game with high replayability using procedural level generation
Intake: APD3F2305CGD

Supervisor's name: Dr. Lai Ngan Kuen **Supervisor's signature: ……...…**

Items for discussion (noted by student before mandatory supervisory meeting):

1. **Overall scope of the project**
2. **Viability of various subjects to implement**
3. **Priority of required components of the project**
4. **Methods to implement said components, such as theory of the level generation algorithm**

Record of discussion (noted by student during mandatory supervisory meeting):

1. **The overall scope of the project is decided.**
2. **Discussed about a draft design document, and discussed on the viability of each subject.**
3. **Sorted the priority of which components should be implemented first.**
4. **Came up with a few ideas for the level generation algorithm**

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

1. **Draft the theory for a procedural generation algorithm**
2. **Write a basic game design documentation**
3. **Read resource about utilizing Generative AI to aid in developing the project provided by the supervisor**

Note: A student should make an appointment to meet his or her supervisor (via the consultation system) at least ONE (1) week prior to a mandatory supervisor session – please see document on project timelines. In the event a supervisor could not be booked for consultation, the project manager should be informed ONE (1) week prior to the session so that a meeting can be subsequently arranged.

(APU: Serial Number)

PLS V1.0

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX (6) during the course of the project (SIX mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session.
7. The log sheet is an important deliverable for the project and an important record of a student's organisation and learning experience. The student **must** hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's name: Goh Shao Hang **Date: 21/7/2023** **Meeting No: 2**

Project title: Creating A Highly Replayable Roguelike Game
Intake: APD3F2305CGD

Supervisor's name: Dr. Lai Ngan Kuen **Supervisor's signature:** ……..…

Items for discussion (noted by student before mandatory supervisory meeting):

1. Discuss on changing the title of the project
2. Redefine the scope of the project to fit the new title
3. Discuss the ethics problem to avoid when developing the game project, eg. religion, violence and more
4. Discussion on the questions to include in the questionnaire for the project

Record of discussion (noted by student during mandatory supervisory meeting):

1. After some consideration, the title of the project is changed from **"Developing a 3D game with high replayability using procedural level generation"** to **"Creating a Highly Replayable Roguelike Game"**. This is because the student decided to focus on delivering an enjoyable game rather than only the generation algorithm.
2. The scope of the project is rediscussed to fit the new title, shifting the focus away from the generation algorithm
3. Some part of the game story is revised to avoid any potential ethics problems, eg. fake religions that exist in the game story.
4. Complete the questionnaire and searching for respondents to aid in research

(APU: Serial Number)

PLS V1.0

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX (6) during the course of the project (SIX mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session.
7. The log sheet is an important deliverable for the project and an important record of a student's organisation and learning experience. The student **must** hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's name: Goh Shao Hang **Date: 2/8/2023** **Meeting No: 3**

Project title: Creating A Highly Replayable Roguelike Game
Intake: APD3F2305CGD

Supervisor's name: Dr. Lai Ngan Kuen **Supervisor's signature: ……...…**

Items for discussion (noted by student before mandatory supervisory meeting):

1. Discussion of the components of the game in more detail
2. Update the supervisor on the current progress of the project
3. Showcase developed prototype and new ideas of the level generation algorithm
4. Finalization of the Investigation Report

Record of discussion (noted by student during mandatory supervisory meeting):

1. Discussed on adding some special features into the game project to make it more appealing and stand out form other games in the market, eg. allowing the player to grab a weakened enemy to block enemy projectiles
2. Submitted justification for the questionnaire shared to respondents to receive public feedback on their thoughts on game replayability and roguelike games
3. Displayed prototype of a previous attempt of level generation made in 2D and shared thoughts on how to improve upon it
4. Received guidance on writing the Investigation Report, and discussed future plans of the project

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

1. Begin dedicated development on the main project
2. Complete and submit the Investigation Report before the deadline
3. Review documents shared by the supervisor to aid in developing the project and writing the Investigation Report

(APU: Serial Number)

PLS V1.0

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX (6) during the course of the project (SIX mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session.
7. The log sheet is an important deliverable for the project and an important record of a student's organisation and learning experience. The student must hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's name: Goh Shao Hang **Date:** 1/11/2023 **Meeting No:** 4

Project title: Creating A Highly Replayable Roguelike Game
Intake: APD3F2305CGD

Supervisor's name: Dr. Lai Ngan Kuen **Supervisor's signature:** .....

Items for discussion (noted by student before mandatory supervisory meeting):

1. Inquiring the parts that has to be improved in the investigation report
2. Update the supervisor on the current progress of the project
3. Demonstrate the current state of the project
4. Inquiring about the submission of the final report

Record of discussion (noted by student during mandatory supervisory meeting):

1. Showed the supervisor the investigation report and asked for feedback on how to improve when writing the final report
2. Showcased the current state of the project to the supervisor such as the player controller and received feedback
3. Displayed prototype of improved groundwork of level generation algorithm that will serve as the base of the final algorithm
4. Asked questions regarding the submission of the final report

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

1. Continue working on the level generation algorithm and generate concrete levels
2. Introduce basic gameplay to the project to make it minimally playable
3. Develop AI system for enemies to enrich the gameplay and provide motives for the players

(APU: Serial Number)

PLS V1.0

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX (6) during the course of the project (SIX mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session.
7. The log sheet is an important deliverable for the project and an important record of a student's organisation and learning experience. The student must hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's name: Goh Shao Hang **Date: 12/12/2023** **Meeting No: 5**

Project title: Creating A Highly Replayable Roguelike Game
Intake: APD3F2305CGD

Supervisor's name: Dr. Lai Ngan Kuen **Supervisor's signature:** ..

Items for discussion (noted by student before mandatory supervisory meeting):

1. Validation of the work-in-progress final report
2. Update the supervisor on the current progress of the project
3. Showcase new features developed for the project
4. Discuss about the final presentation

Record of discussion (noted by student during mandatory supervisory meeting):

1. Showed the supervisor the work-in-progress final report and received feedback
2. Updated the current state of the project to the supervisor, including improvements made to existing systems
3. Displayed the finished level generation algorithm and its various implementation, such as spawning levels and rendering map
4. Inquired about more details on what to prepare during final presentation

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

1. Start creation of more levels to be applied to the level generation algorithm to create variety
2. Finish the gameplay loop of the game, providing the player with a win condition
3. Complete all necessary systems for the game such as AI, audio, UI and more

(APU: Serial Number)

PLS V1.0

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX (6) during the course of the project (SIX mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session.
7. The log sheet is an important deliverable for the project and an important record of a student's organisation and learning experience. The student must hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's name: Goh Shao Hang **Date:** 5/1/2023 **Meeting No:** 6

Project title: Creating A Highly Replayable Roguelike Game
Intake: APD3F2305CGD

Supervisor's name: Dr. Lai Ngan Kuen **Supervisor's signature:** .....

Items for discussion (noted by student before mandatory supervisory meeting):

1. Final checking of the final report
2. Inquire extra questions about the final submission of the project
3. Showcase the final outcome of the project
4. Finalization on the final presentation

Record of discussion (noted by student during mandatory supervisory meeting):

1. Displayed the final report and received feedback on what to fix and improve upon
2. Understood how to prepare and submit the poster and video required accordingly
3. Showcased the final game project and some playtesting
4. Confirmed the date of the final presentation

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

1. Change the final report according to supervisor feedback and get ready for submission
2. Finish preparing the video and posters
3. Prepare slides and provisional form for the upcoming presentation

Note: A student should make an appointment to meet his or her supervisor (via the consultation system) at least ONE (1) week prior to a mandatory supervisor session – please see document on project timelines. In the event a supervisor could not be booked for consultation, the project manager should be informed ONE (1) week prior to the session so that a meeting can be subsequently arranged.

Appendix D: Poster

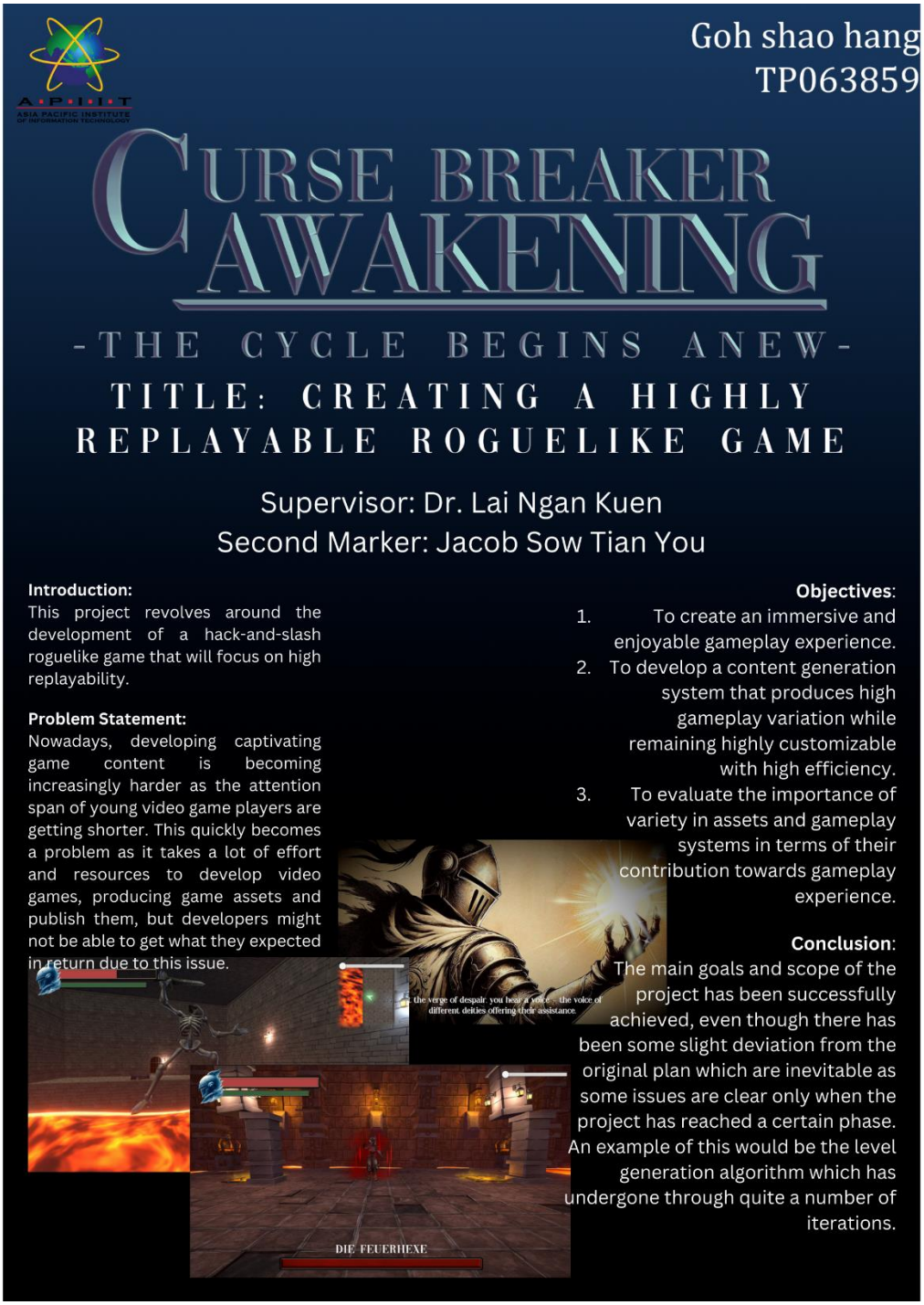


Figure 35 Poster

Appendix E: Gantt Chart

Task Name	Start Date	End Date	Duration (In Days)
Chapter 1: Introduction to the study			
1.1 Project Background	5/11/2023	6/11/2023	2
1.2 Project Aim	6/11/2023	6/11/2023	1
1.3 Objectives	6/11/2023	6/11/2023	1
1.4 Scopes	6/11/2023	6/11/2023	1
1.5 Potentail Benefits	8/11/2023	9/11/2023	1
1.6 Overview of the IR	10/11/2023	12/11/2023	3
1.7 Project Plan	7/1/2024	5/1/2024	1
Chapter 2: Literature Review			
2.1 Domain Research	14/11/2023	20/11/2023	7
2.2 Similar Systems / Works	21/11/2023	23/11/2023	3
2.3 Technical Research	24/11/2023	28/11/2023	5
Chapter 3: Methodology			
3.1 System Development Methodology	1/12/2023	3/12/2023	3
3.2 Data Gathering Design	4/12/2023	10/12/2023	7
3.3 Analysis	11/12/2023	12/12/2023	2
Chapter 4: Design and Implementation			
4.1 Introduction	13/12/2023	13/12/2023	1
4.2 System Design	14/12/2023	20/12/2023	7
4.3 Interface Design	21/12/2023	22/12/2023	2
4.4 Execution	22/12/2023	23/12/2023	2
4.5 Screenshot	23/12/2023	24/12/2023	2
4.6 Summary	25/12/2023	25/12/2023	1
Chapter 5: Result and Discussion			
5.1 Introduction	25/12/2023	25/12/2023	1
5.2 Testing Design / Plan	25/12/2023	27/12/2023	3
5.3 System Testing and Discussion	27/12/2023	28/12/2023	2
5.4 Summary	29/12/2023	29/12/2023	1
Chapter 6: Conclusion			
6.1 Critical Evaluation	2/1/2024	2/1/2024	1
6.2 Limitations	3/1/2024	3/1/2024	1
6.3 Recommendations	4/1/2024	4/1/2024	1

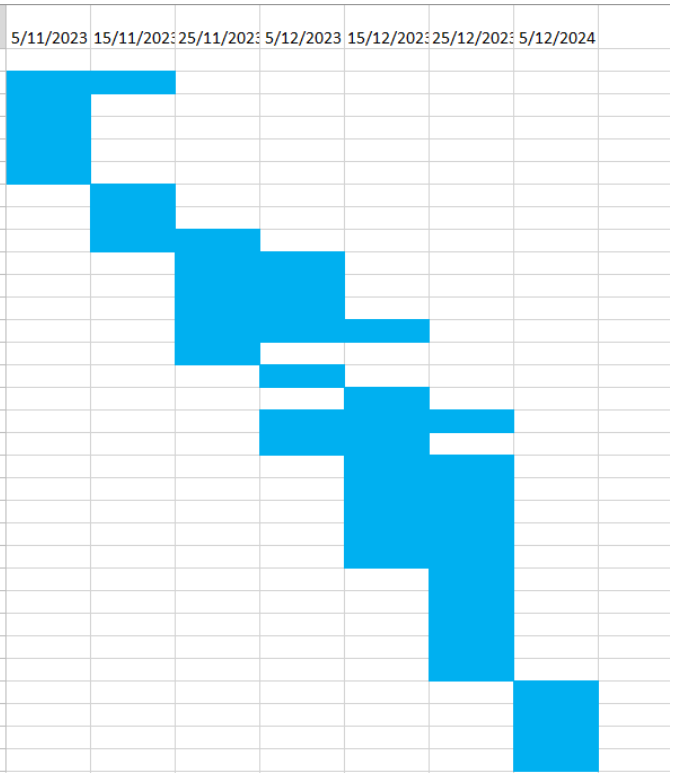


Figure 36 Gantt Chart

Appendix F: Sample Code Implementation

Level Generation

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using System;
using Random = UnityEngine.Random;
using GameCells.Utilities;

namespace CBA.LevelGeneration
{
    Unity Script (1 asset reference) | 3 references
    public class LevelGenerator : MonoBehaviour
    {
        [Header(GameData.SETTINGS)]
        [field: SerializeField] public Vector2Int BoardSize;
        [SerializeField] private Vector2Int _startPosition = Vector2Int.zero;
        [SerializeField] private int _maxIterations = 1000;
        [SerializeField] private int _maxRooms = 25;
        [SerializeField] private bool _allowLoops = true;
        [SerializeField] private bool _isolateFirstRoom = true;

        [Header(GameData.DEBUG)]
        [SerializeField] private float _delay;
        private float _startTime; //For debug

        public event Action<Cell> OnCellUpdate;
        public event Action OnGenerationCompleted;

        26 references
        public Cell[,] Board { get; private set; }
        private Dictionary<Cell, int> _cellIndexDict = new Dictionary<Cell, int>();

        private Vector2Int _currentCell;
        private Stack<Vector2Int> _path = new Stack<Vector2Int>();

        private int _roomCount = 0;

        Unity Message | 0 references
        private void Start()
        {
            StartCoroutine(Generate());
        }

    private IEnumerator Generate()
    {
        _startTime = Time.time;

        //Board Initialization
        Board = new Cell[BoardSize.x, BoardSize.y];
        for (int i = 0; i < BoardSize.x; i++)
        {
            for (int j = 0; j < BoardSize.y; j++)
            {
                Board[i, j] = new Cell(i, j);
            }
        }

        int iterations = 0;
        _currentCell = _startPosition;

        while (iterations <= _maxIterations) //Hard iteration limit to prevent endless looping
        {
            iterations++;

            if (!Board[_currentCell.x, _currentCell.y].Visited)
            {
                _roomCount++;

                //Assign index
                _cellIndexDict.Add(Board[_currentCell.x, _currentCell.y], _roomCount);

                OnCellUpdate?.Invoke(Board[_currentCell.x, _currentCell.y]);
            }

            //Board[_currentCell.x, _currentCell.y].Room.UpdateExits(Board[_currentCell.x, _currentCell.y].Exits);
            Board[_currentCell.x, _currentCell.y].Visited = true;

            if (_roomCount >= _maxRooms)
                break; //End iteration

            yield return WaitHandler.GetWaitForSeconds(_delay);

            List<Vector2Int> neighbours = CheckNeighbours(_currentCell);
```

```
80
81                  //If no available neighbours to extend towards for current cell, start backtracking
82                  if (neighbours.Count == 0)
83                  {
84                      if (_path.Count == 0)
85                      {
86                          //Backtracked to the beginning, exit now
87                          break;
88                      }
89                      else
90                      {
91                          //Backtrack to the previous cell (top of the stack) and check for other neighbours
92                          _currentCell = _path.Pop();
93                      }
94                  }
95                  //Neighbours found
96                  else
97                  {
98                      _path.Push(_currentCell); //Add this cell to the top of the stack
99
100                     //Pick a random neighbour to extend towards
101                     Vector2Int nextCell = neighbours[Random.Range(0, neighbours.Count)];
102
103                     //Up
104                     if (nextCell.y > _currentCell.y)
105                     {
106                         Board[_currentCell.x, _currentCell.y].Exits[0] = true; //Up exit is open
107                         Board[nextCell.x, nextCell.y].Exits[2] = true;
108                     }
109                     //Down
110                     else if (nextCell.y < _currentCell.y)
111                     {
112                         Board[_currentCell.x, _currentCell.y].Exits[2] = true; //Down exit is open
113                         Board[nextCell.x, nextCell.y].Exits[0] = true;
114                     }
115                     //Right
116                     else if (nextCell.x > _currentCell.x)
117                     {
118                         Board[_currentCell.x, _currentCell.y].Exits[1] = true; //Right exit is open
119                         Board[nextCell.x, nextCell.y].Exits[3] = true;
120                     }
121                     //Left
122                     else
123                     {
124                         Board[_currentCell.x, _currentCell.y].Exits[3] = true; //Left exit is open
125                         Board[nextCell.x, nextCell.y].Exits[1] = true;
126                     }
127
128                     OnCellUpdate?.Invoke(Board[_currentCell.x, _currentCell.y]);
129
130                     _currentCell = nextCell;
131                 }
132             }
133
134             OnGenerationCompleted?.Invoke();
```

```
136             Debug.LogWarning($"GENERATION COMPLETE IN {Time.time - _startTime} SECONDS");
137         }
138
            1 reference
139         public void UpdateAllCellTypes()
140         {
141             for (int i = 0; i < BoardSize.x; i++)
142             {
143                 // Loop through columns
144                 for (int j = 0; j < BoardSize.y; j++)
145                 {
146                     Cell cell = Board[i, j];
147                     if (cell.Visited)
148                     {
149                         cell.UpdateTypeAndRotation();
150                     }
151                 }
152             }
153         }
154
155
156         /// <summary>
157         /// Return a list of positions of nearby available cells that are not out of bounds
158         /// </summary>
159         /// <param name="cell"></param>
160         /// <returns></returns>
            1 reference
161         private List<Vector2Int> CheckNeighbours(Vector2Int cell)
162         {
163             List<Vector2Int> neighbours = new List<Vector2Int>();
164
165             //Each of the following directional checks checks if the adjacent cell to that direction is out of bounds, and if not, if it is not visited before.
166             //If both returns true, it is a valid neighbour to step towards.
167             //Additional Note: Keep in mind that the positive values on the board goes right and down.
168
169             //Left
170             if (cell.x - 1 >= 0)
171             {
172                 if (_allowLoops || !Board[cell.x - 1, cell.y].Visited) //Valid if loop allowed or cell is not yet visited
173                 {
174                     if (!_isolateFirstRoom || _startPosition != new Vector2Int(cell.x - 1, cell.y)) //Not valid if first room is isolated and neighbour is the first room
175                         neighbours.Add(new Vector2Int(cell.x - 1, cell.y));
176                 }
177             }
178
179             //Right
180             if (cell.x + 1 < BoardSize.x)
181             {
182                 if (_allowLoops || !Board[cell.x + 1, cell.y].Visited)
183                 {
184                     if (!_isolateFirstRoom || _startPosition != new Vector2Int(cell.x + 1, cell.y)) //Not valid if first room is isolated and neighbour is the first room
185                         neighbours.Add(new Vector2Int(cell.x + 1, cell.y));
186                 }
187             }
188
189             //Up
190             if (cell.y - 1 >= 0)
191             {
192                 if (_allowLoops || !Board[cell.x, cell.y - 1].Visited)
193                 {
194                     if (!_isolateFirstRoom || _startPosition != new Vector2Int(cell.x, cell.y - 1)) //Not valid if first room is isolated and neighbour is the first room
195                         neighbours.Add(new Vector2Int(cell.x, cell.y - 1));
196                 }
197             }
198
199             //Down
200             if (cell.y + 1 < BoardSize.y)
201             {
202                 if (_allowLoops || !Board[cell.x, cell.y + 1].Visited)
203                 {
204                     if (!_isolateFirstRoom || _startPosition != new Vector2Int(cell.x - 1, cell.y + 1)) //Not valid if first room is isolated and neighbour is the first room
205                         neighbours.Add(new Vector2Int(cell.x, cell.y + 1));
206                 }
207             }
208
209             return neighbours;
210         }
211
                4 references
212         public int GetCellIndex(Cell cell)
213         {
214             return _cellIndexDict[cell];
215         }
216     }
217 }
```

Figure 37 LevelGenerator.cs

```
using System;
using System.Collections.Generic;
using UnityEngine;
using Random = UnityEngine.Random;

namespace CBA.LevelGeneration
{
    Unity Script (2 asset references) | 2 references
    public class RoomSpawner : MonoBehaviour
    {
        [Header(GameData.DEPENDENCIES)]
        [SerializeField] private LevelGenerator _levelGenerator;
        [SerializeField] private RoomTypeData[] _roomTypeDatas;
        [SerializeField] private RoomSpawnerRule[] _rules;

        [Header(GameData.SETTINGS)]
        [SerializeField] private Vector2 _roomOffset;

        [Header(GameData.DEBUG)]
        [SerializeField] private bool _visualizeGeneration;

        public Dictionary<Cell, Room> RoomsDict = new Dictionary<Cell, Room>();

        private Dictionary<Cell, ERoomType> _roomTypeDict = new Dictionary<Cell, ERoomType>();
        private Dictionary<ERoomType, Dictionary<ERoomShape, List<Room>>> _roomsPrefabsDict = new Dictionary<ERoomType, Dictionary<ERoomShape, List<Room>>>();

        public event Action OnAllRoomsSpawned;

        Unity Message | 0 references
        private void Awake()
        {
            //Dictionary initialization
            foreach (var data in _roomTypeDatas)
            {
                if (!_roomsPrefabsDict.ContainsKey(data.RoomType))
                {
                    _roomsPrefabsDict.Add(data.RoomType, new Dictionary<ERoomShape, List<Room>>());

                    foreach (Room room in data.RoomPrefabs)
                    {
                        if (!_roomsPrefabsDict[data.RoomType].ContainsKey(room.RoomShape))
                        {
                            _roomsPrefabsDict[data.RoomType].Add(room.RoomShape, new List<Room>());
                        }
                        _roomsPrefabsDict[data.RoomType][room.RoomShape].Add(room);
                    }
                }
            }
        }

        Unity Message | 0 references
        private void OnEnable()
        {
            if (!_visualizeGeneration)
                _levelGenerator.OnGenerationCompleted += SpawnRooms;
            else
            {
                _levelGenerator.OnCellUpdate += SpawnIndividualRoom;
                _levelGenerator.OnGenerationCompleted += () => OnAllRoomsSpawned?.Invoke();
            }
        }

        Unity Message | 0 references
        private void OnDisable()
        {
            if (!_visualizeGeneration)
                _levelGenerator.OnGenerationCompleted -= SpawnRooms;
            else
            {
                _levelGenerator.OnCellUpdate -= SpawnIndividualRoom;
                _levelGenerator.OnGenerationCompleted -= () => OnAllRoomsSpawned?.Invoke();

            }
        }

        1 reference
        public ERoomType GetRoomType(Cell cell)
        {
            if (_roomTypeDict.ContainsKey(cell))
            {
                return _roomTypeDict[cell];
            }

            return ERoomType.Normal;
        }

```

```
83      private Room GetRandomRoomOfShapeAndType(ERoomShape shape, ERoomType type)
84      {
85          List<Room> PossibleRooms = _roomsPrefabsDict[type][shape];
86          return PossibleRooms[Random.Range(0, PossibleRooms.Count)];
87      }
88
        2 references
89      private void SpawnIndividualRoom(Cell cell)
90      {
91          cell.UpdateTypeAndRotation();
92
93          //Search for relevant rule
94          int roomIndex = _levelGenerator.GetCellIndex(cell);
95
96          ERoomType roomToSpawn = GetRoomTypeToSpawnFromRules(roomIndex);
97          Room room = GetRandomRoomOfShapeAndType(cell.RoomShape, roomToSpawn);
98
99          if (_roomTypeDict.ContainsKey(cell))
100         {
101             _roomTypeDict.Remove(cell);
102         }
103         _roomTypeDict.Add(cell, roomToSpawn);
104
105         if (RoomsDict.ContainsKey(cell))
106         {
107             Destroy(RoomsDict[cell].gameObject);
108             RoomsDict.Remove(cell);
109         }
110
111         RoomsDict.Add(cell, Instantiate(room, new Vector3(cell.Position.x * _roomOffset.x, 0, cell.Position.y * _roomOffset.y), Quaternion.Euler(0f, cell.RoomRotation, 0f), transform));
112     }
113
114
```

```
115     private void SpawnRooms()
116     {
117         _levelGenerator.UpdateAllCellTypes();
118
119         for (int i = 0; i < _levelGenerator.BoardSize.x; i++)
120         {
121             for (int j = 0; j < _levelGenerator.BoardSize.y; j++)
122             {
123                 Cell currentCell = _levelGenerator.Board[i, j];
124                 if (currentCell.Visited)
125                 {
126                     //Search for relevant rule
127                     int roomIndex = _levelGenerator.GetCellIndex(currentCell);
128
129                     ERoomType roomToSpawn = GetRoomTypeToSpawnFromRules(roomIndex);
130                     Room room = GetRandomRoomOfShapeAndType(currentCell.RoomShape, roomToSpawn);
131
132                     if (!_roomTypeDict.ContainsKey(currentCell))
133                     {
134                         _roomTypeDict.Add(currentCell, roomToSpawn);
135                     }
136
137                     RoomsDict.Add(currentCell, Instantiate(room, new Vector3(i * _roomOffset.x, 0, j * _roomOffset.y), Quaternion.Euler(0f, currentCell.RoomRotation, 0f), transform));
138                 }
139             }
140         }
141
142         OnAllRoomsSpawned?.Invoke();
143     }
144
```

```
145         private ERoomType GetRoomTypeToSpawnFromRules(int roomIndex)
146         {
147             ERoomType roomToSpawn = ERoomType.Normal;
148 
149             List<RoomSpawnerRule> rulesSatisfied = new List<RoomSpawnerRule>();
150             float totalSpawnChance = 0f;
151             foreach (var rule in _rules)
152             {
153                 if (roomIndex >= rule.MinSpawnIndex && roomIndex <= rule.MaxSpawnIndex)
154                 {
155                     rulesSatisfied.Add(rule);
156                     totalSpawnChance += rule.SpawnChance;
157                 }
158             }
159 
160             float random = Random.Range(0, totalSpawnChance);
161             float cumulativeChance = 0f;
162             foreach (var rule in rulesSatisfied)
163             {
164                 cumulativeChance += rule.SpawnChance;
165 
166                 if (random <= cumulativeChance)
167                 {
168                     roomToSpawn = rule.roomType;
169                     break;
170                 }
171             }
172 
173             return roomToSpawn;
174         }
175 
176 
                Unity Message | 0 references
177         private void OnValidate()
178         {
179             foreach (var rule in _rules)
180             {
181                 rule.UpdateName();
182             }
183 
184             foreach (var data in _roomTypeDatas)
185             {
186                 data.UpdateName();
187             }
188         }
189     }
190 
```

```
191     [Serializable]
        1 reference
192     public class RoomTypeData
193     {
194         [HideInInspector] public string RoomTypeName;
195
196         public ERoomType RoomType = ERoomType.Normal;
197         public Room[] RoomPrefabs;
198
            1 reference
199         public void UpdateName()
200         {
201             RoomTypeName = RoomType.ToString();
202         }
203     }
```

Figure 38 RoomSpawner.cs

```
using CBA.Core;
using CBA.Entities.Player;
using DG.Tweening;
using GameCells.Utilities;
using System.Collections.Generic;
using UnityEngine;

namespace CBA.LevelGeneration
{
    Unity Script (1 asset reference) | 2 references
    public class LevelManager : Singleton<LevelManager>
    {
        [Header(GameData.DEPENDENCIES)]
        [SerializeField] private LevelGenerator _levelGenerator;
        [SerializeField] private RoomSpawner _roomSpawner;
        [SerializeField] private MapRenderer _mapRenderer;
        [SerializeField] private CanvasGroup _transitionCanvas;

        private const float _roomTransitionDuration = 0.5f;

        private Dictionary<Cell, Room> _roomsDict;
        private Cell _currentCell;
        13 references
        private Room _currentRoom => _roomsDict[_currentCell];
        private Transform _safePoint; //Last entrance used will be determined as the safepoint.

        private PlayerManager _player;
        4 references
        private PlayerController _playerController => _player.PlayerController;
        3 references
        private PlayerCameraController _playerCameraController => _player.PlayerCameraController;


        private bool _isTransitioning;

        Unity Message | 0 references
        private void Start()
        {
            _transitionCanvas.alpha = 1;
            _isTransitioning = true;
        }
```

```
        private void OnEnable()
        {
            _roomSpawner.OnAllRoomsSpawned += StartLevel;
        }

        Unity Message | 0 references
        private void OnDisable()
        {
            _roomSpawner.OnAllRoomsSpawned -= StartLevel;
        }
```

```
48          public void StartLevel()
49          {
50              _roomsDict = _roomSpawner.RoomsDict;
51
52              foreach (var room in _roomsDict)
53              {
54                  //Activate if is first room
55                  if (_levelGenerator.GetCellIndex(room.Key) == 1)
56                  {
57                      _currentCell = room.Key;
58                      _currentRoom.gameObject.SetActive(true);
59                  }
60                  else
61                  {
62                      room.Value.gameObject.SetActive(false);
63                  }
64
65              }
66
67              #region Initialize Player
68              _player = GameManager.Instance.PlayerManager;
69              _player.ActivateComponents(true);
70
71              _playerController.transform.position = _currentRoom.Exits[1].SpawnPoint.position;
72              _playerCameraController.SetCameraRotation(_currentRoom.Exits[1].SpawnPoint.rotation.eulerAngles.y, _currentRoom.Exits[1].SpawnPoint.rotation.eulerAngles.x);
73              _playerController.gameObject.SetActive(true);
74              #endregion
75
76              _mapRenderer?.SetCurrentRoom(_currentCell);
77              _currentRoom.OnPlayerExitRoom += TransitionToRoom;
78              _currentRoom.OnPlayerEnter();
79
80              _safePoint = _currentRoom.Exits[1].SpawnPoint;
81
82              AudioManager.Instance.CrossFadeBGM("ExplorationTheme_1", 2f, false);
83
84              _transitionCanvas.DOFade(0, 2f).OnComplete(() => _isTransitioning = false);
85          }
86
```

```
87          public void TransitionToRoom(EExitDirection exitDirection)
88          {
89              if (_isTransitioning)
90                  return;
91
92              _isTransitioning = true;
93
94              _transitionCanvas.DOFade(1, _roomTransitionDuration).OnComplete(() => LoadNextRoom(exitDirection));
95          }
96
            1 reference
97          private void LoadNextRoom(EExitDirection exitDirection)
98          {
99              _currentRoom.gameObject.SetActive(false);
100             _currentRoom.OnPlayerExitRoom -= TransitionToRoom;
101
102             //Adjust direction considering rotation of current cell
103             int adjustedDirection = ((int)exitDirection + (_currentCell.RoomRotation / 90)) % 4;
104
105             Cell nextCell = GetNextCell(adjustedDirection);
106             int entranceDirection = ((adjustedDirection + 2 - (nextCell.RoomRotation / 90))) % 4;
107             if (entranceDirection < 0)
108             {
109                 entranceDirection += 4;
110             }
111
112             EExitDirection nextRoomEntrance = (EExitDirection)entranceDirection;
113
114             _currentCell = nextCell;
115             _currentRoom.gameObject.SetActive(true);
116             _mapRenderer?.SetCurrentRoom(_currentCell);
117
118             Exit entrance = _currentRoom.GetEntrance(nextRoomEntrance);
119             _playerController.transform.position = entrance.SpawnPoint.position;
120             _playerCameraController.SetCameraRotation(entrance.SpawnPoint.rotation.eulerAngles.y, entrance.SpawnPoint.rotation.eulerAngles.x);
121             //_playerReference.gameObject.SetActive(true);
122
123             _safePoint = entrance.SpawnPoint;
124
125             _currentRoom.OnPlayerExitRoom += TransitionToRoom;
126             _currentRoom.OnPlayerEnter();
127
128             _transitionCanvas.DOFade(0, _roomTransitionDuration).OnComplete(() => _isTransitioning = false);
129         }
130
```

```
131         private Cell GetNextCell(int direction)
132         {
133             switch (direction)
134             {
135                 case 0:
136                     return _levelGenerator.Board[_currentCell.Position.x, _currentCell.Position.y + 1];
137                 case 1:
138                     return _levelGenerator.Board[_currentCell.Position.x + 1, _currentCell.Position.y];
139                 case 2:
140                     return _levelGenerator.Board[_currentCell.Position.x, _currentCell.Position.y - 1];
141                 case 3:
142                     return _levelGenerator.Board[_currentCell.Position.x - 1, _currentCell.Position.y];
143                 default:
144                     return _currentCell;
145             }
146         }
147
                1 reference
148         public async void TeleportPlayerToSafePoint()
149         {
150             if (_isTransitioning)
151                 return;
152
153             _isTransitioning = true;
154
155             await _transitionCanvas.DOFade(1, _roomTransitionDuration).AsyncWaitForCompletion();
156
157             _playerController.transform.position = _safePoint.position;
158             _playerCameraController.SetCameraRotation(_safePoint.rotation.eulerAngles.y, _safePoint.rotation.eulerAngles.x);
159
160             await _transitionCanvas.DOFade(0, _roomTransitionDuration).AsyncWaitForCompletion();
161
162             _isTransitioning = false;
163         }
164     }
165 }
```

Figure 39 LevelManager.cs

Player Controller

```
using GameCells.Entities;
using GameCells.Utilities;
using System;
using System.Collections;
using UnityEngine;

namespace CBA.Entities.Player
{
    Unity Script (2 asset references) | 14 references
    public class PlayerController : StateManager<EPlayerMovementState>
    {
        [Header(GameData.DEPENDENCIES)]
        [SerializeField] private PlayerInputHandler _playerInputHandler;
        [SerializeField] private PlayerCameraController _playerCameraController;
        [SerializeField] private PlayerCombatManager _playerCombatManager;
        [SerializeField] private Transform _cameraRootTransform;
        [SerializeField] private MovementModule _movementModule;
        [SerializeField] private CapsuleCollider _playerCollider;
        [SerializeField] private PhysicsQuery _groundChecker;
        [SerializeField] private RaycastCheck _slopeChecker;

        [Header("Movement Settings")]
        [SerializeField] private float _walkMovementForce = 50f;
        [SerializeField] private float _sprintMovementForce = 100f;
        [SerializeField] private float _crouchMovementForce = 30f;
        [SerializeField] private float _maxSpeed = 15f;
        [SerializeField] private float _groundDrag = 7f;
        [SerializeField] private float _jumpForce = 5f;
        [SerializeField] private float _jumpBufferDuration = 0.2f;
        [SerializeField] private float _airMovementMultiplier = 0.1f;
        [SerializeField] private float _maxSlopeAngle = 45f;

        [Header("Stamina Settings")]
        [SerializeField] private float _maxStamina = 100f;
        [SerializeField] private float _staminaRegenRate = 20f;
        [SerializeField] private float _sprintingStaminaConsumption = 10f;
        [SerializeField] private float _wallRunStaminaConsumption = 5f;
        [SerializeField] private float _staminaRegenDelay = 1f;

        [Header("Wall Run")]
        [SerializeField] private bool _canWallRun = true;
        [SerializeField] private float _wallRunMovementForce = 50f;
        [SerializeField] private float _wallCheckDistance = 0.5f;
        [SerializeField] private float _minWallRunHeight;
        [SerializeField] private float _wallRunGravity = 3f;
        [SerializeField] private float _wallRunCameraTilt = 10f;
        [SerializeField] private LayerMask _wallRunLayers;

```

```
39          [Header("Wall Run")]
40          [SerializeField] private bool _canWallRun = true;
41          [SerializeField] private float _wallRunMovementForce = 50f;
42          [SerializeField] private float _wallCheckDistance = 0.5f;
43          [SerializeField] private float _minWallRunHeight;
44          [SerializeField] private float _wallRunGravity = 3f;
45          [SerializeField] private float _wallRunCameraTilt = 10f;
46          [SerializeField] private LayerMask _wallRunLayers;
47
48          [Header("Charge Attack")]
49          [SerializeField] private float _minChargedAttackMovementSpeed = 10f;
50          [SerializeField] private float _maxChargedAttackMovementSpeed = 15f;
51
52          #region Getters
53          //Dependencies
            11 references
54          public PlayerInputHandler PlayerInputHandler => _playerInputHandler;
            0 references
55          public PlayerCameraController PlayerCameraController => _playerCameraController;
            6 references
56          public PlayerCombatManager PlayerCombatManager => _playerCombatManager;
            8 references
57          public MovementModule MovementModule => _movementModule;
            2 references
58          public CapsuleCollider PlayerCollider => _playerCollider;
            2 references
59          public PhysicsQuery GroundChecker => _groundChecker;
            4 references
60          public Transform CameraRootTransform => _cameraRootTransform;
61
```

```
74          //Wall Run
            1 reference
75          public float WallRunGravity => _wallRunGravity;
            2 references
76          public float WallRunCameraTilt => _wallRunCameraTilt;
            2 references
77          public bool CanWallRun => _canWallRun;
            3 references
78          public bool IsRunnableWallDetected
79          {
80              get
81              {
82                  if (Physics.Raycast(transform.position, Vector3.down, _minWallRunHeight, _wallRunLayers))
83                  {
84                      return false;
85                  }
86
87                  if (Physics.Raycast(transform.position, _cameraRootTransform.right, out runnableWallHitInfo, _wallCheckDistance, _wallRunLayers))
88                  {
89                      return true;
90                  }
91                  else if (Physics.Raycast(transform.position, -_cameraRootTransform.right, out runnableWallHitInfo, _wallCheckDistance, _wallRunLa
92                  {
93                      return true;
94                  }
95
96                  return false;
97              }
98          }
99          public RaycastHit runnableWallHitInfo;
100
101         //Charged Attack
            1 reference
102         public float MinChargedAttackMovementSpeed => _minChargedAttackMovementSpeed;
            1 reference
103         public float MaxChargedAttackMovementSpeed => _maxChargedAttackMovementSpeed;
            3 references
104         public bool IsChargingAttack { get; private set; }
            6 references
105         public bool IsPerformingChargedAttack { get; private set; }
            2 references
106         public float LastChargePercentage { get; private set; } = 0f;
107
```

```
        //Physics
        6 references
        public bool IsGrounded => _groundChecker.Hit();
        2 references
        public bool IsOnSlope
        {
            get
            {
                float slopeAngle = Vector3.Angle(Vector3.up, _slopeChecker.HitInfo.normal);
                return _slopeChecker.Hit() && slopeAngle != 0 && slopeAngle < _maxSlopeAngle;
            }
        }
        2 references
        public Vector3 ColliderBottomPoint => _playerCollider.center - (Vector3.up * (_playerCollider.height * 0.5f));
        #endregion

        #region Variables
        private Coroutine _jumpBufferCO = null;

        private float _movementForce;
        private float _staminaRegenTimer;
        private Vector3 _moveDirection;
        private Vector3 _horizontalVelocity;
        #endregion

        #region Events
        public event Action OnStaminaChanged;
        public event Action<bool> OnWallRunStarted;
        public event Action OnWallRunEnded;
        #endregion
```

```
    protected override void Awake()
    {
        base.Awake();

        //State Machine Initialization
        _StatesDict.Add(EPlayerMovementState.Walk, new PlayerWalkState(EPlayerMovementState.Walk, this));
        _StatesDict.Add(EPlayerMovementState.Sprint, new PlayerSprintState(EPlayerMovementState.Sprint, this));
        _StatesDict.Add(EPlayerMovementState.Crouch, new PlayerCrouchState(EPlayerMovementState.Crouch, this));
        _StatesDict.Add(EPlayerMovementState.WallRun, new PlayerWallRunState(EPlayerMovementState.WallRun, this));
        _StatesDict.Add(EPlayerMovementState.ChargedAttack, new PlayerChargedAttackState(EPlayerMovementState.ChargedAttack, this, this._playerCameraController));

        _currentState = _StatesDict[EPlayerMovementState.Walk];

        //Initialize Variables
        CurrentStamina = MaxStamina;

        IsLimitingMaxSpeed = true;
    }

    Unity Message | 0 references
    private void OnEnable()
    {
        _playerInputHandler.OnJumpInput += OnJumpPressed;
        _playerCombatManager.OnChargingStarted += OnChargingStarted;
        _playerCombatManager.OnChargedAttackReleased += OnChargedAttackReleased;
        _playerCombatManager.OnChargedAttackEnded += OnChargedAttackEnded;
    }

    Unity Message | 0 references
    private void OnDisable()
    {
        _playerInputHandler.OnJumpInput -= OnJumpPressed;
        _playerCombatManager.OnChargingStarted -= OnChargingStarted;
        _playerCombatManager.OnChargedAttackEnded -= OnChargedAttackEnded;
    }
```

```
170         protected override void Update()
171         {
172             base.Update();
173 
174             _movementModule.SetDrag(IsGrounded ? _groundDrag : 0f);
175 
176             if (JumpBuffer && IsGrounded)
177             {
178                 Jump();
179             }
180 
181             LimitMaxSpeed();
182 
183             RegenerateStamina();
184         }
            Unity Message | 2 references
185         protected override void FixedUpdate()
186         {
187             base.FixedUpdate();
188 
189             HandleMovement();
190         }
191 
```

```
192         private void HandleMovement()
193         {
194             if (_movementForce == 0f)
195                 return;
196 
197             _moveDirection = _cameraRootTransform.right * _playerInputHandler.MoveInput.x
198                 + _cameraRootTransform.transform.forward * _playerInputHandler.MoveInput.y;
199 
200             if (IsOnSlope)
201             {
202                 _moveDirection = Vector3.ProjectOnPlane(_moveDirection, _slopeChecker.HitInfo.normal);
203 
204                 if (_playerInputHandler.MoveInput == Vector2.zero && IsGrounded)
205                 {
206                     _movementModule.SetUseGravity(false);
207                 }
208                 else
209                 {
210                     _movementModule.SetUseGravity(true);
211                 }
212             }
213             else
214             {
215                 if (_currentState.StateKey != EPlayerMovementState.WallRun)
216                 {
217                     _movementModule.SetUseGravity(true);
218                 }
219             }
220 
221             float finalMovementForce = _movementForce * (IsGrounded ? 1 : _airMovementMultiplier);
222             _movementModule.AddForce(_moveDirection * finalMovementForce, ForceMode.Acceleration);
223         }
224 
```

```
225         private void LimitMaxSpeed()
226         {
227             if (!IsLimitingMaxSpeed)
228                 return;
229 
230             if (!IsOnSlope)
231             {
232                 _horizontalVelocity.Set(_movementModule.CurrentVelocity.x, 0f, _movementModule.CurrentVelocity.z);
233 
234                 if (_horizontalVelocity.magnitude > _maxSpeed)
235                 {
236                     _horizontalVelocity = _horizontalVelocity.normalized * _maxSpeed;
237                     _movementModule.SetVelocity(_horizontalVelocity.x, _movementModule.CurrentVelocity.y, _horizontalVelocity.z);
238                 }
239             }
240             else
241             {
242                 if (_movementModule.CurrentVelocity.magnitude > _maxSpeed)
243                 {
244                     _movementModule.SetVelocity(_movementModule.CurrentVelocity.normalized * _maxSpeed);
245                 }
246             }
247 
248         }
249 
                2 references
250         public void SetIsLimitingMaxSpeed(bool set)
251         {
252             IsLimitingMaxSpeed = set;
253         }
254 
                2 references
255         private void OnJumpPressed()
256         {
257             //Set jump buffer that can be consumed upon request to perform certain actions, etc. jump and wall jump
258             JumpBuffer = true;
259 
260             if (_jumpBufferCO != null)
261             {
262                 StopCoroutine(_jumpBufferCO);
263             }
264 
265             _jumpBufferCO = StartCoroutine(ResetJumpBufferCO());
266         }
267 
```

```
268         public void ConsumeJumpBuffer()
269         {
270             JumpBuffer = false;
271
272             if (_jumpBufferCO != null)
273             {
274                 StopCoroutine(_jumpBufferCO);
275                 _jumpBufferCO = null;
276             }
277         }
278
                1 reference
279         private IEnumerator ResetJumpBufferCO()
280         {
281             yield return WaitHandler.GetWaitForSeconds(_jumpBufferDuration);
282
283             ConsumeJumpBuffer();
284         }
285
                1 reference
286         private void Jump()
287         {
288             ConsumeJumpBuffer();
289
290             //Reset Y velocity
291             _movementModule.SetYVelocity(0f);
292
293             //Add jump force
294             _movementModule.AddForce(Vector3.up * _jumpForce, ForceMode.Impulse);
295         }
296
                4 references
297         public void SetMovementForce(float movementForce)
298         {
299             this._movementForce = movementForce;
300         }
301
```

```
302     public void RegenerateStamina()
303     {
304         if (CurrentStamina >= MaxStamina)
305             return;
306
307         if (_staminaRegenTimer <= 0f)
308         {
309             _staminaRegenTimer = 0f;
310             SetStamina(CurrentStamina + _staminaRegenRate * Time.deltaTime);
311         }
312         else
313         {
314             _staminaRegenTimer -= Time.deltaTime;
315         }
316     }
317
        4 references
318     public void SetStamina(float value)
319     {
320         CurrentStamina = Mathf.Clamp(value, 0f, MaxStamina);
321
322         OnStaminaChanged?.Invoke();
323     }
324
        3 references
325     public void StartStaminaRegenTimer()...
329
        1 reference
330     public void WallRunStarted(bool tiltRight)
331     {
332         OnWallRunStarted?.Invoke(tiltRight);
333     }
334
        1 reference
335     public void WallRunEnded()
336     {
337         OnWallRunEnded?.Invoke();
338     }
339
        2 references
340     public void OnChargingStarted()
341     {
342         IsChargingAttack = true;
343     }
344
345     public void OnChargedAttackReleased(float chargePercentage)
346     {
347         IsChargingAttack = false;
348         LastChargePercentage = chargePercentage;
349         IsPerformingChargedAttack = true;
350     }
351
        2 references
352     public void OnChargedAttackEnded()
353     {
354         IsPerformingChargedAttack = false;
355     }
356
357 }
358 }
```

Figure 40 PlayerController.cs

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

namespace CBA.Entities.Player
{
    13 references
    public abstract class PlayerMovementBaseState : BaseState<EPlayerMovementState>
    {
        protected PlayerController _playerController;

        6 references
        protected PlayerMovementBaseState(EPlayerMovementState key, PlayerController playerController) : base(key)
        {
            this._playerController = playerController;
        }
    }
}
```

Figure 41 PlayerMovementBaseState.cs

```
using UnityEngine;

namespace CBA.Entities.Player
{
    2 references
    public class PlayerWalkState : PlayerMovementBaseState
    {
        1 reference
        public PlayerWalkState(EPlayerMovementState key, PlayerController playerController) : base(key, playerController)
        {
        }

        8 references
        public override void Enter()
        {
            _playerController.SetMovementForce(_playerController.WalkMovementForce);
        }

        2 references
        public override EPlayerMovementState GetNextState()
        {
            if (_playerController.IsPerformingChargedAttack)
            {
                return EPlayerMovementState.ChargedAttack;
            }

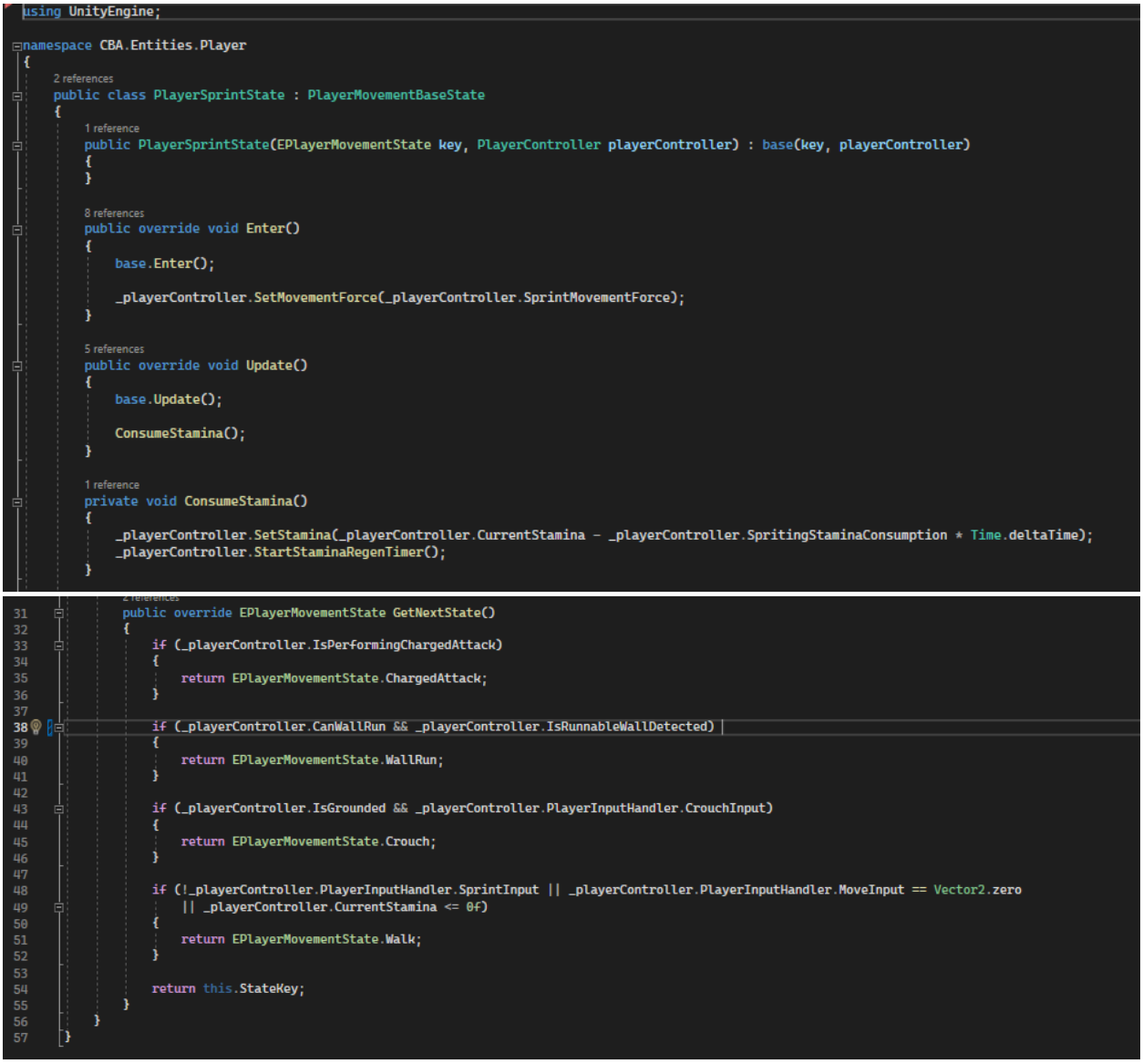
            if (_playerController.CanWallRun && _playerController.IsRunnableWallDetected)
            {
                return EPlayerMovementState.WallRun;
            }

            if (_playerController.IsGrounded && _playerController.PlayerInputHandler.CrouchInput)
            {
                return EPlayerMovementState.Crouch;
            }

            if (_playerController.CurrentStamina > 0 && _playerController.PlayerInputHandler.SprintInput
                && _playerController.PlayerInputHandler.MoveInput != Vector2.zero)
            {
                return EPlayerMovementState.Sprint;
            }

            return this.StateKey;
        }
    }
}
```

Figure 42 PlayerWalkState.cs



```
using UnityEngine;

namespace CBA.Entities.Player
{
    2 references
    public class PlayerSprintState : PlayerMovementBaseState
    {
        1 reference
        public PlayerSprintState(EPlayerMovementState key, PlayerController playerController) : base(key, playerController)
        {
        }

        8 references
        public override void Enter()
        {
            base.Enter();

            _playerController.SetMovementForce(_playerController.SprintMovementForce);
        }

        5 references
        public override void Update()
        {
            base.Update();

            ConsumeStamina();
        }

        1 reference
        private void ConsumeStamina()
        {
            _playerController.SetStamina(_playerController.CurrentStamina - _playerController.SpritingStaminaConsumption * Time.deltaTime);
            _playerController.StartStaminaRegenTimer();
        }

        2 references
        public override EPlayerMovementState GetNextState()
        {
            if (_playerController.IsPerformingChargedAttack)
            {
                return EPlayerMovementState.ChargedAttack;
            }

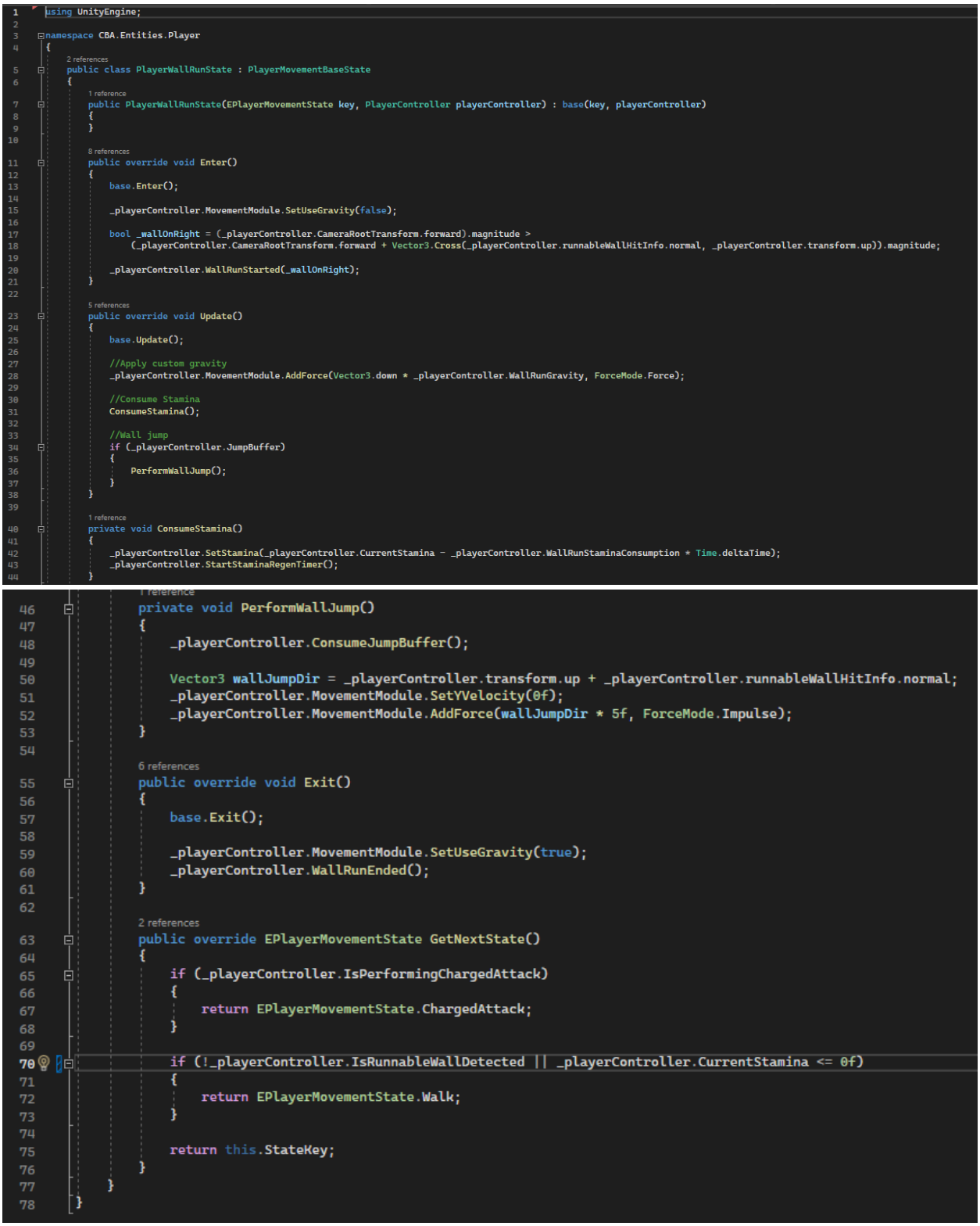
            if (_playerController.CanWallRun && _playerController.IsRunnableWallDetected)
            {
                return EPlayerMovementState.WallRun;
            }

            if (_playerController.IsGrounded && _playerController.PlayerInputHandler.CrouchInput)
            {
                return EPlayerMovementState.Crouch;
            }

            if (!_playerController.PlayerInputHandler.SprintInput || _playerController.PlayerInputHandler.MoveInput == Vector2.zero
                || _playerController.CurrentStamina <= 0f)
            {
                return EPlayerMovementState.Walk;
            }

            return this.StateKey;
        }
    }
}
```

Figure 43 PlayerSprintState.cs



```
using UnityEngine;

namespace CBA.Entities.Player
{
    2 references
    public class PlayerWallRunState : PlayerMovementBaseState
    {
        1 reference
        public PlayerWallRunState(EPlayerMovementState key, PlayerController playerController) : base(key, playerController)
        {
        }

        8 references
        public override void Enter()
        {
            base.Enter();

            _playerController.MovementModule.SetUseGravity(false);

            bool _wallOnRight = (_playerController.CameraRootTransform.forward).magnitude >
                (_playerController.CameraRootTransform.forward + Vector3.Cross(_playerController.runnableWallHitInfo.normal, _playerController.transform.up)).magnitude;

            _playerController.WallRunStarted(_wallOnRight);
        }

        5 references
        public override void Update()
        {
            base.Update();

            //Apply custom gravity
            _playerController.MovementModule.AddForce(Vector3.down * _playerController.WallRunGravity, ForceMode.Force);

            //Consume Stamina
            ConsumeStamina();

            //Wall jump
            if (_playerController.JumpBuffer)
            {
                PerformWallJump();
            }
        }

        1 reference
        private void ConsumeStamina()
        {
            _playerController.SetStamina(_playerController.CurrentStamina - _playerController.WallRunStaminaConsumption * Time.deltaTime);
            _playerController.StartStaminaRegenTimer();
        }

        1 reference
        private void PerformWallJump()
        {
            _playerController.ConsumeJumpBuffer();

            Vector3 wallJumpDir = _playerController.transform.up + _playerController.runnableWallHitInfo.normal;
            _playerController.MovementModule.SetYVelocity(0f);
            _playerController.MovementModule.AddForce(wallJumpDir * 5f, ForceMode.Impulse);
        }

        6 references
        public override void Exit()
        {
            base.Exit();

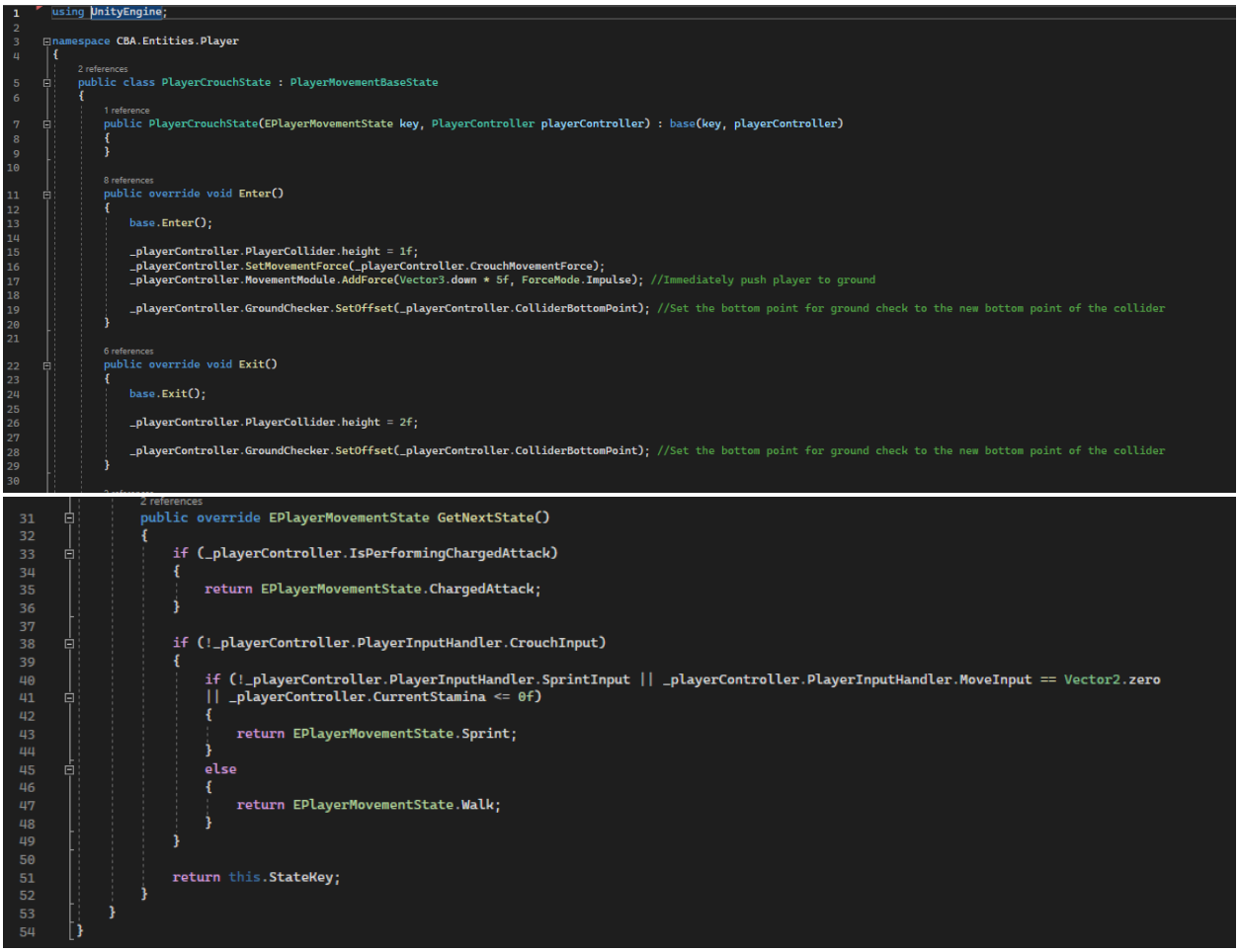
            _playerController.MovementModule.SetUseGravity(true);
            _playerController.WallRunEnded();
        }

        2 references
        public override EPlayerMovementState GetNextState()
        {
            if (_playerController.IsPerformingChargedAttack)
            {
                return EPlayerMovementState.ChargedAttack;
            }

            if (!_playerController.IsRunnableWallDetected || _playerController.CurrentStamina <= 0f)
            {
                return EPlayerMovementState.Walk;
            }

            return this.StateKey;
        }
    }
}
```

Figure 44 PlayerWallRunState.cs



```
using UnityEngine;

namespace CBA.Entities.Player
{
    2 references
    public class PlayerCrouchState : PlayerMovementBaseState
    {
        1 reference
        public PlayerCrouchState(EPlayerMovementState key, PlayerController playerController) : base(key, playerController)
        {
        }

        8 references
        public override void Enter()
        {
            base.Enter();

            _playerController.PlayerCollider.height = 1f;
            _playerController.SetMovementForce(_playerController.CrouchMovementForce);
            _playerController.MovementModule.AddForce(Vector3.down * 5f, ForceMode.Impulse); //Immediately push player to ground

            _playerController.GroundChecker.SetOffset(_playerController.ColliderBottomPoint); //Set the bottom point for ground check to the new bottom point of the collider
        }

        6 references
        public override void Exit()
        {
            base.Exit();

            _playerController.PlayerCollider.height = 2f;

            _playerController.GroundChecker.SetOffset(_playerController.ColliderBottomPoint); //Set the bottom point for ground check to the new bottom point of the collider
        }

        2 references
        public override EPlayerMovementState GetNextState()
        {
            if (_playerController.IsPerformingChargedAttack)
            {
                return EPlayerMovementState.ChargedAttack;
            }

            if (!_playerController.PlayerInputHandler.CrouchInput)
            {
                if (!_playerController.PlayerInputHandler.SprintInput || _playerController.PlayerInputHandler.MoveInput == Vector2.zero
                || _playerController.CurrentStamina <= 0f)
                {
                    return EPlayerMovementState.Sprint;
                }
                else
                {
                    return EPlayerMovementState.Walk;
                }
            }

            return this.StateKey;
        }
    }
}
```

Figure 45 PlayerCrouchState.cs

```
using UnityEngine;

namespace CBA.Entities.Player
{
    2 references
    public class PlayerChargedAttackState : PlayerMovementBaseState
    {
        private float _chargedAttackMovementSpeed;
        private Vector3 _chargedAttackDirection;
        private bool _chargeAttackEnded;

        private readonly PlayerCameraController _playerCameraController;

        1 reference
        public PlayerChargedAttackState(EPlayerMovementState key, PlayerController playerController, PlayerCameraController playerCameraController) : base(key, playerController)
        {
            this._playerCameraController = playerCameraController;
        }

        8 references
        public override void Enter()
        {
            base.Enter();

            _chargeAttackEnded = false;

            _playerController.PlayerCombatManager.OnChargedAttackEnded += OnChargeAttackEnded;

            _playerController.SetMovementForce(0f);

            //Allow player to go through enemies. Another dedicated hitbox will be used to detect hits
            Physics.IgnoreLayerCollision(GameData.PLAYER_LAYER_INDEX, GameData.DAMAGEABLE_LAYER_INDEX, true);

            _chargedAttackMovementSpeed = Mathf.Lerp(_playerController.MinChargedAttackMovementSpeed, _playerController.MaxChargedAttackMovementSpeed,
                _playerController.LastChargePercentage);

            _chargedAttackDirection = _playerCameraController.transform.forward;

            _playerController.MovementModule.SetVelocity(_chargedAttackDirection * _chargedAttackMovementSpeed);
            _playerController.SetIsLimitingMaxSpeed(false);
        }

        2 references
        private void OnChargeAttackEnded()
        {
            _chargeAttackEnded = true;
        }

        5 references
        public override void Update()
        {
            base.Update();

            _playerController.MovementModule.SetVelocity(_chargedAttackDirection * _chargedAttackMovementSpeed);
        }

        6 references
        public override void Exit()
        {
            base.Exit();

            _playerController.PlayerCombatManager.OnChargedAttackEnded -= OnChargeAttackEnded;

            Physics.IgnoreLayerCollision(GameData.PLAYER_LAYER_INDEX, GameData.DAMAGEABLE_LAYER_INDEX, false);

            _playerController.SetIsLimitingMaxSpeed(true);

        }

        2 references
        public override EPlayerMovementState GetNextState()
        {
            if (_chargeAttackEnded)
            {
                return EPlayerMovementState.Walk;
            }

            return this.StateKey;
        }
    }
}
```

Figure 46 PlayerChargedAttackState.cs

Appendix G: Respondence Demographics Profile

Name	Age Range	Gender	Occupation
Robbie	18 - 22	Male	Student
Nicholas	18 - 22	Male	Student
KahWui	18 - 22	Male	Student
KahHou	18 - 22	Male	Student
Shaun	18 - 22	Male	Unemployed
CheeBing	18 - 22	Male	Student
Flint	18 - 22	Male	Student
Ivan	23 - 27	Male	Student
Anonymous	23 - 27	Male	Student
Moses	18 - 22	Male	Student
ZhengKai	18 - 22	Male	Student
NSK	18 - 22	Male	Student
ZhiXuan	18 - 22	Female	Employed
Cheryl	23 - 27	Female	Student
Hebe	18 - 22	Female	Student
Anonymous	23 - 27	Male	Student
ZhiKai	18 - 22	Male	Student
Justin	18 - 22	Male	Student
Anonymous	18 - 22	Male	Student

Tan	18 - 22	Female	Student
Kxy	18 - 22	Male	Student
Wenyi	18 - 22	Female	Student
Marcus	23 - 27	Male	Employed
Anonymous	18 - 22	Male	Student
Chew	18 - 22	Male	Student
Lee	23 - 27	Male	Student
Anonymous	18 - 22	Male	Student
Anonymous	18 - 22	Male	Student
Jianseng	18 - 22	Male	Employed
David	18 - 22	Male	Student